iOS 开发入门手册

Swift 速成 · App 开发全流程 · 实战上手

依据《iOS 开发指北》（冰岩移动组 · 2023-09-16）扩充编写

面向零 Swift 基础的新成员

本手册适配环境：Xcode 27.0 · Swift 6.4 · iOS 27 SDK

目 录

第 0 章 如何使用这本手册 5

0.1 这本手册解决什么问题 5

0.2 手册结构导航 5

0.3 三种读法 6

0.4 阅读约定 6

第 1 章 起点：原《iOS 开发指北》要点 8

1.1 原《指北》说了什么 8

1.2 六件基础工具，逐件说明 8

1.3 技术栈怎么选：Swift、SwiftUI 与包管理器 9

1.4 学习资源清单 10

1.5 从第一天就该养成的三个习惯 11

第 2 章 Swift 速成（上）：看得懂的基础语法 13

2.1 先看一眼真实的 Swift 代码 13

2.2 常量与变量：let 与 var 14

2.3 基本数据类型与类型推断 15

2.4 字符串与字符串插值 16

2.5 集合类型：Array、Dictionary、Set 18

2.6 控制流：if、switch、for、while 20

2.7 函数：参数标签、默认值与返回值 22

2.8 可选值 Optional：Swift 最重要的概念 24

2.9 闭包 Closure：从语法到 @escaping 28

2.10 小结与自测 30

第 3 章 Swift 速成（下）：面向对象与进阶特性 32

3.1 枚举 enum 与关联值 32

3.2 结构体 struct 与类 class 34

3.3 属性：存储、计算与观察器 36

3.4 协议 protocol 与扩展 extension 38

3.5 泛型与 some、any.....39
3.6 错误处理：throws、try 与 do-catch.....41
3.7 属性包装器与 @ 标记家族.....43
3.8 内存管理：ARC、weak 与循环引用.....46
3.9 并发：async/await、Task 与 @MainActor.....48
3.10 模块与访问控制.....50
3.11 Swift 6 带来的变化.....51
3.12 符号速查表：看到这些符号怎么读.....52
3.13 标准库与常用 API 速查.....53
3.14 读代码练习（含答案）.....54
第 4 章 iOS App 开发基本过程（12 步）.....57
4.1 全景流程图.....57
4.2 第 1 步：准备开发环境.....58
4.3 第 2 步：创建第一个 Xcode 工程.....60
4.4 第 3 步：读懂工程目录结构.....62
4.5 第 4 步：理解 App 的骨架与生命周期.....64
4.6 第 5 步：SwiftUI 视图与布局.....66
4.7 第 6 步：状态管理.....69
4.8 第 7 步：网络请求与数据解析.....71
4.9 第 8 步：数据持久化.....74
4.10 第 9 步：引入第三方依赖.....76
4.11 第 10 步：调试与预览.....76
4.12 第 11 步：真机运行与签名.....78
4.13 第 12 步：打包、发布与 TestFlight.....79
4.14 工程规范与 Git 工作流.....81
4.15 常见报错与崩溃速查.....82
第 5 章 实战：从零做一个待办清单 App.....84
5.1 项目目标与最终效果.....84
5.2 第 1 步：创建工程与文件结构.....85
5.3 第 2 步：定义数据模型.....86

5.4 第 3 步：做出列表页面.....87
5.5 第 4 步：新增待办.....89
5.6 第 5 步：完成、删除与排序.....91
5.7 第 6 步：数据持久化.....94
5.8 第 7 步：界面美化与体验优化.....95
5.9 第 8 步：真机运行与打包.....98
5.10 从这个项目继续走下去.....99
第 6 章 学习路线与资源.....102
6.1 三周速成计划.....102
6.2 三个月进阶路线.....105
6.3 资源清单.....106
6.4 给新成员的学习建议.....109
第 7 章 附录.....111
附录 A 术语中英对照表.....111
附录 B 常见问题 FAQ.....115
附录 C Xcode 常用快捷键.....117
附录 D 新项目自查清单.....118

第 0 章 如何使用这本手册

0.1 这本手册解决什么问题

冰岩移动组的《iOS 开发指北》是一份很好的入组说明：它告诉你需要哪些工具、用哪些技术栈、去哪里找资料。但它是"指北"，不是"教程"——它没有教你 Swift 语法，也没有告诉你一个 App 从新建工程到上架中间要经过哪些步骤。

这本手册补上的正是这两块：

- 让你在很短时间内看懂 Swift 代码。目标不是"精通"，而是拿到项目里任何一个 .swift 文件，你能读懂它在做什么，遇到不认识的写法知道去哪里查。
- 让你知道开发一个 iOS App 的完整过程。从装 Xcode 开始，经过建工程、写界面、连网络、存数据、调试、真机，一直到打包上架，一共 12 步，每一步都有可执行的做法。

读完之后你应该能达到的状态：能独立做出一个能跑、能存数据、能联网的小 App，并且能参与冰岩现有 SwiftUI 项目的开发。

> 一句话概括学习策略：Swift 只学到"看得懂"为止，把省下来的时间花在动手写界面上。看懂和写出之间的距离，只能靠敲代码填平。

0.2 手册结构导航

章节	内容	你会得到什么	建议用时
第 0 章	使用说明	知道怎么读这本手册	10 分钟
第 1 章	原《指北》要点与补充	工具清单、技术栈选型、资源清单	30 分钟
第 2 章	Swift 基础语法	看懂变量、类型、集合、控制流、函数、可选值、闭包	1~2 天
第 3 章	Swift 进阶特性	看懂结构体、类、协议、泛型、并发、内存管理，以及一张符号速查表	2~3 天

章节	内容	你会得到什么	建议用时
第 4 章	App 开发基本过程	12 步全流程：环境、工程、界面、状态、网络、持久化、调试、签名、上架	2~3 天
第 5 章	待办清单实战	一份完整可运行的项目代码，以及它是怎么一步步长出来的	1~2 天
第 6 章	学习路线与资源	三周速成计划与三个月进阶路线	20 分钟
第 7 章	附录	术语对照表、FAQ、快捷键、检查清单	随时查

如果时间非常紧，只看第 2 章的 2.8 节（可选值）、第 3 章的 3.12 节（符号速查表）和第 4 章，就能开始跟着项目写代码。

0.3 三种读法

同一本手册，在不同阶段应该用不同方式读。

第一种是速成通读。适合刚入组、需要在一周内看懂项目代码的人。按第 2 章到第 5 章的顺序读一遍，代码全部亲手敲一遍，不要复制粘贴。遇到看不懂的细节先标记，继续往下读，读完一章再回头解决标记。

第二种是当字典查。适合已经开始写代码、卡在某个语法上的人。遇到看不懂的符号直接翻 3.12，遇到不认识的术语翻附录 A，遇到报错翻 4.15 和附录 B。这类查阅不需要从头读。

第三种是边做边查。适合已经开始做冰岩项目的人。先看第 5 章把完整项目跑通，然后做自己的需求，每写一行不确定的代码就回来查对应小节。这是效率最高的方式，也是最接近真实工作的方式。

0.4 阅读约定

本手册的排版有固定的约定，先说明清楚，后面不再重复解释。

- 正文中的等宽字体表示代码、标识符、文件名或命令，例如 `ContentView.swift`、`@State`。

- 灰色底纹的代码块都是可直接编译运行的 Swift 代码。代码里的中文注释解释的是"为什么这么写"，不是复述语法。
- 蓝色底纹的引用框是提示、结论或常见坑，值得单独记住。
- 手册全部代码以 SwiftUI 为主线，基线为 iOS 17 及以上，环境为 Xcode 27.0、Swift 6.4、iOS 27 SDK。如果你看到 iOS 17 之前的老写法，手册会明确指出它已经过时。
- 手册不教 Objective-C，也不把 UIKit 作为主线。你只会在对比说明时看到它们被提一句。

> 最重要的一条约定：不要只读不写。看代码产生的是"我懂了"的错觉，敲代码才会暴露"其实没懂"。每读完一节，至少把它里面的示例重新敲一遍并改几个参数看结果变化。

第 1 章 起点：原《iOS 开发指北》要点

1.1 原《指北》说了什么

《iOS 开发指北》全文只有两页，结构是四段：基础工具及使用、开发技术栈、学习资料、最后。它没有讲技术细节，讲的是"你该准备什么、该往哪个方向走"。

原文的四个结论，值得先完整记下来：

- 工具上，你需要 macOS 环境（Mac 或黑苹果）、Xcode、Git 与 GitHub、CocoaPods、访问部分境外技术站点所需的网络工具，以及 Stack Overflow、GitHub、Google、GPT 这类搜索工具。
- 语言上，Apple 系原生开发语言是 Swift 与 Objective-C，而 Swift 是 Apple 强推的新语言，具有很多高效现代的特性，Objective-C 已经被扔进历史的垃圾车里了。
- UI 框架上，有 SwiftUI 与 Storyboard 两条路。SwiftUI 是新一代响应式声明式 UI 框架，Storyboard 是传统的命令式 UI 框架；业界正在向 SwiftUI 迁移，冰岩的 iOS 项目全部使用 SwiftUI，将来也只会使用 SwiftUI。
- 包管理器上，冰岩的 iOS 项目主要使用 CocoaPods 与 Swift Package Manager。

原文还有一段关于学习方法的判断，是全文最有价值的部分：学习过程中你一定会遇到不了解的概念，在查询这些概念的过程中又会遇到新的概念，于是一直查一直查；当你的知识形成闭环之后，就说明你已经完全掌握这个领域了。期间可能有些概念看起来与 iOS 开发关联不大，不要犹豫，它们会作为计算机世界的基础设施去支持你未来的学习与工作。

这段话解释了一个新手最常见的困惑：为什么学 iOS 会学到计算机网络、内存管理、甚至操作系统原理。答案是这些知识不是跑题，它们是所有软件开发的共同底座。原文也明确鼓励你同时学点别的技术，比如 Python、Rust、HTML，你会在不同技术中发现计算机世界的基本原则与共性。

1.2 六件基础工具，逐件说明

原文列了六项基础工具，这里逐件补充说明它们的实际作用和获取方式。

macOS 环境是硬性前提。iOS 开发依赖 Xcode，而 Xcode 只能在 macOS 上运行。真机或虚拟机安装的黑苹果在原理上可行，但会带来驱动、稳定性、系统升级和 Xcode 版本兼容方面的持续麻烦，

通常不建议作为主力开发环境。如果预算有限，一台二手 Mac（哪怕是 Mac mini）就足够完成本手册的全部内容。

Xcode 是唯一的官方 IDE，包含编译器、调试器、界面预览、模拟器、性能分析工具和打包上传功能。它体积很大（几十 GB），安装后首次启动还需要下载额外组件，请预留足够的磁盘空间和一段完整的时间。你还需要它的命令行工具，装好后可以在终端里执行 `xcodebuild -version` 确认版本。

Git 与 GitHub 是所有协作的基础。冰岩的项目全部托管在 Git 仓库上，代码提交、分支、合并、Code Review 都依赖它。Git 本身不难，先用熟四条命令（`clone`、`pull`、`add` 加 `commit`、`push`）就能参与开发，剩下的边用边学。第 4.14 节会给出一份可直接使用的 .gitignore 和工作流约定。

CocoaPods 在原文写作时还是 iOS 第三方库管理的主流方案。需要更新一下现状：到今天，Swift Package Manager 已经是标准做法，Xcode 内置支持，不需要额外安装任何东西；CocoaPods 仍然能工作，大量老项目和老库还在用它，但它已经不再是新项目的首选。所以你的策略应该是：优先用 SPM，遇到只支持 CocoaPods 的库再学 CocoaPods。第 4.10 节有详细对比。

网络访问工具是现实需求而非可选项。Swift 和 SwiftUI 最权威的资料都是英文的，Apple 官方文档、WWDC 视频、GitHub 上的开源库、Stack Overflow 上的高质量回答，都要求你能稳定访问。请提前把这件事解决好，否则你的学习效率会被反复打断。

搜索工具决定了你解决问题的速度。原文列了 Stack Overflow、GitHub、Google 和 GPT。这里补充用法上的区别：具体报错信息优先用 Google 或 GPT 搜原句；"怎么写某个功能"优先看 GitHub 上的开源实现；"为什么会这样"优先看 Stack Overflow；Swift 和 SwiftUI 的 API 细节则应该先查官方文档，因为官方文档有 Xcode 里可以按 Cmd+Shift+0 打开的版本，能直接跳到签名和用法。

> 原文特别强调的一点，这里再重复一次：大部分官方或权威性的文章都是英文的，从现在起请熟悉并适应阅读全英文的文章。不论你是学习 iOS 还是其他方面都很有用。不要每篇文档都先机翻一遍，先从"看代码示例 + 猜大意"开始，一个月后你会发现英文文档比中文二手教程更好读。

1.3 技术栈怎么选：Swift、SwiftUI 与包管理器

原文给出了技术栈的三个选择，本手册完全沿用，并补充理由。

第一个选择是语言。Swift 是 Apple 在 2014 年推出的语言，语法现代，有可选值、类型推断、值类型、协议扩展、async/await 等特性，并且是开源的。Objective-C 是它的前任，语法陈旧、内存管理繁琐。结论很明确：只学 Swift。你不会写 Objective-C，但一定会读到它——很多老库的接口头文件还是 Objective-C 写的，读得懂方法签名就够了。

第二个选择是 UI 框架。这是新手最容易纠结的地方，所以要说透。Storyboard 和 UIKit 代表传统的命令式写法：你用代码一步步描述"创建这个控件、设置它的位置、把它加进去、更新它的文字"。SwiftUI 代表声明式写法：你描述"界面在某个状态下应该长什么样"，状态变了界面自动更新，位置和布局由框架计算。声明式写法的代码量通常是命令式的三分之一到一半，而且天然适合处理状态变化。

结论是：直接学 SwiftUI。冰岩的项目全部是 SwiftUI，业界新项目也在迁移。需要注意的现实情况是，招聘市场上仍有大量 UIKit 岗位，但如果你已经会 SwiftUI，补 UIKit 的成本远低于反过来；而你如果先学 UIKit，很可能在写完第一个界面之前就失去兴趣。

第三个选择是包管理器。Swift Package Manager（SPM）是 Apple 官方方案，直接集成在 Xcode 里，依赖声明写在 `Package.swift` 或 Xcode 的工程设置里，解析结果记录在 `Package.resolved`。CocoaPods 是社区方案，需要在项目里生成一个 `Podfile`，执行 `pod install` 后改用 `.xcworkspace` 打开工程，并且依赖 Ruby 环境。

对比项	Swift Package Manager	CocoaPods
是否需要额外安装	不需要，Xcode 内置	需要 Ruby 环境和 pod 命令
配置文件	Package.swift / 工程设置	Podfile
打开方式	直接用 .xcodeproj	必须用 .xcworkspace
生态现状	新库的标准做法	老库仍在使用
新手建议	优先使用	遇到必须用时再学

> 冰岩的现状是两者都有。你接手项目时，先看仓库根目录有没有 Podfile：有就用 CocoaPods 的方式打开，没有就正常打开工程文件。这一点搞错会导致"编译报错找不到模块"，是新人踩得最多的坑之一。

1.4 学习资源清单

原文列了五个资源，全部有效，这里按使用顺序重新排列，并补充几条。

官方文档永远排第一。Apple 开发者网站 developer.apple.com/develop 是入口；Swift 语言官方文档《The Swift Programming Language》在 docs.swift.org/swift-book，它同时有中文翻译版，但建议对照英文版读；SwiftUI 的 API 文档在 developer.apple.com/documentation/swiftui，查具体某个修饰符怎么用时最准。

斯坦福公开课 CS193p 是最值得投入的系统课程。它讲的就是用 SwiftUI 开发 iOS 应用，节奏紧凑、示例扎实，B 站有搬运版本（原《指北》给出的链接是 BV1EV411C77B）。建议在第 2 章和第 3 章读完、第 5 章的实战跑通之后再开始看，效果最好；如果一上来就看，会因为缺少语法基础而反复卡住。

菜鸟教程的 Swift 部分（runoob.com/swift）适合当快速语法字典，用来查单个语法点，不适合作为系统学习的主线。它的深度不足以覆盖 SwiftUI 和工程实践。

需要补充的资源有三类。Human Interface Guidelines（Apple 官方的人机界面指南）决定你的 App 看起来是否"像个 iOS 应用"，做界面之前值得翻一遍。WWDC 视频是每年新特性的第一手资料，遇到"这个 API 是什么时候引入的、为什么这么设计"这类问题时最好用。SwiftLint 是代码规范检查工具，冰岩的项目通常会配置它，装好之后它会在你写代码时直接提示不符合规范的地方。

1.5 从第一天就该养成的三个习惯

原文最后一节讲的是编程习惯，这部分比任何技术细节都重要，因为习惯一旦形成就很难改。原文提到的要点整理为三条。

第一条是代码风格与注释。原文说，如果你决定做一名 coder 或 programmer，请从现在起养成好的编程习惯，包括代码风格，以及在复杂或重要的部分添加注释；每学习一门编程语言，都要多了解这门语言的代码规范，比如 Swift 有 SwiftLint 来帮助你保持一个良好的代码规范。具体到执行层面：变量名和函数名用完整的英文单词而不是缩写；保持一致的空格和换行；注释只写在"为什么这么做"的地方，比如一段看起来多余的判断，而不是给每一行都写"这是赋值"。

第二条是拥抱英文资料。这是原文"基础工具"一节里最重的一段话，也是新人最大的分水岭。中文二手教程的更新速度永远落后于 Apple 的版本迭代，你在中文资料里找到的写法很可能已经是两年前的过时方案（例如本书会反复提醒的 `ObservableObject` 与 `NavigationView`）。英文官方文档永远是第一手且最新的。

第三条是把细节做对。原文最后有一个 PS：细节是非常重要的，所以希望大家能把自己名字备注中的 ios 改成 iOS。这句话看起来像玩笑，实际是在说工程态度——命名的大小写、大小写敏感的路径、时区、单位、边界条件，这些"小事"在软件开发里造成的事故远比语法错误多。从现在起就按正确的写法写：iOS 而不是 ios，GitHub 而不是 Github，SwiftUI 而不是 Swiftui。

> 补充第四条：遇到问题先自己查十五分钟。查的顺序是——看完整报错信息、看官方文档、用原句搜索、再问人。问人的时候带上"我想做什么、我试了什么、完整的报错是什么"，这条习惯会让你在冰岩拿到远超平均水平的帮助。

第 2 章 Swift 速成（上）：看得懂的基础语法

2.1 先看一眼真实的 Swift 代码

先看一段真实项目里会出现的代码：一个带计数按钮的界面。

```
import SwiftUI

@Observable
final class CounterModel {
    var count = 0
    func increase() { count += 1 }
}

struct CounterView: View {
    @State private var model = CounterModel()
    var body: some View {
        VStack(spacing: 16) {
            Text("当前计数：\(model.count)")
                .font(.largeTitle)
            Button("加一") {
                model.increase()
            }
            .buttonStyle(.borderedProminent)
        }
        .padding()
    }
}

#Preview { CounterView() }
```

这段代码一行一行拆开看：

代码	这行在干什么
`import SwiftUI`	引入 SwiftUI 框架，后面才能用 View、Text、Button 这些类型
`@Observable`	让这个类的属性变化能被界面自动感知，属于状态管理，第 3 章展开
`final class CounterModel {`	定义一个类，final 表示不允许被继承，现在只要知道它是"装数据和逻辑的盒子"
`var count = 0`	一个可变的整数属性，初始值是 0
`func increase() { count += 1 }`	定义一个没有参数、没有返回值的函数，把 count 加一
`struct CounterView: View {`	定义一个结构体，并声明它遵循 View 协议，SwiftUI 的界面都是 View
`@State private var model = CounterModel()`	声明界面自己的状态，private 表示只在这个类型内部可见
`var body: some View {`	View 协议要求的计算属性，描述"界面长什么样"

代码	这行在干什么
`VStack(spacing: 16) {`	竖向排列的容器，子视图之间间隔 16 点
`Text("当前计数：\(model.count)")`	显示一行文字，反斜杠括号把 count 的值插进字符串
`.font(.largeTitle)`	修饰符，把字体设成大标题
`Button("加一") {`	一个按钮，标题是"加一"，后面的大括号是点击后执行的闭包
`model.increase()`	调用上面定义的函数，让 count 加一
`.buttonStyle(.borderedProminent)`	按钮样式，填充式的高亮按钮
`.padding()`	给整个 VStack 留出内边距
`#Preview { CounterView() }`	让 Xcode 的预览面板渲染这个界面

这张表里还剩几处没解释的东西：`class`、`struct`、协议、属性包装器、修饰符，它们属于第 3 章。除此之外的每一行——变量、类型推断、字符串插值、点语法、闭包——都应该在这一章结束时变得理所当然。

顺便给一个读陌生 Swift 文件的通用顺序，本章之后你都可以照着做：先看这个文件声明了哪些类型和函数，知道它整体负责什么；再从 `body` 或入口函数开始，顺着数据的流向往下读；遇到不认识的符号，先判断它是语言关键字、框架 API，还是项目自定义的名字，这三类东西的查法完全不同。有了固定顺序，一段陌生的代码通常几分钟内就能变得可读。

> 本章的目标只有一个：读完这十节，你再回头看上面那段代码，能说出每一行在做什么，而不是"大概是在写界面"。看懂是写出代码的前提，这一章不要求你能默写语法，只要求你能读。

2.2 常量与变量：let 与 var

一句话结论：`let` 声明常量，赋值后不能再改；`var` 声明变量，随时可以改。Swift 里默认应该用 `let`。

```
let name = "小明"       // 常量：这个名字以后不会变
var score = 60          // 变量：分数后面还要改
score = 80              // 可以重新赋值
// name = "小红"        // 编译错误：let 声明的值不能再赋值
```

怎么读：`let` 和 `var` 写在名字前面，类型可以省略，编译器会推断。被 `//` 注释掉的那行如果真写出来，Xcode 会立刻报错 "cannot assign to value: 'name' is a 'let' constant"。Swift 把"这个值会不会变"变成了编译期就能检查的事实。

为什么默认用 let：一是意图清楚，读代码的人看到 let 就知道这个值在作用域内是稳定的，不需要来回找哪里改过它；二是编译器能据此做优化；三是它天然避免了并发场景下的数据竞争（第 3 章会讲 Sendable 时再展开）。

类型推断与显式标注：

```
let a = 10                  // 推断为 Int
let b: Double = 10          // 显式标注为 Double，字面量 10 被当成 10.0
var c: String               // 只声明不赋值，稍后再给值
c = "hello"
```

怎么读：冒号后面写类型就是显式标注。var c: String 这种"先声明后赋值"的写法在需要提前定义属性时很常见。注意 let b: Double = 10 合法，因为整数字面量能直接变成 Double；但如果右边是一个已经确定类型的 Int 变量，就必须显式转换，见 2.3。

> 提示：Swift 没有"已声明但未赋值的变量"这种中间状态，也没有 C 里那种随机初值。常量用之前必须有值，编译器会检查所有分支是否都赋过值。

命名规范：类型、协议用大驼峰，例如 CounterModel、View；变量、常量、函数、参数、枚举成员用小驼峰，例如 count、increase、buttonStyle。不要用全大写表示常量（那是 C 的习惯），不要用拼音，也不要用 a1、tmp2 这种名字。项目里出现 userName、isLoading、maxRetryCount 这类能读出意思的名字，说明作者遵守了规范。

2.3 基本数据类型与类型推断

一句话结论：日常最常用的基本类型是 Int、Double、String、Bool、Character，它们都是值类型，名字都是大驼峰。

```
let age = 20                    // Int
let height = 1.75               // Double，带小数点就是 Double
let letter: Character = "A"     // Character，单个字符
let isReady = true              // Bool
let pi = 3.14159                // Double
```

怎么读：Int 是整数，默认 64 位，溢出会导致运行时报错而不是像 C 那样悄悄回绕。Double 是双精度浮点。String 是字符串。Bool 只有 true 和 false 两个值。Character 是单个字符，用双引号写，只是类型标注不同——Swift 不分单引号和双引号。

类型转换必须显式写：

```
let a = 3
let b = 1.5
// let c = a + b            // 编译错误：Int 和 Double 不能直接相加
let c = Double(a) + b       // 4.5，必须先把 Int 转成 Double
let d = Int(b)              // 1，直接截断小数部分
let text = String(a)        // "3"
```

怎么读：C 里写 `3 + 1.5`，编译器会悄悄把 3 提升成 double；Swift 认为这种隐式转换会掩盖精度损失，一律拒绝，必须写成 `Double(a) + b`。所以你在项目里会大量看到 `Int(...)`、`Double(...)`、`String(...)` 这种"类型名当函数用"的写法。注意 `Int(1.9)` 得到的是 1，是向零截断，不是四舍五入。

元组用来临时打包几个值：

```
let point = (x: 10, y: 20)
print(point.x)              // 10，用名字取
let (a, b) = point          // 解构成两个常量
let (_, onlyY) = point      // 下划线表示"这个位置我不关心"
```

怎么读：元组适合"函数一次返回多个值"这种临时场景。用名字定义的元组可以按名字访问，也可以整体解构。它不适合当数据模型——需要一个有名字、有方法的类型时应该用结构体（第 3 章）。

`typealias` 是给类型起别名：

```
typealias UserID = Int
let id: UserID = 42         // 读代码时要知道 UserID 其实就是 Int
```

一句结论：看到不认识的类型名，先找 `typealias`，它多半只是某个系统类型的别名，用来让代码表达更清楚。

补充两个容易踩的点。第一，`Bool` 和整数没有关系：C 里写 `if (1)` 合法，Swift 里必须写 `if count > 0`，反过来 `let x: Int = true` 也不合法，两者之间不存在隐式转换。第二，`Character` 和长度为 1 的 `String` 不是同一个类型，`"A"` 默认被推断成 `String`，只有明确标注了 `Character`，或者被 `for` 循环从字符串里取出来时，它才是字符类型。

2.4 字符串与字符串插值

一句话结论：Swift 用 `String` 表示文本，把值塞进字符串的标准写法是插值，语法是反斜杠加一对圆括号。

```
let name = "小明"
let age = 20
let message = "\(name) 今年 \(age) 岁"
let total = "合计：\(3 + 4) 元"      // 括号里可以放任意表达式
```

怎么读：`\( )` 里面的内容会被求值，然后转换成文本拼进字符串。它比用 `+` 拼接更清楚，也不需要对数字做 `String(...)` 转换。项目里的日志、界面文字、网络参数几乎都用它。

多行字符串用三个双引号：

```
let json = """
{
  "name": "小明",
  "age": 20
}
"""
```

怎么读：结尾的 `"""` 决定缩进基准，所以上面这段内容不会被代码缩进污染，JSON 里的双引号也不需要转义。写测试数据、拼 SQL、写长文案时很常用。

常用字符串方法：

```
let s = "hello world"
s.count                                          // 11，字符个数
s.isEmpty                                        // false
s.hasPrefix("hello")                             // true
s.uppercased()                                   // "HELLO WORLD"
s.contains("world")                              // true
s.replacingOccurrences(of: "world", with: "swift")
let parts = s.split(separator: " ")              // ["hello", "world"]
parts.joined(separator: "-")                     // "hello-world"
```

怎么读：`count` 数的是"人能看到的字符"而不是字节数，所以 `"你好".count` 是 2。这些方法都返回新字符串，原字符串不变（`String` 是值类型，下一段解释）。`split` 返回的是 `Substring` 数组，需要真正的 `String` 时写 `String(parts[0])`；`joined` 反过来把字符串片段拼成一整串。

字符串是值类型：

```
var a = "hi"
var b = a
b += "!"
print(a)        // 还是 "hi"，改 b 不影响 a
```

怎么读：`b = a` 是复制，不是让两个名字指向同一块内存。Swift 的 `String`、`Array`、`Dictionary`、`Set` 都是这种值类型（底层是结构体）。这和你可能熟悉的 Java、Python 里"赋值即共享引用"的行为不同，看到"改副本不影响原件"不要觉得奇怪。

字符串还可以直接比较和逐字符遍历：

```
"abc" == "abc"          // true，比较的是内容
"abc" < "abd"           // true，按字典序比较
for c in "你好" {        // 每个 c 是 Character
    print(c)
}
```

怎么读：因为是值类型，`==` 比较的就是文本内容本身，不需要像 Java 那样调用 equals 方法。`for c in s` 拿到的是一个个 `Character`，想拼回字符串要写 `String(c)`。这也解释了为什么 `count` 是按字符数而不是字节数计算。

字符串里可以写转义字符：`\n` 是换行，`\t` 是制表，`\"` 表示一个双引号本身。当文本里反斜杠本来就很多（例如正则表达式、文件路径），可以在双引号前加一个井号写成 `#"..."#`，这种写法里反斜杠不再是转义符，插值则要写成 `\#(...)` 的形式。读代码时看到井号包围的字符串，知道它是"原样字符串"即可，不要当成语法错误。

2.5 集合类型：Array、Dictionary、Set

一句话结论：Swift 有三种常用集合，都是值类型，都要求元素类型一致。Array 有序可重复，Dictionary 按键查值，Set 无序不重复。

Array 是最常用的：

```
var fruits = ["苹果", "香蕉"]
fruits.append("橘子")          // 尾部追加
fruits.count                    // 3
fruits[0]                       // "苹果"，下标从 0 开始
fruits[0] = "红苹果"            // 改
let removed = fruits.remove(at: 1)   // 删除并返回被删的元素
fruits.contains("橘子")         // true
let empty: [String] = []        // 空数组必须写元素类型
```

Dictionary 按键值对存：

```
var ages = ["小明": 20, "小红": 21]
ages["小刚"] = 19               // 键不存在就是新增
ages["小明"] = 21               // 键存在就是覆盖
let ming = ages["小明"]         // 类型是 Int?，因为键可能不存在
let gone = ages.removeValue(forKey: "小红")
for (name, age) in ages { print("\(name) \(age)") }
```

Set 自动去重：

```
var tags: Set<String> = ["swift", "ios", "swift"]   // 只剩两个元素
tags.insert("mobile")
tags.contains("ios")            // true
tags.remove("swift")
```

怎么读：Set 没有 `[]` 这种简写字面量，必须写全集合类型和尖括号里的元素类型，否则编译器会把它当成数组。凡是"只关心某个东西在不在"的场景就该用 Set，`contains` 判断元素是否存在时比数组快得多，因为数组要逐个比较。

遍历和高阶方法：

```
let nums = [1, 2, 3, 4]
for (index, value) in nums.enumerated() { print(index, value) }

nums.map { $0 * 2 }              // [2, 4, 6, 8]，逐个转换
nums.filter { $0 % 2 == 0 }      // [2, 4]，筛选
nums.reduce(0) { $0 + $1 }       // 10，把整个数组压缩成一个值
nums.sorted { $0 > $1 }          // [4, 3, 2, 1]
nums.first { $0 > 2 }            // 3，注意返回的是 Int?
nums.contains(3)                 // true
```

怎么读：`map` 是"每个元素都变成另一个"，`filter` 是"留下满足条件的"，`reduce` 是"从头累积成一个值"，`sorted` 返回新数组（原来的 `nums` 不变，因为数组是值类型）。这几个方法是 Swift 项目里出现频率最高的代码模式，读的时候先看闭包的参数，就知道它在处理什么。

`compactMap` 是转换和筛选的合体，专门用来处理"转换结果可能是可选值"的情况，它会顺手把 nil 丢掉：

```
let inputs = ["1", "abc", "3"]
let numbers = inputs.compactMap { Int($0) }   // [1, 3]，"abc" 转不成整数被丢弃
```

怎么读：`Int("abc")` 的返回值是 `Int?`，因为字符串不一定能转成整数。如果这里用 `map`，你会得到 `[Int?]`（一个装着可选值的数组，很难用）；`compactMap` 则直接给出 `[Int]`。解析用户输入、处理 JSON 里的字符串数组时经常见到它。

还有一个和数组反直觉的差别：Dictionary 的遍历顺序是不确定的。即使你按某个顺序插入，`for (name, age) in ages` 每次运行的顺序也可能不同，因为字典里没有"第几个元素"这个概念。需要固定顺序就先 `ages.keys.sorted()` 再遍历；需要按下标取值，就应该用数组而不是字典。

集合	是否有序	元素能否重复	适合的场景
Array	有序，靠下标访问	可以重复	列表数据、界面渲染的数据源
Dictionary	无序，靠键访问	键唯一，值可重复	按名字或 id 查数据、JSON 对象
Set	无序	不重复	去重、判断存在性、集合运算

两个必须记住的坑。

第一个是下标越界会直接崩溃：

```
let arr = [1, 2, 3]
// arr[3]                                  // 运行时崩溃，Swift 没有可捕获的越界异常
if arr.indices.contains(3) { /* 先判断再访问 */ }
arr.first        // 1，返回 Int?
arr.last         // 3，返回 Int?
```

怎么读：Swift 数组越界不像 Java 那样抛出可以 `catch` 的异常，而是直接终止程序。所以要么先判断 `indices.contains(i)`，要么用 `first`、`last` 这类安全 API。这也是为什么 Swift 代码里很少看到手写下标循环。

第二个是"查不到"是正常情况，所以这类 API 返回可选值：`ages["小明"]`、`arr.first`、`arr.first(where:)`、`dict.removeValue(forKey:)` 全都是可选值。原因很直接——"这个键没有对应的值"不是错误，而是业务上的常见情形。怎么安全地处理它们，是下一节的全部内容。

> 提示：可变集合用 `var` 声明，不可变用 `let`。`let fruits = ["苹果"]` 之后不能 `append`，和 C++ 的 const 容器同理，编译器会拦住你。

2.6 控制流：if、switch、for、while

一句话结论：Swift 的控制流关键字和你学过的语言几乎一样，但多出两个硬性规矩：条件必须是 `Bool`，`switch` 必须穷尽。

if 和 else：

```
let score = 85
if score >= 90 {
    print("优秀")
} else if score >= 60 {
    print("及格")
} else {
    print("不及格")
}
```

怎么读：条件不需要圆括号，但花括号必须写，即使只有一行。这里和 C 最大的区别是条件必须是 `Bool`：`if score` 或 `if somePointer` 都不合法，`0`、`nil`、空字符串都不会被当成假。想表达"非空"就要写清楚，例如 `if !name.isEmpty`。

三目运算符和其他语言一样：

```
let label = score >= 60 ? "及格" : "不及格"
```

switch 必须穷尽，而且不用写 `break`：

```
let grade = "B"
switch grade {
case "A", "B":
    print("不错")          // 多个值用逗号并列
case "C" where score > 80:
    print("勉强")          // where 追加额外条件
default:
    print("其他")          // 必须覆盖所有剩余情况
}
```

怎么读：Swift 的 `case` 命中后执行完就自动结束，不会像 C 那样"贯穿"到下一个 case，所以不需要 `break`。反过来，编译器要求所有可能的值都被覆盖：字符串有无穷多种取值，所以必须写 `default`；如果是对枚举做 switch，写全所有成员就不用 `default`（第 3 章讲）。确实需要贯穿时用 `fallthrough` 关键字显式声明，实际项目里极少见。

范围匹配和值绑定是 switch 的两大利器：

```
switch score {
case 90...:
    print("优秀")          // 闭区间到正无穷
case 60..<90:
    print("及格")
default:
    print("不及格")
}

switch (statusCode, retryCount) {
case (200, _):
    print("成功")
case (let code, 0):
    print("错误码 \(code)，且不再重试")   // let 把匹配到的值绑定成常量
case (let code, let times):
    print("错误码 \(code)，重试 \(times) 次")
}
```

怎么读：`90...` 是省略了上界的闭区间。`case (let code, 0)` 表示同时匹配两个值，第一个绑定给 `code`，第二个必须是 0。这种"元组加值绑定"的写法在解析网络响应、处理状态组合时很常见，它能把一长串 `if` 压成一个表格状的 switch。

for-in 遍历：

```
for i in 0..<3 { print(i) }        // 0 1 2，半开区间，不含 3
for i in 1...3 { print(i) }        // 1 2 3，闭区间
for i in stride(from: 0, to: 10, by: 2) { print(i) }   // 0 2 4 6 8
for (index, fruit) in fruits.enumerated() { print(index, fruit) }
for _ in 0..<3 { print("三次") }    // 不关心循环变量时用下划线
```

怎么读：`1...3` 是闭区间，两头都含；`0..<3` 是半开区间，不含右端。写代码时优先用 `..<`，因为它能自然表示空区间，也不会在算边界时多算一个。遍历数组直接用 `for x in arr`，需要下标时才用 `enumerated()`。

遍历时还可以顺手加过滤条件，嵌套循环可以用标签指定跳出哪一层：

```
for value in nums where value > 2 { print(value) }    // 3 4
outer: for i in 1...3 {
    for j in 1...3 {
        if j == 2 { continue outer }   // 带标签，直接跳到外层循环的下一轮
        print(i, j)
    }
}
```

怎么读：`for ... where` 相当于在循环体开头写一个 if，不过意图更清楚。`outer:` 是给循环起标签，`break outer` 和 `continue outer` 能指定作用在哪一层，比在每层加标志变量干净得多。项目里标签用得不多，但读到要能认出来。

顺带记住 `break` 和 `continue` 的区别：`break` 立刻结束整个循环，`continue` 结束本轮、直接开始下一轮。它们在 `switch` 里还有一个容易误读的用法：`case "X": break` 是"这一条什么都不做"。Swift 的 case 不允许空着，所以需要显式跳过某个值时就得写这个 `break`，它不是多余的代码。

while 和 repeat-while：

```
var n = 3
while n > 0 { n -= 1 }              // 先判断再执行，可能一次都不执行
repeat { n += 1 } while n < 3        // 先执行一次再判断，相当于其他语言的 do-while
```

guard 提前退出（这里只讲基础语法，第 3 章不再重复）：

```
func greet(name: String?) {
    guard let name else {
        print("没有名字")
        return                          // 条件不成立就必须离开当前作用域
    }
    print("你好，\(name)")               // 这里 name 已经是 String
}
```

怎么读：`guard` 的语义是"守门"——条件不成立就立刻退出。所以 `else` 分支里必须写 `return`、`break`、`continue` 或 `throw` 之一。它比 `if let` 好在解包得到的变量在 guard 之后的整个作用域都有效，而 `if let` 只在花括号内有效。项目里的函数经常开头上演三四行 `guard`，把不合法的情况一次性挡掉，剩下的主体代码就不用层层缩进。这就是 Swift 代码"先排除异常、再处理正常逻辑"的典型结构。

2.7 函数：参数标签、默认值与返回值

一句话结论：Swift 函数用 `func` 声明，参数默认带"外部标签"，调用点因此读起来像一句英文。

```
func area(width: Int, height: Int) -> Int {
    return width * height
}

area(width: 3, height: 4)      // 调用时必须写标签
```

怎么读：`width` 既是外部标签也是内部参数名。调用点写成 `area(width: 3, height: 4)`，一眼能看出每个数字的含义，这是 Swift 强制标签的目的。返回类型写在箭头后面，没有返回值就整个省略箭头。

外部名和内部名可以分开写，也可以不要标签：

```
func move(from start: Int, to end: Int) { }   // from 是外部标签，start 是内部名
move(from: 0, to: 10)

func double(_ value: Int) -> Int { value * 2 }   // 下划线表示调用时不写标签
double(5)
```

怎么读：`from start: Int` 里，`from` 给调用点看，`start` 给函数体用。多数情况下两者同名（只写一次 `width` 就相当于同名）。一个参数名前面加下划线 `_`，意思是"这个标签省略"，通常在语义已经很清楚时使用，例如 `double(5)`、`sum(1, 2, 3)`。

默认参数、可变参数和省略 return：

```
func greet(name: String = "同学") -> String {
    "你好，\(name)"                 // 函数体只有一个表达式时可以省略 return
}
greet()                             // "你好，同学"
greet(name: "小明")

func sum(_ numbers: Int...) -> Int {
    numbers.reduce(0, +)            // 函数体内 numbers 是 [Int]
}
sum(1, 2, 3)                        // 6
```

返回元组可以一次给出多个值：

```
func divide(_ a: Int, by b: Int) -> (quotient: Int, remainder: Int) {
    (a / b, a % b)
}
let result = divide(7, by: 2)
result.quotient      // 3
result.remainder     // 1
```

怎么读：返回类型写成带名字的元组，调用者就能用 `.quotient`、`.remainder` 取值。适合返回"两三个强相关的值"，需要更复杂的结构时应该定义结构体（第 3 章）。

`inout` 让函数能修改传入的变量：

```
func addOne(to value: inout Int) {
    value += 1
}
var n = 1
addOne(to: &n)          // 调用点必须写 &，提醒"这个变量会被改"
print(n)                // 2
```

怎么读：Swift 的参数默认是传值，函数里改形参不会影响外面的变量。确实需要修改原变量时才加 `inout`，调用点必须写 `&`。这个符号是给读代码的人看的警告：这个值会被函数改掉。项目里 `inout` 用得不多，遇到时知道含义就够了。

函数是一等公民，可以当参数传，也可以当返回值：

```
func double(_ x: Int) -> Int { x * 2 }

func apply(_ f: (Int) -> Int, to value: Int) -> Int { f(value) }
apply(double, to: 3)              // 6

func makeAdder(_ base: Int) -> (Int) -> Int {
    { $0 + base }                 // 返回一个闭包
}
let add10 = makeAdder(10)
add10(5)                          // 15
```

怎么读：`(Int) -> Int` 是函数类型，读作"接收一个 Int、返回一个 Int 的函数"。任何满足这个签名的函数都能传进去，连匿名写法的闭包也能。项目里的 API 大量接收这种参数——排序规则、点击事件、请求完成回调都是，下一节专门讲。

拿到一个没见过的函数签名，可以按固定顺序拆开读：先看 `func` 后面的函数名，再从左往右逐个看参数（冒号左边是调用时要写的标签，右边是类型，有等号说明有默认值），最后看箭头右边是返回类型。例如 `func request(_ url: URL, method: String = "GET") async throws -> [Item]`，读出来的意思是：第一个参数调用时不写标签，第二个参数有默认值可以省略，函数是异步的、可能失败抛错，成功时返回一个数组。这套拆法在本章够用，第 3 章会补上 `async` 和 `throws` 的细节。

2.8 可选值 Optional：Swift 最重要的概念

一句话结论：可选值表示"这个位置可能有值，也可能是 `nil`"，类型写成 `String?`。编译器会强制你在使用前处理"没有值"的情况，这是 Swift 和 C、Java 最大的体验差别。

```
var nickname: String? = "小明"     // 有值
nickname = nil                    // 没值
// let count = nickname.count     // 编译错误：可选值不能直接当 String 用
```

怎么读：`String?` 读作"可能是 String，也可能是 nil"。`nil` 表示"没有值"，它不等于空字符串 `""`，也不等于 0。`nickname` 是可选值时，你不能直接调用 `count`、不能直接拼进字符串、不能直接和别的字符串比较——必须先把里面的值取出来。语法上，问号只是简写，完整的类型名是 `Optional`，而 `Optional` 本质是一个枚举，只有"有值"和"没有值"两种情况（枚举在第 3 章讲，这里知道它是语法糖即可）。

为什么要有可选值：它把"这个值可能不存在"写进了类型里。函数签名 `func findUser(id: Int) -> User?` 明确告诉你"可能找不到"，调用者不看文档也知道要处理。C 里指针可能是 NULL、Java

里对象可能是 null、Python 里可能是 None，这些都得靠注释和约定，忘了判断就在运行时崩溃。Swift 把这些检查提前到编译期。

强制解包 `!` 和它的危险：

```
var nickname: String? = "小明"
let name = nickname!          // ! 的意思是"我保证这里有值，直接取出来"
print(name.count)             // 5

var empty: String? = nil
// print(empty!.count)        // 运行时崩溃：unexpectedly found nil
```

怎么读：`!` 是"强制解包"，它把 `String?` 变成 `String`，代价是把安全责任全部转移到你身上。运行时如果它正好是 nil，程序立刻崩溃，没有异常可以捕获，也没有补救机会。看到项目里出现 `!`，你要判断两种可能：作者确实确信这里不可能为空；或者只是懒得处理。初学阶段把 `!` 当成危险操作，能不用就不用。

安全解包的首选写法是 `if let`：

```
let nickname: String? = "小明"

if let nickname {                 // Swift 5.7 起可以这样简写
    print(nickname.count)         // 这里 nickname 是 String，不是 String?
} else {
    print("没有昵称")
}

if let name = user.nickname {     // 需要换个名字时写全
    print("你好，\(name)")
}
```

怎么读：`if let nickname` 是 `if let nickname = nickname` 的简写，意思都是"如果右边的可选值不是 nil，就把它取出来赋给一个同名的常量，然后走 if 分支；否则走 else"。关键点是：花括号内部的 `nickname` 是解包后的普通 `String`，出了花括号它还是原来的 `String?`。右边的表达式可以任意复杂，例如 `if let first = items.first`。

`guard let` 是同一个东西的另一半：

```
func printName(_ nickname: String?) {
    guard let nickname else { return }
    print(nickname)               // 从这里到函数结束都能用
}
```

怎么读：`guard let` 要求"必须成功"，失败就在 `else` 里离开当前作用域。它和 `if let` 的区别是解包结果的存活范围：`guard let` 之后整个作用域都能用，`if let` 只在花括号内可用。所以函数的开头通常是一串 `guard`，把参数不合格、数据为空的情况全部挡掉，剩下的主体代码就能直接用非可选值——这是 Swift 项目里最常见的代码形状。

空合运算符 `??` 用来给默认值：

```
let name: String? = nil
let display = name ?? "游客"          // name 有值就用它，是 nil 就用 "游客"
let length = (name ?? "游客").count   // 3
let first = items.first ?? "无数据"
```

怎么读：`a ?? b` 读作"a 有值就用 a，否则用 b"。结果是解包后的普通类型，不再是可选值。注意 `??` 右边的类型必须和左边解包后的类型一致，所以 `let x: Int? = 1; x ?? "空"` 是不合法的。给可选值提供默认展示文案、默认计数、默认值时，这是最短的写法。

可选链 `?.` 用来连续访问：

```
let text: String? = "hello"
let upper = text?.uppercased()        // "HELLO"；如果 text 是 nil，结果就是 nil
let count = text?.count ?? 0          // 用 ?? 兜底成 Int
let head = users.first?.name?.first   // 任意一环是 nil，整体就是 nil
```

怎么读：`?.` 的意思是"如果前面是 nil，后面整个表达式直接得到 nil，方法根本不会被调用"。所以可选链的结果一定是可选值，通常还要再配一个 `??` 才能得到普通值。可以连着写很多层，任意一环断了就短路。读代码时看到 `?.`，你的第一反应应该是"这个表达式的结果可能是 nil"。

隐式解包可选值（Implicitly Unwrapped Optional，IUO）是 `Type!` 这种写法：

```
var cached: [String]! = nil       // 声明成"可以放 nil，但用的时候自动解包"
cached = ["a", "b"]
print(cached.count)                // 2，不用写 !
// 如果上面忘了赋值，这一行就崩溃
```

怎么读：`[String]!` 和 `[String]?` 在存储上没有区别，差别只在使用时：IUO 会自动帮你加 `!`。所以它保留了可选值的全部危险，还把这层危险藏了起来，让读代码的人以为它一定有值。日常写代码应该用 `?`；你会遇到 `!` 的主要场合是历史代码，例如 UIKit 时代的 `@IBOutlet var titleLabel: UILabel!`，属性在初始化时确实还没有值，只能在稍后赋值。同一个家族里还有 `try!` 和 `as!`，含义都是"我保证不会失败，失败就崩溃"，待遇相同：看到就多留一分警惕。

可选值上的 `map` 和 `flatMap`：

```
let input: String? = "42"
let length = input.map { $0.count }        // Int?，有值才算，nil 直接是 nil
let number = input.flatMap { Int($0) }     // Int?，压平一层
let doubled = input.map { Int($0) ?? 0 }   // Int?
```

怎么读：可选值的 `map` 意思是"如果有值，就对它做点什么；如果是 nil，就原样返回 nil"，全程不会崩溃，适合把"可选值变换成另一种可选值"。`flatMap` 用在闭包自己也会返回可选值的时候，否则

`input.map { Int($0) }` 会得到 `Int??` 这种两层可选值。日常最常用的还是 `if let` 和 `??`，`map` 只是让链式处理更紧凑的写法。

可选值在实际项目里几乎只从四类地方冒出来，记住这四类，读代码时就有了一半的警惕心：字典按下标取值（键可能不存在）、`first` 和 `first(where:)`（可能没有满足条件的元素）、类型转换（`Int("abc")` 可能转不了）、以及各种外部 API 的返回值（网络请求会失败、查询会查不到）。看到这几类调用，就要按可选值来处理返回值，不要想当然地认为它一定有值。

至于该选哪种解包方式，判断标准很直接：需要分别处理"有"和"没有"两个分支就用 `if let`；需要提前挡掉异常、后面继续用这个值就用 `guard let`；只是想要一个默认值就用 `??`；只是要链式访问一下就用 `?.`；要在有值时做一次变换就用 `map`。而 `!` 不该出现在新写下的代码里。

多个可选值可以一次绑定：

```
func welcome(name: String?, age: Int?) -> String {
    if let name, let age, !name.isEmpty {
        return "\(name)，\(age) 岁"     // 三个条件全满足才进来
    }
    return "信息不全"
}
```

怎么读：多个 `let` 用逗号连起来，任意一个是 nil 就整体不成立；逗号后面还可以直接跟布尔条件，等价于用 `&&` 串起来。这里 `name` 和 `age` 在花括号里已经是普通类型了。处理多个网络返回参数时这种写法很常见。

最后是判断 `?` 和 `!` 的速查表：

写法	含义	运行时行为	你该怎么做
`String?`	这个值可能是 nil	安全，但用不了	先解包再使用
`value!`	我保证这里有值	是 nil 就崩溃	只在绝对确定时用
`if let x` 或 `guard let x`	取出非 nil 的值	安全	首选写法
`a ?? b`	有值用 a，否则用 b	安全	需要默认值时首选
`a?.b`	链式访问，前面 nil 则整体 nil	安全，结果是可选值	记得处理返回值
`Type!` 声明	隐式解包，使用时自动加 !	是 nil 就崩溃	老代码里遇到，新代码避免

> 一句话记住这一节：`?` 是"可能没有，请处理"，`!` 是"我认为一定有，出事我负责"。项目里绝大多数崩溃都和 `!` 有关，读代码时先把所有 `!` 找出来，你就找到了风险点。

2.9 闭包 Closure：从语法到 @escaping

一句话结论：闭包就是一段可以传来传去的代码块，写法是花括号包起来，参数和函数体之间用 `in` 分隔。

完整的闭包表达式长这样：

```
let add: (Int, Int) -> Int = { (a: Int, b: Int) -> Int in
    return a + b
}
print(add(1, 2))       // 3
```

怎么读：`{ 参数列表 -> 返回类型 in 函数体 }`，`in` 是分隔符，前面是签名，后面是代码。开头的 `(Int, Int) -> Int` 是变量类型标注，闭包本身的类型由上下文推断，所以这两处经常只写一处。

简化是一步步来的，同一个闭包的五种写法如下：

```
let nums = [3, 1, 2]
// 1. 完整写法：参数名、参数类型、返回类型、return 全都写出来
let a = nums.sorted(by: { (x: Int, y: Int) -> Bool in return x < y })
// 2. 省略参数类型：编译器能从 sorted 的签名推断出来
let b = nums.sorted(by: { x, y -> Bool in return x < y })
// 3. 省略返回类型：函数体只有一个表达式，能推断
let c = nums.sorted(by: { x, y in return x < y })
// 4. 省略 return：单表达式闭包隐式返回
let d = nums.sorted(by: { x, y in x < y })
// 5. 参数简写：$0 是第一个参数，$1 是第二个，从此不用起名字
let e = nums.sorted(by: { $0 < $1 })
```

怎么读：这五种写法完全等价，编译器都会得到 `[1, 2, 3]`。Swift 允许一路省略，是因为类型信息可以从上下文推出来。项目里最常见的是第 4、5 种，看到 `$0`、`$1` 时按位置对应参数即可：`$0` 是第一个参数，`$1` 是第二个，以此类推。如果闭包体超过两三行，就应该写回完整的参数名，否则读起来很痛苦。

尾随闭包让调用更像语句：

```
let doubled = [1, 2, 3].map { $0 * 2 }
let sorted = [3, 1].sorted { $0 < $1 }

Button("加一") {          // SwiftUI 里最常见的形状
    model.increase()
}
```

怎么读：当闭包是函数的最后一个参数时，可以把花括号挪到调用括号外面，这叫尾随闭包。如果这个函数只有闭包一个参数，连圆括号都可以省掉。SwiftUI 里满屏都是这种写法——`Button`、`VStack`、`onAppear` 全都在收尾随闭包，所以你必须习惯"大括号不一定属于上一条语句"。

闭包会捕获它用到的外部变量：

```
func makeCounter() -> () -> Int {
    var count = 0
    return {
        count += 1        // 捕获外部的 count
        return count
    }
}

let next = makeCounter()
next()    // 1
next()    // 2
print(next())   // 3
```

怎么读：闭包记住了它引用的外部变量，即使创建它的函数已经返回，这个变量依然活着。这就是"捕获"。它和 C 的局部变量出栈即销毁完全不同，也是闭包能当状态容器用的原因。

逃逸闭包 `@escaping`：

```
final class Loader {
    var onFinish: (([String]) -> Void)?      // 把回调存起来

    func load(completion: @escaping ([String]) -> Void) {
        onFinish = completion                // 闭包被存进属性，函数返回后才可能被调用
    }

    func finish() {
        onFinish?(["数据"])                   // 之后某个时刻才执行
    }
}
```

怎么读：函数参数默认是"非逃逸"的，编译器保证闭包在函数返回之前就用完了，因此它能做优化，也不强求你写 `self.`。一旦你把闭包存进属性、放进数组、交给异步任务，它就会"逃出"函数的生命周期——这就是逃逸。这时必须标 `@escaping`，否则编译不过。所以看到 `@escaping` 你就该意识到：这段代码不会立刻执行，而是在函数返回之后、某个事件发生时才会被调用。网络请求的完成回调、定时器回调、`DispatchQueue.main.asyncAfter` 里排队的任务，都属于这种模式。

两者的差别可以整理成一张表：

对比项	非逃逸闭包	逃逸闭包
声明方式	默认，不写关键字	参数前加 @escaping
何时执行	函数返回之前	函数返回之后的某个时刻
能否存进属性	不可以	可以

对比项	非逃逸闭包	逃逸闭包
访问属性时	不要求写 self	必须显式写 self.

怎么读：这张表解释了 `@escaping` 为什么值得单独记一笔——它同时改变了闭包的执行时机和写法。看到不带 `@escaping` 的参数，可以放心认为闭包是同步执行完的；看到带了，就要去想它会在什么时机被触发，以及触发时它捕获的那些对象是否还活着。

> 提示：逃逸闭包内部访问属性必须显式写 `self.`，这是编译器在提醒你"这个闭包活得比当前对象久"。配套的捕获列表 `[weak self]` 用来避免循环引用，第 3 章会详细讲，这里知道它出现的位置就在闭包开头即可。

2.10 小结与自测

本章要点：

- `let` 是默认选择，只有确实需要修改时才用 `var`；Swift 没有"未初始化的变量"。
- 类型推断很省事，但空数组、空字典和 Set 必须显式写出元素类型。
- Swift 不做隐式数值转换，`Double(a) + b` 这种显式转换是常态，`Int(1.9)` 是截断而非四舍五入。
- 字符串插值是把值拼进文本的标准方式，比用 `+` 拼接更清楚；`String` 是值类型，赋值即复制。
- 三种集合各有分工：Array 有序靠下标，Dictionary 按键取值，Set 用于去重和判断存在性。
- `first`、`ages["key"]`、`first(where:)` 这类"可能查不到"的 API 一律返回可选值。
- 数组下标越界会直接崩溃，优先使用 `first`、`last`、`indices.contains` 这类安全写法。
- `switch` 必须穷尽且不需要 `break`；范围匹配、`where`、元组值绑定让它能替代一长串 `if`。
- 函数参数标签是 Swift 的特色，调用点写出来就像一句英文；`_` 表示省略标签。
- `?` 表示"可能没有值"，使用前必须解包；`!` 表示"我保证有值"，是崩溃的主要来源。
- 解包优先用 `if let` 和 `guard let`，给默认值用 `??`，链式访问用 `?.`；`Type!` 只应出现在理解清楚的老代码里。
- 闭包是能传递的代码块，尾随闭包是 SwiftUI 的基础语法；`@escaping` 说明它会在函数返回之后才被调用。

自测题，先自己说出结果再看答案。

题目 1：

```
let a = 3
let b = 1.5
print(Double(a) + b)
```

答案：4.5。解析：Swift 不做隐式数值转换，Int 必须显式转成 Double 才能和 Double 相加。

题目 2：

```
let nums = [1, 2, 3]
print(nums.first)
print(nums.first ?? 0)
```

答案：先输出 `Optional(1)`，再输出 `1`。解析：`first` 返回 `Int?`，直接打印会带 Optional 包装，用 `??` 解包后才是普通 Int。

题目 3：

```
var ages = ["小明": 20]
ages["小红"] = 21
print(ages.count)
```

答案：2。解析：用下标给字典赋值时，键不存在就是新增，存在才是覆盖；`count` 数的是键值对个数。

题目 4：

```
func check(_ value: Int?) {
    guard let value else { return }
    print(value * 2)
}
check(nil)
check(4)
```

答案：只输出 8。解析：`guard let` 在值为 nil 时提前 return，所以第一次调用什么都不打印。

题目 5：

```
switch 5 {
case 1..<5: print("偏小")
case 5...: print("偏大")
default: print("其他")
}
```

答案：输出"偏大"。解析：`1..<5` 不含 5，`5...` 覆盖所有不小于 5 的整数，switch 从上往下匹配、命中即结束；但 0 和负数没被覆盖，所以 `default` 仍然不能省。

第 3 章 Swift 速成（下）：面向对象与进阶特性

上一章解决了"变量、类型、控制流、函数、可选值、闭包"这些所有语言都有的东西。这一章讲 Swift 真正区别于 C、Java、Python 的部分：枚举、结构体、协议、泛型、并发，以及 SwiftUI 里满屏的 `@` 标记。学完这一章，你应该能读懂一个真实 iOS 项目里 80% 的代码，剩下的 20% 是具体业务和框架细节。

这一章的阅读方式和上一章不同：上一章要你记住语法，这一章要你建立"读代码的直觉"。项目里 90% 的代码不是炫技，而是几种固定套路的重复：用枚举描述状态，用结构体装数据，用协议抽出共性，用 `async/await` 取数据，用 `@State` 和 `@Observable` 驱动界面。你把这几套套路认出来，代码就不再是一行行看不懂的符号。

> 本章的例子都可以直接写进一个 `.swift` 文件或用 `#Preview` 跑起来。遇到看不懂的符号，随时翻到 3.12 的速查表。

3.1 枚举 enum 与关联值

一句话结论：枚举（enum）是"一组有限取值的类型"，Swift 的枚举比 C 和 Java 的枚举强得多，因为它可以携带数据。

最基础的用法是给有限的状态起名字。

```
enum Direction {
    case north, south, east, west   // 一行可以写多个 case
}

let d = Direction.north
```

怎么读：`enum` 定义了一个新类型，`case` 列出所有可能取值。使用时要么写全 `Direction.north`，要么在类型已知时简写成 `.north`。

- `.north` 这种简写叫"隐式成员表达式"，SwiftUI 里遍地都是，例如 `.foregroundStyle(.red)`。
- `switch` 是枚举最自然的搭档，而且必须穷尽所有 `case`，漏一个编译器就报错——这是 Swift 帮你防 bug 的重要手段。

枚举可以带原始值（raw value），常用在需要和字符串或数字互相转换的场景。

```
enum HTTPMethod: String {
    case get = "GET"
    case post = "POST"
}

enum Priority: Int {
    case low = 1, normal, high   // 后续自动递增为 2、3
}

HTTPMethod.post.rawValue          // "POST"
HTTPMethod(rawValue: "GET")       // Optional(.get)，可能失败所以是可选值
```

如果只是想要"能遍历所有 case"，让它遵循 `CaseIterable` 协议即可。

```
enum Flavor: String, CaseIterable {
    case sweet, sour, salty
}

Flavor.allCases.map(\.rawValue)   // ["sweet", "sour", "salty"]
```

关联值（associated value）是重点：每个 `case` 可以额外带一份数据，数据的类型和个数由你定义。

```
enum LoadState {
    case idle
    case loading(progress: Double)
    case success(data: [String])
    case failure(message: String)
}
```

怎么读：`LoadState` 描述了一次加载的四种状态，其中三种各自携带不同数据。用 `switch` 解包时，把 `case` 里的数据绑到局部常量上。

```
func describe(_ state: LoadState) -> String {
    switch state {
    case .idle:
        return "还没开始"
    case .loading(let progress):
        return "加载中 \(Int(progress * 100))%"
    case .success(let data):
        return "拿到 \(data.count) 条"
    case .failure(let message):
        return "失败：\(message)"
    }
}
```

怎么读：`case .loading(let progress)` 表示"如果状态是 loading，就把里面的进度值取出来叫 progress"。带标签的关联值也可以写成 `.failure(message: let m)` 或直接 `.failure(let m)`。不想用某个值就写 `_`，例如 `case .success:`。

> 关键点：关联值是"把一个值和一个标签绑在一起"，而可选值 `String?` 其实是标准库里的 `enum Optional { case none; case some(String) }`。因此 `if let`、`guard let`、`??` 本质上都是在解包一个枚举，理解这一点后，可选值就不再神秘。

递归枚举：当枚举的关联值又是它自己时（例如表达式树、链表），必须加 `indirect` 关键字，否则编译器无法确定内存布局。

```
indirect enum Expr {
    case number(Int)
    case add(Expr, Expr)      // 关联值是枚举自己，所以必须 indirect
}
```

怎么读：`indirect` 让 Swift 在需要时为这一层加一层间接存储，从而允许递归。真实业务里很少见，看到它知道"这是一个可以嵌套自身的树结构"就够了。

最后强调一个判断标准：只要一个值的取值是"有限的几种情况"，就应该用枚举，而不是散落的 `Bool` 加字符串。

```
// 差：调用方要记住哪些字符串合法
var state: String = "loading"

// 好：编译器保证只会是这三种
var status: LoadState = .loading
```

怎么读：枚举把"合法取值集合"写进了类型，配合 `switch` 的穷尽检查，一个新加的 `case` 会让所有漏处理的 `switch` 直接编译失败。这是 Swift 帮你找工作量的方式，也是它比动态语言更安全的地方。

3.2 结构体 struct 与类 class

一句话结论：结构体是值类型，赋值时复制；类是引用类型，赋值时共享同一份数据。这是 Swift 里最重要的一个区别。

先看一段能说明问题的代码。

```
struct PointStruct { var x = 0 }
class PointClass { var x = 0 }

var a = PointStruct()
var b = a
b.x = 10
print(a.x, b.x)      // 0 10 —— 复制了，互不影响

let c = PointClass()
let d = c
d.x = 10
print(c.x, d.x)      // 10 10 —— 同一个对象
```

怎么读：`b = a` 对结构体是"把值拷一份"，所以改 `b` 不动 `a`；`c` 和 `d` 指向同一个堆上的对象，改谁都一样。注意 `let c = PointClass()` 之后仍然能改 `c.x`，因为 `let` 锁住的是引用本身，不是对象内部。

结构体还有一个好处：自动生成成员初始化器（memberwise initializer），不用手写构造函数。

```
struct User {
    var name: String
    var age: Int
}

let u = User(name: "小明", age: 20)   // 编译器自动生成的初始化器
```

类支持继承和 `override`，这是结构体没有的能力。

```
class Animal {
    var name: String
    init(name: String) { self.name = name }
    func speak() -> String { "..." }
}

class Dog: Animal {
    override func speak() -> String { "汪汪" }   // 覆写必须写 override
}

Dog(name: "旺财").speak()   // "汪汪"
```

怎么读：`class Dog: Animal` 表示继承，子类可以重写父类方法，但必须标 `override`，编译器会帮你检查是否真的存在被覆写的方法（拼错方法名会立刻报错）。

- `final` 加在类或方法前，表示"禁止被继承或覆写"。它能让编译器做优化，也能表达设计意图：这个类不该被扩展。
- 子类初始化时如果想复用父类逻辑，用 `super.init(...)`。
- 类的实例要用 `===` 判断"是不是同一个对象"，`==` 需要你自己实现（遵循 `Equatable`）。

什么时候用结构体、什么时候用类？按下面这张表判断。

判断依据	选结构体 struct	选类 class
数据语义	表示一个值，如坐标、用户资料	表示一个有身份的对象，如网络会话
复制行为	希望复制后互不影响	希望多处共享同一份状态
继承需求	不需要继承	需要继承或覆写
与框架配合	SwiftUI 视图、数据模型首选	需要被 Objective-C 调用时用

> 实践准则：默认用结构体，只有在确实需要共享可变状态或继承时才用类。SwiftUI 的视图、模型数据几乎全是结构体。

值类型的方法如果要修改自己的属性，必须标 `mutating`。

```
struct Counter {
    var value = 0
    mutating func increase() { value += 1 }   // 不加 mutating 会编译报错
}
```

为什么必须写：结构体是值类型，方法默认不能改动 `self`。加 `mutating` 相当于告诉编译器"这个方法会替换掉自己"，于是 Swift 才知道 `let` 声明的结构体不能调用它。类的实例是引用类型，不需要也不允许写 `mutating`。

最后一个新手最容易困惑的点：为什么改结构体数组的元素要小心。

```
struct Counter { var value = 0 }

var list = [Counter(value: 1)]
list[0].value += 1        // 合法：直接从数组里取出再写回
for item in list {
    // item.value += 1     // 报错：item 是副本，且是 let
}
```

怎么读：`list[0].value += 1` 之所以合法，是因为下标访问本身可以被赋值；而 `for item in list` 里的 `item` 是每次循环复制出来的常量，改它没有意义，编译器会直接拦下。需要修改就写 `for i in list.indices { list[i].value += 1 }`，或者用类的引用语义。

3.3 属性：存储、计算与观察器

一句话结论：属性分三类——存数据的、算出来的、盯着变化的，分别叫存储属性、计算属性、属性观察器。

存储属性就是普通字段，它真的占用实例的内存。`lazy` 表示"第一次被访问时才初始化"，适合创建开销大的对象，或者初始化时需要用到 self 的其他属性。

```
struct ReportView {
    var rawData: [String]

    lazy var summary: String = {
        rawData.joined(separator: "、")   // 第一次读 summary 时才计算
    }()
}
```

怎么读：`lazy var summary: String = { ... }()` 是一个"立即执行闭包"的写法，闭包只会在第一次访问 `summary` 时运行一次，结果被缓存下来。后面的 `()` 表示立刻调用它。因为要延迟写入，`lazy` 只能修饰 `var`，不能修饰 `let` 或计算属性。

> 注意：`lazy` 属性不是线程安全的。如果多个线程可能同时第一次访问它，要么加锁，要么改用别的方案。

计算属性不存值，每次访问时现算。可以只读，也可以带 setter。

```
struct Rectangle {
    var width: Double
    var height: Double

    var area: Double { width * height }          // 只读简写

    var isSquare: Bool {
        get { width == height }
        set { width = newValue; height = newValue }   // 写入时同步两边
    }
}
```

怎么读：`{ width * height }` 没有 `get` 关键字，是只读计算属性的简写。带 `set` 时，`newValue` 是编译器自动提供的入参名，也可以自己写成 `set(value)` 再用 `value`。

属性观察器用来在属性变化前后插入逻辑。`willSet` 在赋值前调用（新值叫 `newValue`），`didSet` 在赋值后调用（旧值叫 `oldValue`）。

```
var score = 0 {
    willSet { print("即将变成 \(newValue)") }
    didSet { print("从 \(oldValue) 变成了 \(score)") }
}
```

怎么读：观察器只管"属性被赋值后要做什么"，常见用途是写日志、触发刷新、把值同步到别处。注意在 `didSet` 里给同一属性再次赋值不会再触发一遍观察器，避免死循环。

属性观察器有一个容易被忘掉的限制：它不写在初始化器里赋值时触发。

```
struct Player {
    var hp = 100 {
        didSet { print("血量变化") }
    }
}

var p = Player()        // 不打印，初始化不算"变化"
p.hp = 80               // 打印一次
```

怎么读：`init` 里给属性赋初值属于"建立对象"，不触发观察器；对象建好之后的每次赋值才触发。读代码时如果发现日志没打出来，先确认那次赋值是不是发生在初始化阶段。

属性观察器还有一个常见用法：给 `@State` 加副作用。

```
struct SearchView: View {
    @State private var keyword = "" {
        didSet { print("搜索词变成了 \(keyword)") }
    }
    var body: some View { TextField("搜索", text: $keyword) }
}
```

怎么读：`@State` 和观察器可以叠加，属性包装器先包住读写逻辑，观察器再包一层。写 SwiftUI 时这类写法要克制，能用 `onChange` 修饰符表达就用修饰符，逻辑更清晰。

类型属性用 `static` 声明，属于类型本身而不是某个实例；`class` 修饰的计算属性允许子类覆写。

```
struct Config {
    static let appName = "Bingyan"   // 全局唯一，用 Config.appName 访问
}

class Base {
    class var kind: String { "base" }   // 子类可以 override 它
}
```

怎么读：`static` 成员用类型名访问，不需要创建实例，常用于常量、共享配置、单例。`class` 关键字在属性或方法前表示"这个成员可以被继承和覆写"，只能出现在类里。日常你看到的 90% 是 `static`，`class` 主要出现在框架的基类设计中。

3.4 协议 protocol 与扩展 extension

一句话结论：协议是"能力清单"，任何类型只要实现了清单上的要求就算遵循它；扩展是"给已有类型补代码"，不用改原来的源码。

```
protocol Describable {
    var label: String { get }
    func describe() -> String
}

struct Tag: Describable {
    let label: String
    func describe() -> String { "标签：\(label)" }
}
```

怎么读：协议里只写要求，不写实现。`{ get }` 表示遵循者至少要提供可读的 `label`；如果写 `{ get set }`，就必须是可读可写属性。遵循协议时 `struct Tag: Describable` 一行搞定，编译器会强制你实现全部要求。

协议最重要的用法是当类型用——你可以声明一个"遵循某协议的东西"，而不关心它具体是什么类型。

```
let items: [any Describable] = [Tag(label: "swift"), Tag(label: "ios")]
items.map { $0.describe() }
```

协议扩展可以为协议方法提供默认实现，这样遵循者什么都不写也能用。

```
extension Describable {
    func describe() -> String { "未命名：\(label)" }
}
```

怎么读：有了这段扩展，`Tag` 就算不实现 `describe()` 也能编译通过，用的是默认实现。这是 Swift 里"给一组类型批量加能力"的主要手段，也是把重复代码从各个实现里抽走的常用技巧。

- 协议继承：`protocol A: B` 表示遵循 A 必须先遵循 B。SwiftUI 的 `View` 就继承自其他协议，所以你自己写的视图只要遵循 `View`，就顺带获得了那些能力。

- 协议组合：`some P & Q` 表示"同时满足 P 和 Q"。`&` 读作"并且"，常见于函数参数，例如要求传入的东西既能显示又能比较。
- 扩展可以给任何类型加方法、计算属性，甚至补上协议遵循。但不能加存储属性，因为扩展不改变类型的内存布局。

```
protocol Named { var name: String { get } }
protocol Aged { var age: Int { get } }

func show(_ item: some Named & Aged) -> String {
    "\(item.name), \(item.age) 岁"
}
```

怎么读：`some Named & Aged` 表示"某个同时满足两个协议的类型"，函数体里可以同时用 `name` 和 `age`。这是 Swift 里表达"这个参数得有这些能力"的紧凑写法。

```
extension String {
    var isBlank: Bool { trimmingCharacters(in: .whitespaces).isEmpty }
}

"   ".isBlank   // true
```

为什么 SwiftUI 里到处是 extension：每个视图类型都单独写一个 `extension`，能让一个文件里同时放好几个相关视图、又能各自保持独立，还方便按功能给系统类型补小工具方法。读项目时看到 `extension SomeView` 就知道，那是把同一个视图的零散代码组织在一起。

真实项目里你还会看到"给已有类型补协议遵循"的写法，比如把一个普通结构体补成 `Identifiable`。

```
struct Item { let id: Int; let title: String }

extension Item: Identifiable { }   // 有 id 属性就自动满足要求
```

怎么读：`Identifiable` 只要求一个 `id`，`Item` 本来就有，所以空扩展就够了，这样它就能直接塞进 SwiftUI 的 `List`。这个模式读代码时会频繁出现：类型定义保持干净，能力通过扩展补上去。另外还有几个你会天天见到的系统协议值得记住：`Equatable` 支持 `==`，`Hashable` 支持当字典键和放进 `Set`，`Codable` 支持 JSON 编解码，`Identifiable` 支持列表标识。它们大多可以由编译器自动合成，你只要在类型后面写上名字就行。

3.5 泛型与 some、any

一句话结论：泛型让你写一份代码适配多种类型，`some` 表示"具体类型我不想告诉你，但它确定是某一个"，`any` 表示"这里装的是任意一个遵循该协议的值"。

泛型函数用占位类型名（惯例是 T）表示"任意类型"。

```
func firstElement<T>(of array: [T]) -> T? {
    array.first
}

firstElement(of: [1, 2, 3])        // 1
firstElement(of: ["a", "b"])       // "a"
```

怎么读：<T> 声明了一个类型占位符，调用时由实参自动推断成 Int 或 String。泛型类型同理，struct Stack<Element> { ... } 就是一个什么都能装的栈。

类型约束用 where 或冒号表示"这个类型还得满足额外条件"。

```
func sum<T: Numeric>(_ values: [T]) -> T {
    values.reduce(0, +)
}
```

怎么读：T: Numeric 表示 T 必须是数字类型，因此 + 合法。如果没有约束，T 就只能当"可以搬运的值"用，不能做加法。

泛型类型把占位符放在类型名后面，一份代码就能装各种元素。

```
struct Pair<A, B> {
    let first: A
    let second: B
}

let p = Pair(first: "小明", second: 20)   // A = String, B = Int
```

怎么读：Pair<A, B> 声明了两个类型参数，创建实例时由实参自动推断。上一章你见过的 [String] 其实就是 Array<String>，["a": 1] 是 Dictionary<String, Int>，它们都是标准库里的泛型类型。

some 是不透明类型（opaque type）：函数返回一个确定但对外匿名类型。SwiftUI 里最著名的例子就是视图的 body。

```
struct ContentView: View {
    var body: some View {
        VStack {
            Text("标题")
            Text("副标题")
        }
    }
}
```

为什么 body 要写 some View：VStack 的真实类型极其复杂（内部嵌套一堆泛型），写全根本没法读；而写成 View 又不行，因为协议带有关联类型，不能直接当返回值。some View 正好两全其美

——外部只知道"这是一个视图"，编译器内部却知道它的确切类型，因此还能做静态派发和性能优化。

`any` 是存在类型（existential type）：一个能装不同具体类型的盒子，代价是运行时需要装箱、调用走动态派发。

```
let shapes: [any Shape] = [Circle(), Square()]   // 可以混装不同类型
```

对比项	some	any
含义	某一个固定类型，对外隐藏	任意遵循该协议的类型
数组里能否混装	不能，全是同一个类型	可以混装
性能	静态派发，无额外开销	动态派发，有装箱开销
典型场景	SwiftUI 的 body、链式返回	需要异构集合时

> 新手判断法：写 SwiftUI 视图时无脑用 `some View`；只有当你确实需要一个能装多种类型的数组或属性时，才用 `any`。

3.6 错误处理：throws、try 与 do-catch

一句话结论：Swift 用 `throws` 标记"可能失败的操作"，用 `try` 调用它，用 `do-catch` 处理失败。

错误类型必须是遵循 `Error` 协议的类型，惯例是枚举。

```
enum NetworkError: Error {
    case badURL
    case noData
    case statusCode(Int)    // 携带状态码，方便定位问题
}
```

怎么读：`enum NetworkError: Error` 一行就让这个枚举成为可抛出的错误。给错误带关联值能保留上下文，比一串字符串有用得多。

一个函数只要有一个 `throws`，调用链上每一层都得决定"我自己处理还是继续往上抛"，这让错误不可能被悄悄忽略。抛错函数在参数列表后面写 `throws`，用 `throw` 抛出。

```
func fetchText(from urlString: String) throws -> String {
    guard let url = URL(string: urlString) else { throw NetworkError.badURL }
    return try String(contentsOf: url)   // 调用了会抛错的方法，必须 try
}
```

如果这个函数自己不想处理错误，就在签名后面加 `throws`，把错误继续往上层交。

```
func loadUserName() throws -> String {
    let text = try fetchText(from: profileURL)   // 继续往上抛
    guard !text.isEmpty else { throw NetworkError.noData }
    return text
}
```

调用有三种写法：`try` 交给上层，`try?` 把失败转成 `nil`，`try!` 失败直接崩溃。

```
let text = try? fetchText(from: "https://example.com")   // 成功给值，失败给 nil
```

> `try!` 只在你百分百确定不会失败时用（例如读取 App 自带的资源文件）。业务代码里用 `try!` 属于给未来埋雷。

完整处理用 `do-catch`，可以按 `case` 分类处理。

```
do {
    let text = try fetchText(from: urlString)
    print(text)
} catch NetworkError.badURL {
    print("地址不合法")
} catch {
    print("其他错误：\(error)")   // catch 必须兜底，error 是自动提供的内置名
}
```

`defer` 用于"离开当前作用域时一定执行"的清理代码，例如关文件、解锁。它的执行时机是函数返回前，无论中间是正常返回还是抛错返回。

```
func readConfig() throws -> String {
    print("开始")
    defer { print("结束") }     // 无论怎么退出都会执行
    throw NetworkError.noData
}
```

怎么读：调用 `readConfig()` 会先打印"开始"，抛错前先打印"结束"。`defer` 的典型用途是释放资源，Swift 里有 ARC 和自动内存管理，所以业务代码用得不多，但在文件、锁、数据库事务场景里很常见。

`Result` 类型适合把"成功或失败"当值来传递，尤其是回调式 API 或需要存起来稍后处理时。

```
func load(completion: (Result<[String], NetworkError>) -> Void) {
    completion(.success(["swift"]))
}

load { result in
    switch result {
    case .success(let items): print(items.count)
    case .failure(let error): print(error)
    }
}
```

怎么读：`Result` 本身也是一个枚举，两个 `case` 分别是 `.success(值)` 和 `.failure(错误)`，所以照样用 `switch` 解包。

> 判断法：用 `async/await` 能自然表达流程时就用 `throws`；需要把结果存起来、当参数传、或在回调里返回时，用 `Result`。3.9 会给出 async 版本的网络请求。

最后把"一个完整的网络请求失败流程"串一遍，这是读项目时最常见的形态。

```
func loadRepos() async throws -> [String] {
    guard let url = URL(string: "https://api.example.com/repos") else {
        throw NetworkError.badURL                  // 参数问题，本地就拦下
    }
    let (data, response) = try await URLSession.shared.data(from: url)
    guard let http = response as? HTTPURLResponse else { throw NetworkError.noData }
    guard (200..<300).contains(http.statusCode) else {
        throw NetworkError.statusCode(http.statusCode)   // 服务端错误带码往上抛
    }
    return try JSONDecoder().decode([String].self, from: data)
}
```

怎么读：这段代码依次处理四种失败——地址不合法、响应类型不对、状态码非成功、JSON 解析失败。`guard ... else { throw }` 是 Swift 处理错误的惯用形态，读作"必须满足，否则提前结束"。上层拿到错误后，用 3.6 开头的 `do-catch` 决定是提示用户还是重试。

3.7 属性包装器与 @ 标记家族

一句话结论：属性包装器（property wrapper）是"把某个属性的读写逻辑抽成一个可复用的类型"，它的标记方式就是 `@`；SwiftUI 里那一堆 `@State`、`@Binding` 都是这个机制的产物。

自己写一个最小属性包装器：把值限制在 0 到 100 之间。

```
@propertyWrapper
struct Clamped {
    private var value = 0
    var wrappedValue: Int {
        get { value }
        set { value = min(max(newValue, 0), 100) }   // 所有写入都被夹紧
    }
}

struct Score {
    @Clamped var points: Int
}

var s = Score()
s.points = 200
print(s.points)   // 100
```

怎么读：`@Clamped` 的类型必须提供 `wrappedValue`；用 `@Clamped` 声明属性后，对这个属性的读写都会经过包装器的 getter 和 setter。理解了这三十行，SwiftUI 的状态管理就不再是"魔法"。

包装器还能通过 `projectedValue` 暴露一个"投影值"，SwiftUI 里用 `$` 前缀取到的就是它。

```
@propertyWrapper
struct Doubled {
    var wrappedValue: Int
    var projectedValue: Int { wrappedValue * 2 }
}

struct Box { @Doubled var value = 3 }

let b = Box()
print(b.value, b.$value)   // 3 6
```

怎么读：`wrappedValue` 是你平时写 `b.value` 拿到的值，`projectedValue` 是写 `b.$value` 拿到的值。这正是 `@State` 能同时提供"值"和"绑定"两种访问方式的原因。

接下来是你会反复遇到的 `@` 标记家族。

标记	作用	典型场景
@State	视图私有的可变状态	计数器、输入框内容
@Binding	父子视图间的双向绑定	子视图修改父视图状态
@Observable	标记可被观察的类，属性变化自动刷新视图	iOS 17+ 的数据模型
@Environment	从环境读取系统或自定义注入的值	读取配色模式、关闭弹窗
@AppStorage	把属性直接映射到 UserDefaults	记住开关、上次选择
@MainActor	保证成员在主线程执行	更新 UI 的类与函数
@escaping	标记闭包会在函数返回后才被调用	异步回调参数
@available	声明 API 的可用平台或版本	只在 iOS 17+ 使用的代码
@ViewBuilder	让闭包可以写多条视图语句	自定义容器的内容闭包
@Query	SwiftData 的查询声明	从数据库读取列表
@Model	把类声明为 SwiftData 模型	持久化实体

`@State` 和 `@Binding` 是最常见的两个。

```
struct CounterView: View {
    @State private var count = 0
    var body: some View {
        Stepper("数量：\(count)", value: $count)   // $ 取出绑定
    }
}
```

怎么读：`@State` 的属性一变，视图就重新渲染；`$count` 拿到的是绑定（Binding），可以传给需要读写的子视图或控件。

`@Observable` 是 iOS 17 之后取代 `ObservableObject` 与 `@Published` 的推荐写法，需要注意它标记的是类。

```
@Observable
final class CartModel {
    var items: [String] = []      // 不需要再写 @Published
}
```

怎么读：类里任何属性变化，用到该属性的视图都会自动刷新。你会在老代码里看到 `class X: ObservableObject` 配 `@StateObject`、`@Published`，那是旧写法，新项目不要再用。

另外三个标记给一段代码就够了。`@Environment` 的写法是 `@Environment(\.某个键)`，键由系统或你自己通过环境注入。

```
struct SettingsView: View {
    @AppStorage("darkMode") private var darkMode = false       // 直接读写 UserDefaults
    @Environment(\.colorScheme) var colorScheme                // 读系统环境值

    var body: some View {
        Toggle("深色模式", isOn: $darkMode)
            .preferredColorScheme(darkMode ? .dark : nil)
    }
}
```

怎么读：`@AppStorage` 的读写会立刻落到 `UserDefaults`，不需要自己写存取代码；`@Environment` 从环境里拿值，`\.colorScheme` 是系统提供的键，读出来是 `.light` 或 `.dark`。

`@available` 用来声明可用版本，例如把 `@available(iOS 17, *)` 加在一个方法前，表示它只能在 iOS 17 及以上调用，星号表示"其他平台不限制"。

`@ViewBuilder` 让闭包可以写多条视图语句，自定义容器时几乎必用。

```
struct Card<Content: View>: View {
    @ViewBuilder var content: Content      // 调用方可以写多行视图

    var body: some View {
        content.padding().background(.gray.opacity(0.1))
    }
}
```

怎么读：没有 `@ViewBuilder`，闭包里只能有一个返回值；加上之后，编译器会把多行视图自动组合成一个整体。

`@escaping` 只出现在函数参数上，含义是"这个闭包不会在函数返回前就执行完"。

```
func load(completion: @escaping (String) -> Void) {
    DispatchQueue.main.async { completion("done") }   // 闭包逃出了函数作用域
}
```

> 没有 `@escaping` 的闭包参数默认是"非逃逸"的，编译器可以做更好的优化。看到编译报错要求你加 `@escaping`，说明你的闭包被存了起来或异步调用了。

3.8 内存管理：ARC、weak 与循环引用

一句话结论：Swift 用 ARC（自动引用计数）管理类的内存，谁被强引用谁就活着；当两个对象互相强引用时，谁都释放不掉，这就是循环引用，用 `weak` 打破它。

基本规则很简单：每多一个强引用，计数加一；引用消失，计数减一；计数归零，对象释放。结构体是值类型，不参与引用计数。

`weak` 和 `unowned` 都是"不增加引用计数"的引用，区别在于安全性。

修饰	是否置空	使用场景
weak	是，自动变成 nil，所以必须是可选值	代理、闭包里的 self、可能先消失的一方
unowned	否，对象没了再访问会崩溃	生命周期确定不会先消失的一方，如父与子

`weak` 的写法有一个硬性要求：它修饰的属性必须是可选值，且必须是 `var`，因为对象释放后它会被自动置为 `nil`。

```
protocol LoaderDelegate: AnyObject { func didFinish() }

final class Loader {
    weak var delegate: LoaderDelegate?   // 代理必须弱引用，否则双方互指
}
```

怎么读：`AnyObject` 约束表示这个协议只能被类遵循，因为只有类才有引用计数和 `weak`。这是"代理模式必须写 weak"这条规则的由来：控制器通常强引用子视图，子视图的代理指回控制器，只要其中一方是弱引用，环就断了。

闭包捕获 `self` 是循环引用的重灾区。错误写法：

```
final class ProfileViewModel {
    var name = "小明"

    func load() {
        Network.fetch { result in
            self.name = result      // 闭包强引用 self，self 又持有闭包，于是互相锁死
        }
    }
}
```

正确写法：

```
final class ProfileViewModel {
    var name = "小明"

    func load() {
        Network.fetch { [weak self] result in
            guard let self else { return }   // self 已释放，直接退出
            self.name = result
        }
    }
}
```

怎么读：`[weak self]` 写在闭包参数列表最前面，表示"在闭包里弱引用 self"。因为 weak 会自动置空，所以要用 `guard let self else { return }` 把它提升成强引用再用，顺便处理了 self 已释放的情况。

什么时候可以不加 `[weak self]`？判断依据是"闭包会不会被 self 长期持有"。

- 闭包是立即执行的（如 `map`、`filter`、`sorted`），函数返回后闭包就没了，不会形成环，不用加。
- 闭包被存成了 self 的属性，或传给了会长期持有的对象（网络回调、通知观察者、定时器），必须考虑加。
- 闭包捕获的是局部变量或值类型，不涉及 self，不用加。
- 单次异步任务（例如 `Task { await load() }`）通常会执行完，虽然也会强引用 self，但不会永久持有，实践中可以不加；省略后如果发现控制器不释放，再补上。

> 无条件给所有闭包加 `[weak self]` 是错误的做法：它会引入一堆没必要的可选值解包，也会让代码噪音变大。先问"谁持有谁"。

再补一个项目里常见的形态：视图模型发起异步任务时，什么时候该担心 self。

```
final class FeedViewModel {
    private var loadTask: Task<Void, Never>?

    func start() {
        loadTask = Task { [weak self] in      // 任务被存成属性，形成了持有关系
            await self?.reload()
        }
    }

    func reload() async { }
}
```

怎么读：`loadTask` 是 `self` 的属性，Task 闭包又引用 `self`，这里就构成了一条环，所以要加 `[weak self]`。判断标准始终是同一条：闭包是否会被 self 长期持有。如果这里写成 `Task { await reload() }` 而不存到属性上，任务执行完引用就断了，一般无需弱引用。

unowned 的闭包写法是 [unowned self]，它不会把 self 变成可选值，用起来少一层解包，但一旦 self 先释放就会崩溃。只有在你能确定"self 一定比闭包活得久"时才用它，新手阶段建议一律优先 weak，安全第一。

内存问题在 Xcode 里怎么查：打开 Debug Memory Graph 可以看到对象之间的引用关系图，循环引用会明显表现为互相指向的一组节点；deinit 里的打印也是常用手段，如果对象该释放时没有打印，说明有强引用环。养成给重要的类加 deinit 日志的习惯，排查泄漏会快很多。

3.9 并发：async/await、Task 与 @MainActor

一句话结论：async 函数是可以暂停等待的函数，用 await 调用它；Task 从同步代码里启动一个异步任务；UI 更新必须在主线程，所以用 @MainActor 保证。

上一章只讲过闭包回调，回调写多了会层层嵌套。async/await 让异步代码看起来像顺序代码。

```
func fetchTitle() async throws -> String {
    let url = URL(string: "https://example.com")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return String(decoding: data, as: UTF8.self)
}
```

怎么读：async 表示函数可能挂起；await 标记挂起点，读作"在这里等一下，等到了再往下走"。URLSession.shared.data(from:) 是标准库提供的异步网络请求方法，返回数据和一个响应。

Task 用来在同步上下文里启动异步工作；SwiftUI 的 .task 修饰符则是"视图出现时自动启动、视图消失时自动取消"。

```
struct TitleView: View {
    @State private var title = "加载中"
    @State private var errorMessage: String?

    var body: some View {
        VStack(spacing: 12) {
            if let errorMessage {
                Text(errorMessage)
            } else {
                Text(title)
            }
            Button("重新加载") { Task { await load() } }
        }
        .task { await load() }        // 视图出现时执行一次，消失时自动取消
    }

    private func load() async {
        do {
            title = try await fetchTitle()        // 这里已经在主线程
        } catch {
            errorMessage = "加载失败：\(error)"
        }
```

```
    }
}
```

怎么读：`@State` 的数据由主线程更新；SwiftUI 的视图方法默认在主线程执行，所以在这个 `load()` 里直接改 `title` 是安全的。如果 `load()` 写在别的类型里，就必须给它加 `@MainActor`，否则 Swift 6 会报"这个属性只能在主线程访问"。

手动控制线程用 `@MainActor`。

```
@MainActor
final class TitleViewModel {
    var title = ""        // 这个类型的成员只能在主线程访问
}
```

怎么读：`@MainActor` 保证被标记的类、方法或属性只在主线程上执行，编译器会帮你检查。新手遇到"改 UI 数据报错"时，第一反应就应该是给它加上 `@MainActor`。

同时发起多个请求用 `async let`；数量不定或需要并发处理一批任务，用 `TaskGroup`。两者都能让总耗时接近最慢的那个请求，而不是相加。

```
async let a = fetchTitle()
async let b = fetchTitle()
let results = try await [a, b]     // 两个请求并发进行

let ids = [1, 2, 3]

func loadTitle(id: Int) async -> String { "标题 \(id)" }   // 实际的取标题逻辑

let titles = await withTaskGroup(of: String.self) { group in
    for id in ids { group.addTask { await loadTitle(id: id) } }   // 三条同时跑
    var all: [String] = []
    for await t in group { all.append(t) }                        // 谁先完成先收谁
    return all
}
```

怎么读：`TaskGroup` 的用法是"加任务、收结果"两步。`for await` 表示结果是一个接一个到来的异步序列，顺序不保证和添加顺序一致，需要保序就自己按 id 归位。新手不必急着用，看到这段代码知道它在做并发批处理即可。

Swift 6 引入了严格并发检查：跨并发边界传递的数据必须遵循 `Sendable`，否则编译器报错。值类型（结构体、枚举）只要成员都是可发送的，自动就是 `Sendable`；类默认不是，除非你把它声明成 `@MainActor` 或 `final` 且所有属性都不可变。

这对新手意味着什么：以前能编译的代码，在严格模式下可能突然报"捕获了非 Sendable 类型"。最常见的修法不是加各种标记，而是把共享的数据从类换成结构体，或者把它放进 `@MainActor` 的上下文。记住一句话：能复制就别共享。

`actor` 是一段话就能说清的概念：actor 是一种引用类型，它保护自己的可变状态，任何外部访问都必须 `await`，因此同一时刻只有一个任务能改它，天然避免数据竞争。

```
actor Counter {
    private var value = 0
    func increase() -> Int { value += 1; return value }
}

let c = Counter()
await c.increase()      // 跨 actor 访问必须 await
```

> 新手策略：写 UI 相关代码时保持"要么在主线程、要么在 actor 内"的直觉。看到有关并发的报错，先看它有没有提示 `Sendable` 或 `MainActor` 这两个词。

3.10 模块与访问控制

一句话结论：`import` 引入一个模块，访问级别决定"谁能看到这个声明"。

```
import SwiftUI      // 引入 Apple 的框架模块
import Foundation
```

模块（module）就是一次编译产出的整体，一个 App 是一个模块，用 Swift Package Manager 引入的每个包也是一个模块。`import` 之后你才能用里面的公开类型。

访问级别从紧到松如下表。

级别	可见范围	使用建议
private	当前类型内部	90% 的成员都用它
fileprivate	当前文件内	同文件多个类型需要共享时
internal	当前模块内，默认值	不写修饰符就是这个
public	其他模块可见	写库给别人用时
open	其他模块可见且可继承	框架设计者才用

```
struct UserCard: View {
    private let user: User            // 只有 UserCard 内部能用
    var body: some View { Text(user.name) }
}
```

怎么读：`private let user` 表示这个属性只服务于本类型内部，外部想拿也拿不到。这么写不是为了保密，而是为了让读代码的人立刻知道"改它只需要看这一个类型"。

> 新手实践：默认什么都不写（internal）就够用；能加 `private` 的地方尽量加。只有在你把代码打包成库给别的模块用时，才需要考虑 `public` 和 `open`。

有个容易踩的坑：`private` 的可见范围包含"当前类型以及写在同一个文件里的扩展"，所以把类型的一部分实现拆到同文件的 `extension` 里，`private` 成员依然能访问。跨文件的扩展就不行了，这时要么用 `fileprivate`，要么把成员提升为默认的 `internal`。

3.11 Swift 6 带来的变化

一句话结论：Swift 6 最大的变化是把"数据竞争安全"从建议变成了编译期检查，代价是严格模式下容易报一堆并发错误。

主要变化有三点。

1. 严格并发检查默认开启，跨任务传递非 `Sendable` 数据会直接编译失败。

2. 引入了语言模式（language mode）概念，`Swift 5` 模式较宽松，`Swift 6` 模式最严格。

3. 一些旧写法（如全局可变变量、非隔离的单例可变状态）在 Swift 6 模式下不再合法。

常见报错与读法：

- 提到 "is not Sendable"：某个类被跨线程传递了。先把类改成结构体，或给它加 `@MainActor`。
- 提到 "main actor-isolated"：你在非主线程上下文里访问了主线程数据。给这个函数或类型加 `@MainActor`。
- 提到 "mutation of captured var in concurrently-executing code"：闭包并发修改了外部变量。改成在闭包内用局部变量汇总后返回。
- 提到 "static property is not concurrency-safe"：全局或静态可变变量在 Swift 6 下不合法。把它放进一个 `@MainActor` 类型，或者改成 `let`。

下面这个对比能覆盖大半新手报错。

```
// 旧写法：全局可变状态，Swift 6 严格模式下报错
var cachedUser: String?

// 改法一：变成不可变
let appVersion = "1.0"

// 改法二：把可变状态收进受隔离的类型
@MainActor
final class Cache {
    static let shared = Cache()
    var user: String?
}
```

怎么读：改法一说明"能不变就不变"；改法二说明"要变的东西必须有个明确的归属，并且规定它在哪个线程上变"。`@MainActor` 修饰的类，它的静态属性和实例属性都在主线程访问，因此编译器放心了。

> 诚实的建议：刚接手项目时不要急着打开 Swift 6 严格模式。先用宽松模式把项目跑通、看懂代码，再逐步迁移。

需要说明的是，语言模式和编译器版本是两件事。Xcode 27 里的编译器是 Swift 6.4，它默认使用 Swift 5 语言模式来编译你的代码，所以旧代码仍然能直接跑；只有当你把语言模式改成 Swift 6 时，严格并发检查才会变成编译错误。也就是说，"用新编译器"不等于"必须立刻写严格并发代码"。

在 Xcode 中切换语言模式：在 Target 的 Build Settings 里搜索 Swift Language Version，选择 Swift 5 或 Swift 6。如果是 Swift Package Manager 项目，则在 `Package.swift` 里设置对应的 Swift 语言版本。切换之前先确认项目能正常编译，这样一旦爆出并发错误，你能确定是新模式带来的，而不是原来就有问题。

新手应对策略有三条：优先用值类型（结构体）减少共享状态；把涉及 UI 的类型统一加 `@MainActor`；遇到并发报错不要乱加 `nonisolated(unsafe)` 之类的逃生舱，先想清楚"这段数据到底归谁管"。最后一条尤其重要，逃生舱会关掉编译器的检查，等于把数据竞争的风险重新交回到你手上。

3.12 符号速查表：看到这些符号怎么读

一句话结论：这一节按符号查，遇到看不懂的写法直接翻。

符号	怎么读
?	类型后面：可选值，可能是 nil；`try?` 是失败转 nil
!	类型后面：隐式解包；值后面：强制解包，为 nil 会崩溃
??	空合并，读作"如果左边是 nil 就用右边"
?.	可选链，左边为 nil 时整个表达式直接给 nil
->	箭头，左边是参数右边是返回值
$0	闭包的第 0 个参数，`$0`、`$1` 依次类推
_	我不关心这个值，或者这个参数用不到
...	闭区间，`1...5` 包含 5

符号	怎么读
..<	半开区间，`1..<5` 不包含 5
# 加双引号	原始字符串边界，里面的反斜杠不当转义
@	属性或属性包装器标记，如 `@State`
#Preview	声明一个 Xcode 预览
#selector	取出一个 Objective-C 方法引用，老式 target-action 用
反斜杠加括号	字符串插值，把变量值嵌进字符串
[weak self]	闭包捕获列表，弱引用 self 以防循环引用
some	不透明类型，某个确定但匿名类型
any	存在类型，任意遵循该协议的类型
throws	这个函数可能抛错
try	调用可能抛错的函数，必须写
await	等待一个异步调用完成，标记挂起点
async	这个函数是异步的，可以挂起
in	闭包体开始，或者 for-in 循环里的 in
where	附加条件，`where T: Comparable`，也可用于 for 循环筛选
as?	类型转换，失败给 nil
as!	强制类型转换，失败崩溃
is	判断是不是某个类型，返回 Bool
? :	三元运算符，条件 ? 真值 : 假值
&	协议组合读作"并且"；参数前表示 inout 引用传递

3.13 标准库与常用 API 速查

一句话结论：这一节列出你读项目时最常撞见的 API，记住"它大概是干什么的"即可。

API	说明与调用形式
map	逐个转换，`[1,2].map { $0 * 2 }` 得到 `[2,4]`
filter	按条件筛选，`items.filter { $0.isActive }`
reduce	归并成一个值，`nums.reduce(0, +)` 求和
sorted	排序，`names.sorted()`，带参数可自定义比较
first 与 last	取首尾元素，返回可选值，可能要 `??` 兜底
contains	是否包含，`items.contains(where: { $0 > 3 })`
compactMap	转换并丢掉 nil，`strings.compactMap(Int.init)`

API	说明与调用形式
Dictionary	键值表，`dict["key"]` 取值返回可选值
String	字符串，`s.count`、`s.isEmpty`、`s.hasPrefix("a")`
URLSession.data	异步请求，`try await URLSession.shared.data(from: url)`
JSONDecoder	JSON 转模型，`try decoder.decode(User.self, from: data)`
Date 与 Calendar	时间，`Date()`、`Calendar.current.dateComponents(...)`
print	调试输出，`print("值: \(x)")`，多值用逗号分隔
UserDefaults	轻量持久化，`UserDefaults.standard.bool(forKey: "flag")`
Bundle.main	读 App 内置资源，`Bundle.main.url(forResource: "data", withExtension: "json")`

怎么用这张表：读代码时先认出方法名属于哪一类——`map`、`filter`、`reduce` 属于集合的高阶函数，意思是"对每个元素做点什么"；`URLSession`、`JSONDecoder` 属于网络与解析；`UserDefaults`、`Bundle` 属于存取数据。认出类别比记住参数更重要，具体签名去 Xcode 里按住 Option 点方法名就能看到。下面给两个组合起来的小段，都是项目里的常见形态。

```
let names = ["小明", "小红", "小刚"]
let short = names.filter { $0.count <= 2 }.map(\.count)   // 先筛再转换

let data = try JSONDecoder().decode([Repo].self, from: rawData)   // 数据转模型数组
let pinned = UserDefaults.standard.string(forKey: "pinnedRepo")
```

3.14 读代码练习（含答案）

题目：每段代码问"这段代码在做什么"或"输出是什么"。先自己读一遍，再看答案。

第一题。

```
enum Status { case ok, failed(String) }

let list: [Status] = [.ok, .failed("超时"), .ok]
let fails = list.filter {
    if case .failed = $0 { return true }
    return false
}
print(fails.count)
```

答案与解析：输出 1。`filter` 只保留满足条件的元素；`if case .failed = $0` 是"模式匹配判断"，不管关联值是什么内容都算匹配。这是 `switch` 的简化写法，常用于列表筛出某类状态。

第二题。

```
struct Item { var count = 0 }
var a = Item()
var b = a
b.count = 5
print(a.count, b.count)
```

答案与解析：输出 `0 5`。`Item` 是结构体，`b = a` 复制了一份，改 `b` 不影响 `a`。如果把 `struct` 换成 `class`，输出就会是 `5 5`。

第三题。

```
protocol Greeter {
    var name: String { get }
}

extension Greeter {
    func greet() -> String { "你好，\(name)" }
}

struct Student: Greeter { let name: String }

print(Student(name: "小明").greet())
```

答案与解析：输出"你好，小明"。`Student` 只提供了协议要求的 `name`，`greet()` 用的是协议扩展里的默认实现。这解释了为什么有些类型看起来没写某个方法却能调用它——去协议扩展里找。

第四题。

```
func firstMatch<T: Equatable>(_ target: T, in list: [T]) -> T? {
    list.first { $0 == target }
}

print(firstMatch(3, in: [1, 2, 3, 4]) ?? -1)
print(firstMatch("z", in: ["a", "b"]) ?? "没找到")
```

答案与解析：输出 `3` 和 `没找到`。`T: Equatable` 是类型约束，保证 `==` 可用；同一个泛型函数被推断成 `Int` 和 `String` 两个版本。`first` 返回可选值，所以用 `??` 兜底。

第五题。

```
final class Loader {
    var onDone: (() -> Void)?

    func start() {
        onDone = { self.finish() }
    }

    func finish() { print("done") }

    deinit { print("释放了") }
}

var loader: Loader? = Loader()
loader?.start()
loader = nil
```

答案与解析：只打印一行 done，最后不会打印 释放了。onDone 是 self 的属性，闭包里又强引用了 self，形成循环引用，所以对象永远不会被释放，deinit 也就永远不执行。正确写法是 onDone = { [weak self] in self?.finish() }。

第六题。

```
@Observable
final class RepoModel {
    var repos: [String] = []
    @MainActor func load() async {
        try? await Task.sleep(for: .seconds(1))
        repos = ["swift", "ios"]
    }
}

struct RepoView: View {
    @State private var model = RepoModel()
    var body: some View {
        List(model.repos, id: \.self) { Text($0) }
            .task { await model.load() }
    }
}
```

答案与解析：这是一个 iOS 17+ 风格的"数据模型加列表视图"。@Observable 让 repos 变化时视图自动刷新；@MainActor 保证 repos 在主线程写入；.task 在视图出现时启动一次异步加载，视图消失时自动取消。读到这种结构，你只要抓住"状态在 model 里、视图读 model、task 负责触发"这条线就够了。

到这里，Swift 语言的语法层面基本齐了。下一章进入工程与 SwiftUI 视图层，把这些语法拼成能跑的 App。

第 4 章 iOS App 开发基本过程（12 步）

4.1 全景流程图

从一台干净的 Mac 到一个能装在别人手机上的 App，中间会经过 12 个环节。它们不是线性的：第 5 步到第 9 步会反复来回，改一行视图、调一次状态、加一个接口，然后再跑一遍。下面这张图给出的是主干顺序。



每一步都有明确产出物。产出物是判断"这一步做完了没有"的唯一依据，不要凭感觉往下走。

阶段	做什么	完成标志
1 准备环境	装 Xcode 与命令行工具、准备模拟器	终端能打印 Xcode 与 Swift 版本
2 创建工程	用 iOS App 模板生成项目	模拟器里跑出 Hello World
3 目录结构	认识每个文件是谁生成的	能说清哪些文件要进 Git
4 骨架与生命周期	理解 @main、App、Scene、View	能画出从启动到出界面的路径

阶段	做什么	完成标志
5 视图与布局	用 SwiftUI 拼出目标界面	模拟器和预览里外观一致
6 状态管理	决定数据放哪一层、谁能改	数据变化时界面自动刷新
7 网络	请求接口、解析 JSON、渲染列表	断网有提示，有数据能出列表
8 持久化	把需要留存的数据写进磁盘	杀掉 App 重开数据还在
9 第三方依赖	判断要不要引库、怎么引	Package.resolved 进版本库
10 调试与预览	会打断点、会看崩溃日志	能独立定位一个崩溃到具体行
11 真机与签名	配好证书、装到自己手机上	手机桌面出现 App 图标并能启动
12 发布	归档、上传、TestFlight 测试	测试员能在手机上装到构建版本

> 结论：新手最容易在第 7 步和第 11 步卡住。前者是异步与数据流不熟，后者是签名概念不清。遇到问题先回到"产出物"这一列，确认上一环是否真的完成了。

下面 12 个小节逐一展开。第 5 章会拿一个完整项目把这些步骤从头到尾串一遍，本章的任务是把每一步讲清楚。

4.2 第 1 步：准备开发环境

iOS 开发对硬件有硬性要求：**必须是 Mac**。原因不是苹果故意限制，而是编译工具链（Xcode、clang、Swift 编译器、iOS SDK）只在 macOS 上提供，签名工具也只认 macOS 的钥匙串（Keychain）。本章以 macOS + Xcode 27.0 + Swift 6.4 + iOS 27 SDK 为基准环境。

常见替代方案的真实代价：

- 黑苹果：系统更新容易失败，Xcode 升级可能直接装不上，模拟器与真机调试报错难以排查。省下的钱会用大量时间还回去。
- 虚拟机装 macOS：性能打折，模拟器运行卡顿，真机调试常常连不上 USB 设备。
- 租用云端 Mac：可以作为补充（比如团队统一构建环境），但本机没有 Mac 时开发体验很差。

Xcode 从 Mac App Store 安装即可，搜索 Xcode，注意认准 Apple 官方发布者，别装到同名的山寨应用。安装包很大（十几 GB 量级），磁盘建议预留 40 GB 以上，因为还要装模拟器运行时和构建缓存。

首次启动 Xcode 时会有一个安装附加组件的提示，需要输入管理员密码，这一步是在装编译器和平台支持组件，必须走完。如果跳过或中途失败，后面编译会报找不到 SDK 之类的错误。不同 Xcode 版本的提示文案和位置可能略有差异，遇到弹窗一律选"安装"即可。

命令行工具可以单独确认和安装：

```
# 查看当前命令行工具指向的路径，正常应指向 Xcode 内的 Developer 目录
xcode-select -p

# 如果没装或指向错误，执行安装或切换
xcode-select --install
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer

# 确认 Xcode 版本与构建版本
xcodebuild -version

# 确认 Swift 编译器版本
swift --version
```

`xcode-select` 的作用是告诉系统"命令行工具应该用哪一个 Developer 目录"。macOS 上可能同时存在多个 Xcode，或者残留过旧版本，这时 `xcodebuild` 会用错 SDK 并抛出难以理解的编译错误。`xcode-select -s` 就是把指向纠正回正确的那个目录。

模拟器（Simulator）是 Xcode 自带的独立 App，但它需要"运行时"（Runtime）才能启动某代 iOS。Simulator 本身只是一个"外壳"，里面跑的系统是运行时。运行时按 iOS 版本分别下载，所以在 Xcode 设置里可以同时准备多个版本的运行时，用不同设备验证兼容性。这一步很占磁盘，一个运行时通常几 GB，装两三个版本是常见做法。下载位置在不同版本里可能略有差异。查看本机已有哪些模拟器设备：

```
# 列出所有模拟器设备及其状态
xcrun simctl list devices

# 只列出可用运行时
xcrun simctl list runtimes
```

Apple ID 的作用有三层：登录 Xcode 后可以下载工具与文档；可以用免费的个人团队（Personal Team）给真机装测试证书；如果要上架 App Store，则需要加入 Apple Developer Program（付费开发者账号，每年 99 美元）。

免费账号与付费账号的差别，直接决定你能做哪些事：

对比项	免费 Apple ID 个人团队	付费开发者账号
费用	免费	每年 99 美元

对比项	免费 Apple ID 个人团队	付费开发者账号
真机调试	可以，但证书 7 天过期，需要重新构建	可以，证书有效期长得多
推送通知	不支持	支持
App Store 上架	不支持	支持
TestFlight	不支持	支持
团队协作	基本不支持多人证书管理	支持多角色与多设备
可注册设备数	少量	每年 100 台

> 提示：学习阶段用免费账号完全够用，只是每 7 天要重新跑一次到手机上。不要把"证书过期"当成代码问题，它只是免费账号的机制。

环境装好之后，先别急着写代码。可以随便找个已有工程编译一次，确认整条链路能跑通。环境阶段最常踩的坑有三个：同时装了多个 Xcode 导致命令行工具指向不对；磁盘空间不足导致归档或下载运行时中途失败；网络或代理异常导致 SDK 组件下载不完整。这三个问题的共同特征是报错信息与实际原因不直接对应，所以排查时先确认环境本身是干净的。

4.3 第 2 步：创建第一个 Xcode 工程

在 Xcode 中新建工程时选择 **iOS > App 模板**（入口在欢迎窗口或菜单栏的 File > New > Project，具体位置不同版本可能略有差异）。随后会有一个选项弹窗，这些选项必须逐个看懂，因为它们决定了工程从一开始长什么样。

- Product Name：产品名，也是默认的工程名和目标名（Target 名）。用英文，不要空格，例如 `HelloiOS`。
- Team：签名团队。没登录 Apple ID 时这里是空的，可以先留空，等第 11 步真机调试时再选。
- Organization Identifier：组织标识，反向域名写法，例如 `com.bingyan`。它加上 Product Name 就组成了 Bundle Identifier。
- Bundle Identifier（包标识符，Bundle ID）：默认形如 `com.bingyan.HelloiOS`。规则是只允许字母、数字、连字符和点，通常用反向域名；**它必须在整个 App Store 里全局唯一**，别人占用了你就不能再提交。所以正式项目里会认真设计，例如 `com.bingyan.helloios`。
- Interface：界面框架。选 **SwiftUI**，这是现在唯一的主线选择。

- Language：语言。选 **Swift**。
- Storage：持久化方案。三个选项的含义是 None（不生成持久化代码）、SwiftData（生成 SwiftData 模型相关代码）、Core Data（生成老的 Core Data 栈）。本章和第 5 章都用 SwiftData，但如果你只想先看最干净的项目，选 None 也完全没问题。
- Include Tests：是否生成测试目录。建议勾选，生成单元测试与 UI 测试两个 Target，后面加测试时不用重新配置。

这些选项里只有三个改起来麻烦：Product Name、Organization Identifier 和 Storage。前两个改动会连带影响 Bundle ID，已经上传过 App Store 的包不能再改；Storage 一旦生成代码就有了一套数据结构，换方案要重写模型层。其余选项（Team、Tests）都可以随时在工程设置里补。所以填错不用慌，但要趁早上传之前改完。

Team 这一项现在留空是可以的：模拟器不需要签名，所以不影响第一次运行。等你做真机调试时再回到目标的签名设置里选团队，那时 Xcode 会要求你登录 Apple ID。

创建完成后，Xcode 会打开工程，并默认停在入口文件上。此时不写任何代码，直接第一次运行：

1. 在窗口顶部的 Scheme 选择区旁边选一个模拟器，例如某个 iPhone 型号。
2. 点击运行按钮（Run），或按快捷键 Command + R。
3. 等待编译完成，模拟器自动启动并显示一个写着 Hello, world! 的界面。

看到这一行字，说明工具链、SDK、模拟器、签名（模拟器不需要真机证书）整条链路都是通的。第一次编译明显慢于之后，因为要生成整套构建缓存；如果始终卡在构建阶段，优先怀疑索引或缓存问题，而不是代码问题。

模拟器和预览（Preview）的分工要提前分清：模拟器里是一个完整的 iOS 系统，能验证键盘、通知、多任务、深色模式这些真实行为，是判断"功能对不对"的地方；预览只是写界面时的快速反馈，方便调间距和配色，不能替代模拟器验证。很多人只在预览里看着正常就以为做完了，最后在模拟器上发现导航或生命周期有问题，返工成本更高。

模板生成的入口代码大致长这样，先不用改，第 5 步会解释每一行：

```
import SwiftUI

@main
struct HelloiOSApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

> 提示：真机运行需要签名，配置不对会连编译都过不去。所以第一次跑通请务必先在模拟器上完成，把"签名问题"和"代码问题"分开排查。

4.4 第 3 步：读懂工程目录结构

新建的 SwiftUI 工程看起来文件不多，但每个都有明确职责。下面是一个典型的目录树（名称随工程名变化）：

```
HelloiOS/
├── HelloiOS.xcodeproj/          # 工程文件，Xcode 用它组织一切
│   └── project.pbxproj          # 工程配置的实体，纯文本
├── HelloiOS/
│   ├── HelloiOSApp.swift        # 带 @main 的入口文件
│   ├── ContentView.swift        # 模板给的第一个视图
│   ├── Assets.xcassets/         # 图片、颜色、App 图标等资源
│   └── Preview Content/         # 只给预览用的资源
├── HelloiOSTests/               # 单元测试
└── HelloiOSTestsUITests/        # 界面测试
```

逐个说明：

- **App 入口文件**：文件里有一个带 `@main` 标记的结构体，它遵循 `App` 协议。程序从这里开始执行。文件名模板会按工程名生成，叫 `HelloiOSApp.swift`。
- **ContentView.swift**：模板生成的第一个视图，遵循 `View` 协议。可以把它当成"首页"的起点，后续拆分成多个文件。
- **Assets.xcassets**：资源目录（Asset Catalog）。App 图标、启动图、自定义颜色、图片素材都放这里。它的好处是自动适配深色模式与不同分辨率。
- **Preview Content**：只服务于 Xcode 预览的素材，比如给预览用的示例图片。它不会被打进正式包，所以可以放一些只在设计阶段用的东西。
- **Info.plist 的作用**：记录 App 的元信息，例如支持的设备方向、App 显示名称、权限用途说明文案。现代 Xcode 里很多键由构建设置自动生成（Generate Info.plist File），所以你在模板工程里可能搜不到这个文件，它会出现在构建产物里。需要加权限说明时，通常是在目标的 Info 页签里添加。

- **Tests 与 UITests 目录**：单元测试验证逻辑（函数输入输出），界面测试模拟用户点击。两者用途不同，命名上就能看出来。
- **.xcodeproj 与 project.pbxproj**：`.xcodeproj` 是个目录，`project.pbxproj` 是纯文本文件，记录了所有文件引用、编译顺序、构建设置、依赖。因为它是文本，冲突时 Git 无法自动合并，谁都改过、谁该保留往往要靠人对着 diff 判断，所以团队协作时它会成为冲突高发区。缓解办法是：一个功能一个分支、合并前先同步主干、尽量避免多人同时新增文件。

关于 Info.plist 还有一点要知道：当你需要相机、定位、相册这类权限时，必须在 Info 里补上对应的用途说明字符串。系统弹窗会原样展示这段话，没有它 App 会直接被拒绝访问权限，甚至启动即崩溃。用途说明要写清"用来做什么、数据怎么处理"，不要只写"需要此权限"。App Transport Security 的例外配置也在这里，用来给个别 http 域名开口子。

资源目录也不只是放图片：自定义品牌色可以定义在 Assets.xcassets 里，代码用 `Color("brandBlue")` 引用，这样同一份颜色能自动适配深色模式，不需要在代码里到处写十六进制值；App 图标也是在资源目录里按尺寸配置，第 12 步发布时会用到。

哪些进 Git、哪些不进，是有明确答案的：

文件或目录	是否提交	原因
源代码与 Assets.xcassets	提交	项目本体
.xcodeproj 与 project.pbxproj	提交	缺少它工程结构就丢了
xcuserdata	不提交	记录的是你个人的窗口布局与断点
DerivedData	不提交	构建产物缓存，可重建
.DS_Store	不提交	macOS 生成的目录元数据
Package.resolved	提交	锁定依赖版本，保证构建一致

提交前用 `git status` 看一眼将要提交的文件列表，是成本最低的一道保险。`.gitignore` 只能拦住规则匹配到的路径，`xcuserdata` 这类目录经常在第一次提交时就被误加进去，之后再删会留下历史记录，清理起来很麻烦。另外要知道 DerivedData 不在工程目录里（默认在用户资源库下的 Developer 目录），所以它不会因为删掉工程而消失；机器变慢或构建行为异常时，清理派生数据再重建是常规操作，代价只是下一次编译变慢。

4.5 第 4 步：理解 App 的骨架与生命周期

一个新手的困惑往往是："程序到底从哪一行开始跑？"在 SwiftUI 里答案很整齐，四个概念串成一条线：`@main` → `App` → `Scene` → `View`。

- `@main`：标记整个程序的入口类型，一个工程只能有一个。它是编译器识别的属性，不是你自己写的调用。
- `App` 协议：描述"这个 App 由哪些场景组成"，必须实现 `body`，返回一个 `Scene`。
- `Scene`：界面的容器，管理一组窗口。`WindowGroup` 是最常用的场景类型，支持多窗口，在单窗口设备上表现为一个窗口。
- `View` 协议：描述"界面长什么样"，必须实现 `body`，返回另一个 `View`。

最小骨架：

```
import SwiftUI

@main
struct HelloiOSApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()       // 场景里的根视图
        }
    }
}

struct ContentView: View {
    var body: some View {
        Text("Hello, world!")
    }
}
```

这段代码怎么读：系统启动时找到带 `@main` 的类型，实例化它，读取 `body` 得到 `WindowGroup`，`WindowGroup` 再实例化里面写的根视图，最终把 `ContentView.body` 描述的内容画到屏幕上。

为什么要分成 App 和 Scene 两层？因为一个 App 可能有不止一组界面。`App` 描述这个 App 由哪些场景组成，`Scene` 描述一组窗口的生命周期。`WindowGroup` 表示"可以开多个同类型窗口"，在 iPhone 上通常只开一个，在 iPad 或 Mac 上可以开多个。多窗口的创建、关闭、状态隔离都由 SwiftUI 负责，你写代码时只需要关心场景里的内容。

`App` 结构体还有一个常见用途：持有全局状态并注入环境，4.7 演示 `@Environment` 时的 `AppState` 就放在这一层。它天然是整个程序里创建最早、销毁最晚的对象，适合做全局唯一的容器。

生命周期指的是 App 在系统中的状态迁移：**未运行 → 前台活跃 → 前台非活跃 → 后台 → 被系统回收**。用户切到别的 App 会进入后台，锁屏或下拉通知中心会进入非活跃，系统内存紧张会回收后台进程。

三个状态各自意味着什么：`.active` 表示界面可见且可交互；`.inactive` 表示界面可能还看得见，但用户交互被系统接管（比如正在切换 App），此时不要启动动画或播放视频；`.background` 表示已经不可见，系统给的时间很短，几秒后就会挂起进程，所以要在这一小段时间里把该保存的保存完。

SwiftUI 用环境值（Environment Value）`scenePhase` 让你观察这些变化：

```
import SwiftUI

struct ContentView: View {
    @Environment(\.scenePhase) private var scenePhase
    @State private var lastEvent = "尚未记录"

    var body: some View {
        VStack(spacing: 12) {
            Text("当前状态：\(describe(scenePhase))")
            Text(lastEvent).font(.footnote).foregroundStyle(.secondary)
        }
        .onChange(of: scenePhase) { _, newPhase in
            // 只在需要保存草稿或停掉定时器时才写这类代码
            if newPhase == .background {
                lastEvent = "已进入后台，可以在这里保存草稿"
            }
        }
    }

    private func describe(_ phase: ScenePhase) -> String {
        switch phase {
        case .active: return "前台活跃"
        case .inactive: return "前台非活跃"
        case .background: return "后台"
        @unknown default: return "未知"
        }
    }
}
```

`onChange` 的第一个参数是要观察的值，第二个是变化后的新值，旧的用 `_` 忽略。`@unknown default` 是为了将来系统新增状态时不至于编译不过。

几条实践原则：不要在后台开始时启动动画或发起长任务，系统会直接挂起你；不要试图阻止 App 进入后台，这既做不到也会被审核拒绝；`scenePhase` 观察的是场景整体的状态，不是某个具体视图的出现与消失。单个视图的 appeared 与 disappeared 用修饰符单独处理，两者不要混着用。

> 结论：你几乎不需要手写生命周期代码。SwiftUI 会自动处理界面创建与销毁，只有"进入后台要保存数据""回到前台要刷新数据"这类真正的业务需求，才需要碰 `scenePhase`。把这类逻辑集中在一两个地方，不要散落在每个视图里。

4.6 第 5 步：SwiftUI 视图与布局

一句话结论：SwiftUI 的界面是**用结构体描述出来的**，不是画出来的。视图都是遵循 `View` 协议的结构体，界面内容写在计算属性 `body` 里。

修饰符（Modifier）是 SwiftUI 的核心机制：每个修饰符都返回一个新的视图，所以可以链式调用。**顺序会直接改变结果**，因为每个修饰符都在包装上一层的结果。

```
Text("冰岩移动组")
    .padding()                          // 先加内边距
    .background(.yellow)                // 再给这个更大的区域上色
// 与下面写法效果完全不同：
Text("冰岩移动组")
    .background(.yellow)                // 先给文字上色
    .padding()                          // 再加一圈透明边距
```

布局容器决定子视图怎么排列：

```
VStack(alignment: .leading, spacing: 8) {   // 垂直排列，左对齐，间距 8
    Text("标题").font(.title)
    HStack(spacing: 6) {                     // 水平排列
        Image(systemName: "star.fill")
        Text("收藏")
    }
    ZStack {                                 // 层叠，后写的盖在上面
        RoundedRectangle(cornerRadius: 12).fill(.blue.opacity(0.2))
        Text("层叠内容")
    }
    .frame(height: 80)
    Spacer()                                 // 占满剩余空间，用于把内容顶开
}
.padding()
```

常用修饰符汇总：

修饰符	作用	常见用法
padding	内边距	padding() 或 padding(.horizontal, 16)
frame	尺寸约束	frame(width:height:) 或 frame(maxWidth: .infinity)
background	背景	background(.gray.opacity(0.1))
foregroundStyle	前景色	foregroundStyle(.secondary)
font	字体	font(.headline)、font(.system(size: 18))

修饰符	作用	常见用法
cornerRadius / clipShape	圆角与裁剪	clipShape(RoundedRectangle(cornerRadius: 12))
opacity	透明度	opacity(0.6)
shadow	阴影	shadow(radius: 4)

图片、控件与列表的常用写法：

```
Image(systemName: "bolt.fill")        // SF Symbols 系统图标，直接写符号名
    .font(.largeTitle)
    .foregroundStyle(.orange)
Image("logo")                          // Assets 里的图片，写资源名
    .resizable()
    .scaledToFit()

Button("提交") { submit() }            // 点击执行闭包
    .buttonStyle(.borderedProminent)

@State private var agreed = false
Toggle("同意用户协议", isOn: $agreed)   // $ 取出绑定，见 4.7

TextField("请输入昵称", text: $nickname)
    .textFieldStyle(.roundedBorder)
```

列表与滚动是最常见的界面结构。`List` 自带滚动、分隔线和单元格复用，`ForEach` 负责把集合展开成一组视图；需要自定义布局时用 `ScrollView`。

```
List(items) { item in
    HStack {
        Image(systemName: item.icon)
        Text(item.title)
    }
}
// 或者自己控制可滚动区域
ScrollView {
    LazyVStack {                  // Lazy 表示按需创建，长列表更省内存
        ForEach(items) { item in
            Text(item.title)
        }
    }
}
```

`List` 和 `ForEach` 里的元素必须能唯一标识：数据遵循 `Identifiable` 协议，或者显式写 `id: \.self`。

导航、弹层与标签页对应三种不同层级的跳转：

```
NavigationStack {                           // 导航容器，替代老的 NavigationView
    List(items) { item in
        NavigationLink(item.title) { DetailView(item: item) }
    }
    .navigationTitle("列表")
    .sheet(isPresented: $showingSheet) {    // 从底部弹出的模态页
        Text("这是一个 sheet")
    }
}

TabView {                                   // 底部标签栏
    HomeView().tabItem { Label("首页", systemImage: "house") }
    MineView().tabItem { Label("我的", systemImage: "person") }
}
```

新式导航用值驱动，跳转目标由 `navigationDestination` 统一声明：

```
@State private var path: [Item] = []

NavigationStack(path: $path) {
    List(items) { item in
        Button(item.title) { path.append(item) }
    }
    .navigationDestination(for: Item.self) { item in
        DetailView(item: item)      // 类型对上才走这里
    }
}
```

当 `body` 变长，不要一直往下堆，把一块界面抽成独立的结构体。SwiftUI 的容器之所以能接收多个子视图，靠的是 `@ViewBuilder`：它把闭包里的多个视图自动组合成一个。需要根据条件显示不同内容时，`if` 和 `switch` 都能直接写在容器里。

```
struct StatusBadge: View {
    let isOnline: Bool

    var body: some View {
        Text(isOnline ? "在线" : "离线")
            .font(.caption)
            .padding(.horizontal, 8)
            .padding(.vertical, 4)
            .background(isOnline ? .green.opacity(0.2) : .gray.opacity(0.2))
            .clipShape(Capsule())      // 胶囊形，圆角半径自动等于高度一半
    }
}
```

拆成小视图除了可读，还有性能意义：一个结构体就是一个独立的重新求值单元，SwiftUI 能更准确地判断改动影响了哪一块。全写在一个巨大的 `body` 里，任何状态变化都会让整个界面重新求值。

> 提示：`NavigationView`、`PreviewProvider` 这类写法你在老项目里一定会见到，它们已经过时。新代码统一用 `NavigationStack` 和 `#Preview`。

4.7 第 6 步：状态管理

状态管理是新手最容易混乱的地方。混乱的根源是一个问题没想清楚：**这份数据属于谁？** 想清楚这一点，选哪个属性包装器就是自然结果。

先看对比表：

写法	数据属于谁	谁可以改	什么时候用
@State	当前视图私有	只有本视图	视图内部的临时状态：输入框内容、是否展开
@Binding	别人（父视图）持有	通过绑定改父视图的数据	子视图要修改父视图的状态
@Observable + @State	当前视图持有模型对象	本视图及其子视图	视图自己创建并独占的数据模型
@Observable + .environment / @Environment	上层注入，全局共享	拿到对象的任何视图	跨多层共享、整个 App 级别的状态
@Environment	系统或上层环境	只读（除非是绑定）	读取系统信息：颜色模式、场景状态、尺寸类
@AppStorage	UserDefaults	读到的地方都能改	少量用户偏好：开关、主题、上次选中项
@StateObject / @ObservedObject	视图持有或外部传入	视图中可改	iOS 17 之前的老写法，新代码不要用

逐条拆开看。

@State 与 @Binding：父子联动。 `@State` 声明视图自己的状态，值变化时视图重新求值。子视图不能直接改父视图的 `@State`，需要用 `@Binding` 接收一份可读可写的引用。

```
struct ParentView: View {
    @State private var nickname = ""      // 数据真正存在父视图里

    var body: some View {
        VStack {
            Text("昵称：\(nickname)")
            NameField(text: $nickname)     // 用 $ 交出绑定
        }
    }
}

struct NameField: View {
    @Binding var text: String             // 不持有数据，只借用

    var body: some View {
        TextField("输入昵称", text: $text)
```

```
    }
}
```

@Observable 作为数据模型。 iOS 17 起用 Observation 框架：类上加 `@Observable`，属性就是可观察的，不需要 `@Published`。视图用 `@State` 持有它。

```
import Observation

@Observable
final class CounterModel {
    var count = 0
    var isEven: Bool { count % 2 == 0 }    // 计算属性自动跟随

    func increase() { count += 1 }
}

struct CounterView: View {
    @State private var model = CounterModel()   // 视图独占这个模型

    var body: some View {
        VStack {
            Text("计数：\(model.count)")
            Button("加一") { model.increase() }
        }
    }
}
```

跨层共享：.environment 注入。 当多个不相邻的视图都要用同一份数据时，从上层注入、下层用 `@Environment` 取出，避免把参数一层层往下传。

```
@Observable
final class AppState {
    var currentUser: String = "未登录"
}

@main
struct DemoApp: App {
    @State private var appState = AppState()

    var body: some Scene {
        WindowGroup {
            RootView().environment(appState)     // 注入到环境
        }
    }
}

struct ProfileView: View {
    @Environment(AppState.self) private var appState   // 按类型取出

    var body: some View {
        Text("当前用户：\(appState.currentUser)")
    }
}
```

`@AppStorage` 适合开关这类小配置，它直接读写 UserDefaults，4.9 会展开。

判断"状态放哪一层"，按这几条走：

1. 只有一个视图用，就放这个视图的 `@State`。

2. 父子两个视图用，父视图持有 `@State`，子视图用 `@Binding`。

3. 一个功能模块内多个视图用，做一个 `@Observable` 模型，在模块根部用 `@State` 持有。

4. 跨模块、全局都要用（登录态、主题），用 `.environment` 注入。

5. 需要重启后还在的小配置，用 `@AppStorage`。

> 概念：单向数据流。状态只从上层流向下层，下层想改数据必须通过绑定或模型方法往上回传。这让你永远能回答"这个值是谁改的"，从而避免界面和数据不一致。

新手在状态上踩的坑高度集中在三类：

- **数据莫名其妙重置。** 在子视图里写了 `@State`，而这个子视图会被反复创建，于是它每次都拿到初始值。判定方法很直接：问这份数据在逻辑上属于谁。属于父视图的，就不要在子视图里用 `@State` 存档。
- **以为改了就等于界面会刷新。** 普通存储属性不会触发刷新，只有 `@State`、`@Binding`、`@Observable` 模型的属性、`@AppStorage` 才有这个能力。
- **在错误的层级加包装器。** `@State`、`@Binding`、`@Environment` 是视图侧的属性包装器；`@Observable` 加在数据模型类上，不是加在视图属性上。两者位置搞反是编译错误的常见来源。

另外，从 `@Observable` 模型取绑定用 `@Bindable`。当你把一个模型属性直接接到 `TextField` 或 `Toggle` 上时，需要双向绑定：

```
struct EditorView: View {
    @Bindable var model: DraftModel      // 由外部传入的 @Observable 模型

    var body: some View {
        Form {
            TextField("标题", text: $model.title)
            Toggle("已完成", isOn: $model.isDone)
        }
    }
}
```

> 提示：`ObservableObject` 配合 `@StateObject` / `@ObservedObject` / `@Published` 是 iOS 17 之前的标准做法。你在老项目里会大量见到，维护时要能看懂，但新代码用 `@Observable`。

4.8 第 7 步：网络请求与数据解析

现代写法是 `URLSession` 配合 `async/await`。一次请求的完整链条是：构造 URL → 构造请求（方法和请求头）→ 发出请求拿到数据和响应 → 校验状态码 → 解析 JSON。

定义数据模型用 `Codable`（可编解码），它由 `Encodable` 和 `Decodable` 合成。字段名和 JSON 不一致时，用 `CodingKeys` 做映射。

```
struct Post: Codable, Identifiable {
    let id: Int
    let userId: Int
    let title: String
    let body: String
}

// 如果 JSON 用下划线风格（created_at），两种处理方式：
struct Article: Codable {
    let createdAt: String

    enum CodingKeys: String, CodingKey {
        case createdAt = "created_at"     // 逐字段映射，最明确
    }
}

// 或者交给解码器统一转换，适合大量同风格字段
let decoder = JSONDecoder()
decoder.keyDecodingStrategy = .convertFromSnakeCase
```

请求本身写成一个 `async` 函数，用 `throws` 把错误交给调用方：

```
enum NetworkError: Error {
    case badStatus(Int)
    case decodingFailed
}

func fetchPosts() async throws -> [Post] {
    let url = URL(string: "https://jsonplaceholder.typicode.com/posts")!
    var request = URLRequest(url: url)
    request.httpMethod = "GET"
    request.setValue("application/json", forHTTPHeaderField: "Accept")

    let (data, response) = try await URLSession.shared.data(for: request)
    guard let http = response as? HTTPURLResponse else {
        throw NetworkError.badStatus(-1)
    }
    guard (200..<300).contains(http.statusCode) else {
        throw NetworkError.badStatus(http.statusCode)   // 把状态码带出去便于排查
    }
    do {
        return try JSONDecoder().decode([Post].self, from: data)
    } catch {
        throw NetworkError.decodingFailed
    }
}
```

界面需要有四种状态：初始、加载中、成功、失败。用一个枚举表达，比三个布尔变量清晰得多：

```
enum LoadState {
    case idle
    case loading
    case success([Post])
    case failure(String)
}

@MainActor
@Observable
final class PostListModel {
```

```
    var state: LoadState = .idle

    func load() async {
        state = .loading
        do {
            state = .success(try await fetchPosts())
        } catch {
            state = .failure("加载失败，请检查网络后重试")
        }
    }
}
```

`@MainActor` 标记这个类只能在主线程访问，因此它内部改 `state` 一定是安全的。SwiftUI 的界面更新必须在主线程，这个标记就是给编译器的保证。

视图用 `.task` 在出现时触发请求，不用自己写"页面已加载"的回调：

```
struct PostListView: View {
    @State private var model = PostListModel()

    var body: some View {
        NavigationStack {
            Group {
                switch model.state {
                case .idle, .loading:
                    ProgressView("加载中")
                case .success(let posts):
                    List(posts) { post in
                        VStack(alignment: .leading, spacing: 4) {
                            Text(post.title).font(.headline)
                            Text(post.body).font(.footnote).foregroundStyle(.secondary)
                        }
                    }
                case .failure(let message):
                    Text(message).foregroundStyle(.red)
                }
            }
            .navigationTitle("文章")
        }
        .task { await model.load() }     // 视图出现时执行一次，消失时自动取消
    }
}
```

`.task` 的生命周期跟着视图走：视图消失时任务会被取消，这既是优点也是陷阱——如果你在请求尚未返回时快速退出页面，取消是正确行为，不要试图绕过它。需要用户主动下拉刷新时，加 `.refreshable` 即可：

```
List(posts) { post in
    Text(post.title)
}
.refreshable { await model.load() }
```

示例用的是 `jsonplaceholder.typicode.com`，它提供公开的测试数据、不需要申请密钥，适合练手。

解码失败分三类，报错信息会直接告诉你类型：`keyNotFound` 是模型里有 JSON 没有的字段，`typeMismatch` 是类型不同（比如你想要 String 但 JSON 给的是数字），`dataCorrupted` 是数据本身不是合法 JSON。先看是哪一个，再去看对应的字段，比盲目改模型快得多。

工作层的一条纪律：网络函数只负责"拿到数据或抛出错误"，不碰界面；状态枚举和 `@MainActor` 的模型负责把结果翻译成界面状态。这样网络代码可以单独写测试，也不会出现"忘了回主线程"的问题。

> 提示：App Transport Security 默认只允许 HTTPS 连接，明文 http 请求会被直接拦下。如果接口只提供 http，必须在 Info 的权限配置里为对应域名单独开例外。练手时请优先找 HTTPS 接口，不要随手关掉全局限制。

4.9 第 8 步：数据持久化

持久化就是"把数据写到磁盘，下次启动还能读回来"。三条路线的定位完全不同：

路线	适合存什么	优点	局限
UserDefaults 与 @AppStorage	少量键值配置	一行代码读写，最简单	只适合小数据，不适合列表与关系
SwiftData	结构化对象与关系	自动建表、支持查询与迁移	需要定义模型，有学习成本
文件与 JSON	自定义格式、导出导入	完全可控，便于和其他系统交换	查询和并发要自己处理

小配置首选 @AppStorage，它把 UserDefaults 包装成属性，值一变界面自动刷新：

```
struct SettingsView: View {
    @AppStorage("isDarkMode") private var isDarkMode = false
    @AppStorage("launchCount") private var launchCount = 0

    var body: some View {
        Form {
            Toggle("深色模式", isOn: $isDarkMode)
            Text("启动次数：\(launchCount)")
        }
        .onAppear { launchCount += 1 }
    }
}
```

结构化数据首选 SwiftData。 定义模型用 `@Model`，在 App 层挂上 `.modelContainer`，视图里用 `@Query` 直接取数据。不需要写 SQL，也不需要手写增删改查的脚手架。

```
import SwiftData

@Model
final class Todo {
    var title: String
    var isDone: Bool
    var createdAt: Date

    init(title: String, isDone: Bool = false, createdAt: Date = .now) {
        self.title = title
        self.isDone = isDone
        self.createdAt = createdAt
    }
}

@main
struct TodoApp: App {
    var body: some Scene {
        WindowGroup { TodoListView() }
            .modelContainer(for: Todo.self)      // 建库并注入环境
    }
}

struct TodoListView: View {
    @Environment(\.modelContext) private var context   // 用于增删
    @Query(sort: \Todo.createdAt, order: .reverse) private var todos: [Todo]

    var body: some View {
        List(todos) { todo in
            Text(todo.title)
        }
        .toolbar {
            Button("添加") {
                context.insert(Todo(title: "新任务"))   // 插入后界面自动更新
            }
        }
    }
}
```

文件与 JSON 适合需要自己掌控格式的场景。App 的 Documents 目录会随 App 一起备份，是放用户数据的地方：

```
func saveJSON() throws {
    let items = [Post(id: 1, userId: 1, title: "标题", body: "内容")]
    let data = try JSONEncoder().encode(items)

    let dir = try FileManager.default.url(
        for: .documentDirectory, in: .userDomainMask,
        appropriateFor: nil, create: true
    )
    try data.write(to: dir.appendingPathComponent("posts.json"))
}
```

> 结论：小配置用 `@AppStorage`，结构化数据用 SwiftData，需要自定义格式或导出时再走文件。不要一开始就上数据库，也不要把列表塞进 UserDefaults。

4.10 第 9 步：引入第三方依赖

现在的事实标准是 **Swift Package Manager（SPM）**。在 Xcode 中通过 File > Add Package Dependencies 添加，具体入口位置可能随版本不同；也可以搜索包名或直接粘贴仓库地址。

CocoaPods 是历史方案，它的存在有现实原因：早期 Swift 生态里很多库只发布 Pod，不发布 Swift Package。CocoaPods 的问题是需要 Ruby 环境、多一个 Podfile、装完依赖后必须打开 `.xcworkspace` 而不是 `.xcodeproj`，否则会报找不到模块。

对比项	Swift Package Manager	CocoaPods
是否内置	Xcode 内置，无需额外安装	需要安装 Ruby 与 pod 命令
配置文件	Package.swift 由库提供	项目里维护 Podfile
打开方式	直接开 .xcodeproj 或 .xcworkspace	必须开 .xcworkspace
版本锁定	Package.resolved	Podfile.lock
现状	新项目的默认选择	维护老项目时才需要

常见第三方库（知道名字和用途即可，不要急着引）：

- Alamofire：网络请求封装，比 URLSession 语法更短。
- Kingfisher：异步加载与缓存网络图片。
- Lottie：播放设计师导出的动画文件。
- SwiftyJSON：简化 JSON 取值，在 Codable 出现前很流行。

> 结论：新手优先用系统框架，能不引库就不引库。系统框架免费、体积为零、不需要担心第三方停止维护；只有当你确实被某个问题反复消耗时间时，再引入成熟库。

SPM 会把解析结果写到 `Package.resolved`，它锁定了每个依赖的具体版本。**这个文件必须提交到 Git**，否则不同机器解析出的版本可能不一致，"我这里能编译"这种问题就会反复出现。

4.11 第 10 步：调试与预览

调试的第一原则：不要让 `print` 成为唯一的排查手段。

断点有三类常用形态：

- 普通断点：点击代码行左侧的空白区域添加，执行到该行会暂停。

- 条件断点：右键断点编辑条件，例如只在 `index == 42` 时停下。循环里排查某一次迭代时非常有用。
- 异常断点：在断点导航器的加号菜单里添加 Exception Breakpoint。它能在抛异常的那一刻停下，而不是等到程序崩溃，是定位崩溃最快的手段之一。

断点不生效时先确认三件事：代码是否真的执行到了那一行（加一条日志验证比盯着代码猜快）；当前运行的构建版本是不是你改过的那份；断点是否被意外禁用（断点导航器里会显示为灰色）。另外，`#Preview` 与模拟器是两个独立进程，给预览打断点行为不稳定，调试请用模拟器。

暂停后使用 LLDB 命令检查现场：

```
po user.name          # 打印对象描述，最常用
p items.count         # 打印表达式结果，带类型
expr total = 10       # 运行期改变量，改完可继续执行
bt                    # 打印调用栈，看是谁调到这里的
continue              # 继续执行
step over             # 单步，不进入函数内部
step into             # 单步，进入函数内部
```

日志输出有两种方式：`print` 简单但不带级别与分类；`os.Logger` 是系统统一日志，能看到子系统、分类和级别，排查线上问题时更有用。

```
import OSLog

let logger = Logger(subsystem: "com.bingyan.helloios", category: "network")

func load() {
    logger.debug("开始加载")
    logger.error("状态码异常：\(statusCode, privacy: .public)")
}
```

SwiftUI 的预览用 `#Preview` 宏，写在文件底部：

```
#Preview {
    PostListView()
}

#Preview("空数据") {
    ContentUnavailableView("暂无内容", systemImage: "tray")
}
```

预览报错的常见原因：预览代码里的依赖没有提供（比如 SwiftData 模型没挂 container）、模型文件被删了但预览还在引用、编译错误导致预览无法构建。排查顺序是先看编译是否通过，再看预览宏里的视图是否可以独立构建；给预览单独提供 `.modelContainer(for: Todo.self, inMemory: true)` 常常能立刻解决。

其他两个工具各记一句话：视图层级调试用 Debug View Hierarchy，能三维展开当前界面的图层，排查"为什么这个控件被挡住了"很有效；内存问题用 Instruments 的 Leaks 或 Allocations 模板，以及 Memory Graph，能看清对象之间的引用关系，排查循环引用。

定位一个崩溃的标准顺序：

1. 先看崩溃日志（Xcode 控制台或设备的崩溃报告），找到异常类型和关键线程。
2. 加上异常断点，重新复现一次，让程序停在抛出点。
3. 用 `bt` 看调用栈，定位到具体文件和行号。
4. 读错误信息本身。`Unexpectedly found nil` 说明某个可选值是 nil 却被强制解包，回到那一行找出为什么是 nil。

> 提示：崩溃日志里的行号只有在你保留了对应版本的 dSYM 时才有意义。本地调试时 Xcode 会自动处理，发布版本需要归档时保留符号文件。

4.12 第 11 步：真机运行与签名

签名的目的只有一个：让系统确认"这个 App 是谁做的、允许装在哪些设备上"。它由四个东西配合完成。

- **证书（Certificate）**：证明"你是你"。存在本机钥匙串里，配合 Apple ID 生成。
- **Provisioning Profile（描述文件）**：把证书、Bundle ID、可安装的设备列表绑在一起的一份授权文件。
- **Team（团队）**：证书与描述文件的归属方，个人账号就是一个只有你自己的团队。
- **Bundle ID**：App 的唯一身份，签名时 profile 里的 Bundle ID 必须和工程里的一致。

概念	存在哪	作用	出错时的典型现象
证书	本机钥匙串	证明开发者身份	No signing certificate 报错
描述文件	Xcode 自动管理	授权某证书在某设备装某 App	设备不在 profile 中无法安装
Team	Apple 开发者账号	归属与权限	Team 未选择，签名项为空
Bundle ID	工程设置	App 唯一标识	Bundle ID 冲突无法注册

证书按用途分几类：开发证书用于把 App 装到自己或测试设备上，分发证书用于打包给测试员或提交商店，推送证书（或密钥）用于给 App 发通知。日常调试只需要开发证书。描述文件也分成开发型和分发型，前者的设备列表里必须有你的设备，后者用于发布。搞清楚"证书证明身份、描述文件授予权限"这个分工，签名相关的报错就基本能自己读懂了。

免费 Apple ID 的个人团队能真机调试，但有限制：证书 7 天过期、不支持推送通知、不能上架 App Store、不能使用 TestFlight。付费账号没有这些限制，代价是每年 99 美元。付费账号每年还能注册 100 台测试设备，超出后要等下一年度才能再加。

在 Xcode 里的标准做法是打开目标的 Signing & Capabilities 设置，勾选 **Automatically manage signing**，然后选择 Team。Xcode 会自动创建证书与描述文件，你不需要手动去网站下载。这一步的具体界面分布在版本间可能略有差异，但"自动管理签名"这个开关一直存在。

真机连接步骤：

1. 用数据线把 iPhone 连到 Mac，手机上选择"信任此电脑"。
2. 在 Xcode 的 Scheme 选择区把运行目标从模拟器切换到你的设备。
3. 首次运行后手机上会提示"不受信任的开发者"，需要在设备上通过 设置 > 通用 > VPN 与设备管理（不同 iOS 版本的入口名称可能略有不同）找到开发者 App 并选择信任。
4. 如果设备开启了开发者模式，按系统提示在设备上打开它并重启一次。

> 提示：签名报错先做三件事：确认 Bundle ID 全局唯一、确认 Team 已选择、确认设备已添加到账号。绝大多数签名问题都是这三项之一。

4.13 第 12 步：打包、发布与 TestFlight

发布是把构建版本交给苹果审核、再分发给用户的过程。归档是第一步：在 Xcode 的 Product > Archive 生成发布用的归档（菜单位置可能随版本不同）。归档完成后会打开 Organizer 窗口，在这里选择 Distribute App，按向导上传到 App Store Connect。

上传前需要在 App Store Connect 网站上创建 App 记录，填写名称、主要语言、Bundle ID 和 SKU。一个 Bundle ID 只能对应一条 App 记录，名称也要全局唯一。上传的构建版本会先进入"处理中"，处理完成可以提交审核。

版本号与构建号是两个不同的字段，很多人会混淆：版本号（Version）是给用户看的，形如 1.0.0，同一个版本可以上传多个构建；构建号（Build）是给系统区分包的，每次上传必须比上一次大。忘改构建号会直接导致上传被拒。第一次上传还可能遇到出口合规（Export Compliance）的询问，如果用了系统提供的标准加密能力，按提示如实声明即可，不需要额外材料。

TestFlight 用来做测试分发，分两种：

- 内测：最多 100 名内部成员，不需要审核，上传后很快可用。
- 外测：可以邀请外部测试员（上限 10000 人），需要经过一次审核，通常一两天内有结果。

常见被拒原因：

- 崩溃或明显功能不可用，审核员点几下就挂掉。
- 缺少隐私政策链接，或声明收集的数据和实际行为不符。
- 使用私有 API，或引导用户到 App 外支付。
- 首次启动没有任何内容，界面看起来像未完成。
- 缺少隐私清单（Privacy Manifest），未声明使用的必要理由 API 与数据类型。

从提交到上架的大致节奏：上传构建版本后处理几十分钟，提交审核后常见一两天出结果，高峰期可能更久。被拒后修改并重新提交，重新排队。

需要提前准备的材料：

材料	要点
App 图标	一张 1024 像素的方形图，不能有透明通道
截图	覆盖各必需尺寸的机型，展示真实界面
隐私政策	可公开访问的页面地址
隐私清单	说明收集的数据类型与用途
年龄分级	按内容如实填写问卷

上架流程清单：

1. 确认版本号与构建号已更新（构建号必须比上次大）。

2. 确认 App 图标与启动配置完整，没有占位图。

3. 在真机上完整走一遍主流程，特别是首次启动和网络异常。

4. Product > Archive 归档，Distribute App 上传。

5. 在 App Store Connect 创建或更新 App 记录，填写描述与关键词。

6. 上传截图、隐私政策链接、隐私清单，完成年龄分级问卷。

7. 添加内测成员，用 TestFlight 验证安装版本是否正常。

8. 提交审核，被拒则按反馈修改后重新提交。

9. 审核通过后选择手动发布或自动发布。

4.14 工程规范与 Git 工作流

工程一旦超过一个人，规范就是效率。`.gitignore` 要拦住的是"本机生成、别人不需要"的文件：

```
# macOS 系统文件
.DS_Store

# Xcode 个人配置与构建缓存
xcuserdata/
DerivedData/
*.xcuserstate
*.moved-aside

# Swift Package Manager
.build/

# 如果不用 CocoaPods 就不要提交它，用的话由 Podfile 重建
Pods/

# 依赖锁文件要提交，不要忽略
# Package.resolved
```

分支策略用最简单的一种就够：

- `main`：随时可构建、可发布的稳定分支，不要直接在它上面写功能。
- `dev`：集成分支，功能完成后合并到这里联调。
- `feature-xxx`：每个功能一个分支，例如 `feature/post-list`，完成后合并回 `dev`。

提交信息用前缀加一句话，前缀固定含义，一眼能看出这次改动性质：

前缀	含义	示例
feat	新功能	feat: 增加文章列表页
fix	修缺陷	fix: 修复断网时列表空白
refactor	重构，不改行为	refactor: 抽出网络请求层
docs	文档	docs: 补充签名说明

常用命令：

```
git switch -c feature/post-list     # 从当前分支切出新功能分支
git add .
git commit -m "feat: 增加文章列表页"
git switch dev
git pull --rebase
git merge feature/post-list
git push
```

提交粒度也有讲究：一个提交只做一件事，做完能编译、能运行。把"加功能"和"顺手格式化整个文件"混在一个提交里，review 时根本看不出真正的改动在哪，出问题也无法单独回滚那一部分。宁可多提交几次，也不要把一天的改动攒成一个巨大的提交。

Code Review 的意义不是找茬，而是让第二个人在合并前理解这次改动。它能拦下笔误、遗漏的错误分支和不符合项目约定的写法，也顺便完成了知识传递：以后这个模块出问题，至少有两个人知道它长什么样。

Xcode 内置了 Git 支持，能看到文件改动、提交、比对分支。但合并冲突、变基、查看历史这类操作在命令行里更清晰，也更容易查到资料。稳妥的分工是：日常提交用 Xcode 界面，遇到冲突和分支操作用命令行。

> 提示：`project.pbxproj` 的冲突不要硬合。先确认双方各自新增了哪些文件，然后在合并后的工程里把文件重新加一遍，比对着文本 diff 手工修要可靠得多。

4.15 常见报错与崩溃速查

下面这些错误几乎每个新手都会遇到一次。排查时先看现象，再看原因，最后按"怎么办"操作，不要一上来就重装 Xcode。

现象	原因	怎么办
运行目标里找不到模拟器	运行时未安装或被删除	在 Xcode 设置里下载 iOS 运行时，再重启 Xcode
No signing certificate found	没有可用证书或 Team 未选	打开 Signing 设置勾选自动管理并选择 Team
Bundle ID 冲突	该标识已被占用	改 Organization Identifier 或 Product Name 重新生成
预览一直转圈或直接崩溃	预览依赖缺失或编译未通过	先确认编译通过，给预览补上 modelContainer 等依赖
索引卡住，代码没有补全	索引进程异常或缓存损坏	等几分钟；仍无反应则清理派生数据后重开工程

现象	原因	怎么办
Thread 1: Fatal error: Unexpectedly found nil	对 nil 的可选值做了强制解包	用 `bt` 找到崩溃行，改为可选绑定或给默认值
并发相关编译错误，提示 MainActor	在非主线程改了界面状态	给模型或方法加 `@MainActor`，或把更新派回主线程
Could not build Objective-C module	模块映射或依赖配置损坏	确认命令行工具指向正确的 Xcode，清理缓存后重建
视图看不到内容或位置不对	修饰符顺序或 frame 约束冲突	逐个去掉修饰符定位，检查 padding 与 frame 的先后
List 或 ForEach 报错要求唯一标识	数据未遵循 Identifiable	让模型遵循 Identifiable，或用 `id:` 显式指定
解码失败，提示 keyNotFound	JSON 字段与模型不匹配	检查 CodingKeys 与 keyDecodingStrategy
真机上装完提示不受信任	开发者证书未被手动信任	在设备的 VPN 与设备管理里信任该开发者
证书 7 天后失效无法运行	免费个人账号的机制限制	重新构建安装一次，或加入付费开发者账号
改了数据但界面不刷新	数据不是可观察状态，或改的是副本	确认数据用 @State 或 @Observable 持有，传递时没有被复制
模型加字段后启动即崩溃	数据库结构与新模型不匹配	先用内存容器验证，再按 SwiftData 的迁移方式处理

> 结论：报错信息里通常已经写了答案。先完整读完错误，再决定要不要搜索；搜索时把报错原文和 Xcode 版本一起带上，命中率会高很多。

第 5 章 实战：从零做一个待办清单 App

前面几章你已经把 Swift 语法过了一遍，也知道了一个工程在 Xcode 里是怎么跑起来的。这一章不再讲新概念，而是把学过的东西串成一条线：从新建工程开始，一步一步做出一个能真正用起来的待办清单（To-Do List）App。每一步都先讲清楚要做什么、为什么这么做，再给完整代码，再逐行解释，最后告诉你这时候运行应该看到什么。照着顺序抄下来，这个 App 就能跑起来。

5.1 项目目标与最终效果

这一章要做的 App 叫 TodoList。它只做一件事：管理一份待办清单。功能清单如下。

- 新增：通过右上角的加号打开输入框，输入标题后添加到列表。
- 完成：点击行首的圆圈，切换完成状态，标题出现删除线并且颜色变浅。
- 删除：在任意一行上向左滑动，出现删除按钮，点击即可移除。
- 时间：每一行显示这条待办的创建时间。
- 持久化：关掉 App 再打开，数据还在。

最终会用到 4 个 Swift 文件，每个文件只负责一件事。

- `TodoListApp.swift`：App 的入口，负责创建数据容器。
- `Item.swift`：数据模型，描述"一条待办长什么样"。
- `ContentView.swift`：主页面，负责列表、新增和删除的组织。
- `ItemRow.swift`：单行子视图，负责一行的样子和勾选交互。

界面的结构可以用下面这张图理解。它从上到下描述了屏幕上会出现的元素，以及每个元素由哪段代码负责。

```
NavigationStack                  标题栏：待办清单                          右上角：加号按钮
└── List                         负责可滚动的列表容器
    ├── ItemRow                  未完成：○  买牛奶          8 月 3 日 14:20
    ├── ItemRow                  已完成：●  写周报          8 月 3 日 09:05（删除线 + 灰字）
    └── 左滑任意一行             出现"删除"按钮
列表为空时                       整个列表区域显示占位提示
```

这一章的写法是"每一步都可运行"。八个步骤里，每一步结束后工程都处在能编译、能运行的状态，不存在"先把所有代码写完再调试"的段落。所以你最好的做法是：打开 Xcode，照着抄一步、运行一步，确认看到的东西和文中描述一致，再进入下一步。如果哪一步运行结果和描述不符，先回到那一步检查括号、修饰符顺序和类型名，Swift 的报错信息绝大多数时候会直接指到出错的那一行。另

外，本章涉及的 API 都在 iOS 17 之后可用，只要工程的最低部署版本不低于 iOS 17，就能原样编译。

四个文件的分工和数据流是这样的：`TodoListApp` 创建 SwiftData 容器并把数据放进环境；`ContentView` 通过查询属性拿到数据数组，把数组交给 `List` 和 `ForEach`；`ForEach` 为每一条数据创建一个 `ItemRow`；`ItemRow` 只关心"一条待办怎么画、怎么点"。新增和删除走的是另一条路：`ContentView` 从环境里取出写入上下文（Context），把新对象插进去或者把旧对象删掉。

> 提示：这一章所有代码的基线是 iOS 17，模拟器上就能完整跑通。真机也支持，但最后一步之前不需要真机。

5.2 第 1 步：创建工程与文件结构

第一步是新建工程。打开 Xcode，在欢迎窗口里选择创建新工程，模板选 iOS 平台下的 App，然后按下面的表格填写。

选项	填什么	说明
Product Name	TodoList	工程名，同时也是 App 显示名
Team	先不选	只跑模拟器不需要签名
Organization Identifier	例如 com.bingyan	和 Product Name 拼成唯一标识
Interface	SwiftUI	唯一主线，不要选 Storyboard
Language	Swift	没有别的选择
Storage	SwiftData	想自己动手可以选 None，然后在第 3 步手写模型

模板里的 Testing System 之类的选项全部不勾也没问题。Storage 选 SwiftData 时，Xcode 会顺手生成一个 `Item.swift` 示例模型和一个带 `.modelContainer` 的入口文件；选 None 时这两个文件要靠自己建。两种选择最终的代码完全一样，本章按"选 None、全部手写"的路径讲，这样你能看清每一行是从哪来的。手工建文件的方法是新建一个 Swift File，文件名按本章给的填，内容直接抄。

工程建好后还有一处值得确认：在工程设置的部署信息里，最低部署版本要不低于 iOS 17。SwiftData、`ContentUnavailableView`、`@Query` 的排序写法都要求 iOS 17，如果这里被设成了更早的版本，代码会直接报"仅在某版本后可用"。Xcode 27 的新工程默认已经满足这个要求，如果你打开的是别人建的旧工程，记得先看一眼这一项。

工程建好后，左侧的文件导航区应该长这样。方括号里是本章要动手写的文件。

```
TodoList
├── TodoListApp.swift      App 入口
├── ContentView.swift      主页面
├── Item.swift             [待创建] 数据模型
├── ItemRow.swift          [待创建] 单行子视图
└── Assets.xcassets        图标与配色资源
```

接下来要说明一个容易被忽略的问题：为什么不能把所有代码都塞进 `ContentView.swift`。原因有三条。第一，SwiftUI 的视图是结构体（Struct），一个文件里堆十几个视图，类型名会互相干扰，改一个地方要看一大段无关代码。第二，Xcode 的增量编译以文件为单位，改一行只重编一个文件比重编一个巨型文件快得多，工程大了差别很明显。第三，数据模型属于"数据层"，和"界面层"混在一起后，将来想换存储方案或者加个小组件就会牵连一大片。真实项目里，文件拆分几乎是第一件要做对的事。

> 提示：文件拆分不等于过度设计。这个 App 一共四个文件，每个文件几十行，这就是恰当的粒度。不要为了"架构好看"再套五六层目录。

5.3 第 2 步：定义数据模型

这一步定义整个 App 的核心类型：一条待办。它是数据层唯一的类型，后面的列表、新增、删除都围着它转。

在项目里新建文件 `Item.swift`，内容如下。

```
import Foundation
import SwiftData

// @Model 把一个普通类变成可以持久化的数据模型
@Model
final class Item {
    // 待办内容。给默认值是为了让旧数据在模型升级时能平滑迁移
    var title: String = ""
    // 是否已完成
    var isDone: Bool = false
    // 创建时间，用来排序和显示
    var createdAt: Date = Date.now

    init(title: String, isDone: Bool = false, createdAt: Date = Date.now) {
        self.title = title
        self.isDone = isDone
        self.createdAt = createdAt
    }
}
```

逐点解释这段代码。

`@Model` 是 Swift 的宏（Macro）。它在编译期读一遍这个类的结构，然后自动生成一大堆代码：类对 `PersistentModel` 协议的实现、数据库表结构描述（Schema）、把属性读写翻译成数据库操作的访问器、以及一整套属性变化通知机制。正是这套通知机制让界面能在数据变化时自动刷新，所以你不需要、也绝对不能在这个类上再加 `@Observable`，两者叠加会冲突。

这里必须用类（Class）而不是结构体。结构体是值类型，赋值时会被复制，SwiftData 没法靠它追踪"同一条待办"；类是引用类型，每条记录在内存里是一个对象，有一个稳定身份，多个视图引用的是同一个对象。

三个属性分别对应界面上要展示的东西。`title` 存文本；`isDone` 存勾选状态；`createdAt` 存创建时间，排序和标题下方的小字都靠它。属性都写了初始值，这不是随手加的：SwiftData 在读取旧版本写下的数据文件时，会先做一次轻量迁移，哪个属性有默认值，它就能给缺失的旧数据补上哪个值；如果不给默认值，那就得把类型写成可选值（Optional），或者自己写迁移方案。养成习惯，新加的属性一律给默认值或设为可选。

`init` 给后两个参数提供了默认值，所以创建一条待办只要写 `Item(title: "买牛奶")`，完成状态和创建时间会自动补上。`createdAt` 的默认值写成 `Date.now` 而不是在初始化器里写死一次，是因为默认参数每次调用时才会求值，这样每条待办拿到的是各自真实的时间。

这时候还没有界面可以运行，运行也只会看到模板留下的白屏。可以确认的是：这个文件不报错，说明模型定义合法。下一节就把数据画到屏幕上。

5.4 第 3 步：做出列表页面

这一步的目标很单纯：让数据出现在屏幕上。先不写新增和删除，只做一个"只读"的列表，把"数据从哪来、怎么变成界面"这条路走通。

第一件事是让 App 创建数据容器。打开 `TodoListApp.swift`，改成下面这样。

```
import SwiftUI
import SwiftData

@main
struct TodoListApp: App {
    var body: some Scene {
        // 这一句为整个 App 准备 SwiftData 容器，并把 Item 注册进去
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Item.self)
    }
}
```

`@main` 标记 App 的起点。`WindowGroup` 描述窗口内容，在 `WindowGroup` 后面加 `.modelContainer(for: Item.self)`，意思是"给这个窗口里所有视图准备好一个 Item 的数据容器"。这个容器一旦创建，就会自动放进 SwiftUI 的环境（Environment）里，任何子视图都能取到它——这就是后面 `@Query` 和 `@Environment` 能拿到数据的原因。

第二件事是写列表。把 `ContentView.swift` 整个替换成下面这个最简版本。

```
import SwiftUI
import SwiftData

struct ContentView: View {
    // 从环境里取出写入用的上下文，现在只用来临时插数据
    @Environment(\.modelContext) private var modelContext
    // 查询所有 Item，数据一变，界面自动重新计算
    @Query private var items: [Item]

    var body: some View {
        NavigationStack {
            List {
                ForEach(items) { item in
                    Text(item.title)
                }
            }
            .navigationTitle("待办清单")
            .toolbar {
                // 临时按钮，下一节会换成真正的输入界面
                Button("临时加一条") {
                    modelContext.insert(Item(title: "我的第一条待办"))
                }
            }
        }
    }
}
```

三个关键类型各管一件事。`List` 是列表容器，负责滚动、行背景、分隔线，以及后面要用的滑动删除手势。`ForEach` 负责"把数组展开成一行行视图"，它需要数组元素能唯一标识自己，`@Model` 已经为 `Item` 自动提供了这个身份，所以直接 `ForEach(items)` 就行，不用写 `id:`。`@Query` 是一个属性包装器，它去数据容器里把符合条件的记录全部取出来，并且和数据保持同步：只要数据变化，用

到 items 的视图就会重新计算。它和 @State 的区别在于数据来源——@State 的数据活在视图里，@Query 的数据活在数据库里。

@Environment(\.modelContext) 取的是写入上下文。@Query 负责读，上下文负责写，读写分开是 SwiftData 的基本模型。

最外层套一个 NavigationStack，是为了给页面提供标题栏和工具栏的容器。.navigationTitle 和 .toolbar 这两个修饰符只有在导航容器内部才有意义，写在外面不会报错但也不会生效。现在这个 App 只有一层页面，看起来用不用导航容器差别不大，但标题栏、右上角按钮以及将来要加的二级页面都依赖它，所以从一开始就放上。

这时候数据本来是空的，所以先用工具栏上的临时按钮插一条，验证查询链路是通的。想看预览的话，把 ContentView.swift 底部的预览加上内存容器，数据就不会写进真实数据库。

```
#Preview {
    ContentView()
        .modelContainer(for: Item.self, inMemory: true)
}
```

这时候运行应该看到什么：标题栏显示"待办清单"，右上角有一个"临时加一条"按钮，列表区域是空的。点一下按钮，列表立刻多出一行"我的第一条待办"；连点几次就有多行。如果点完没有任何变化，八成是 TodoListApp.swift 里漏了 .modelContainer(for: Item.self)，请回去补上。

> 提示：inMemory: true 表示数据只放在内存里，App 退出就没了。预览里用它，可以避免调试数据被写进真实数据库。

5.5 第 4 步：新增待办

上一步的临时按钮证明了数据能读能写，但它插进去的内容是写死的。这一步把它换成真正的输入：点加号弹出一个带输入框的提示框，用户输入标题，确认后添加。

需要三样东西。第一，一个 @State 变量保存输入框里的文本；第二，一个 @State 布尔值控制提示框的显示；第三，一个添加函数，负责把文本校验后变成 Item 插进上下文。

把 ContentView.swift 改成下面这样。

```
import SwiftUI
import SwiftData

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext
```

```
    @Query private var items: [Item]

    // 输入框里的文本，由 SwiftUI 和用户共同维护
    @State private var newTitle = ""
    // 控制提示框是否弹出
    @State private var isAdding = false

    var body: some View {
        NavigationStack {
            List {
                ForEach(items) { item in
                    Text(item.title)
                }
            }
            .navigationTitle("待办清单")
            .toolbar {
                ToolbarItem(placement: .topBarTrailing) {
                    Button {
                        isAdding = true
                    } label: {
                        Label("新增", systemImage: "plus")
                    }
                }
            }
            // 提示框里可以直接放输入框，输入内容通过绑定同步到 newTitle
            .alert("新增待办", isPresented: $isAdding) {
                TextField("要做什么？", text: $newTitle)
                Button("取消", role: .cancel) {
                    newTitle = ""
                }
                Button("添加") {
                    addItem()
                }
            } message: {
                Text("输入标题后点添加")
            }
        }
    }

    private func addItem() {
        // 去掉首尾空格和换行，避免存进一堆看不见的空白
        let trimmed = newTitle.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else {
            newTitle = ""
            return
        }
        // 插入上下文，@Query 会自动把新数据带进界面
        modelContext.insert(Item(title: trimmed))
        newTitle = ""
    }
}
```

逐点解释。

`@State private var newTitle = ""` 是视图自己拥有的状态。`@State` 的规则是：值变了，视图重新计算。`TextField("要做什么？", text: $newTitle)` 里的 `$newTitle` 是绑定（Binding），它把输入框和这个变量连起来，用户每敲一个字，`newTitle` 就更新一次；反过来代码改了 `newTitle`，输入框内容也跟着变。绑定写法里的 `$` 是在取属性包装器暴露出来的双向通道，不是取变量本身。

`isAdding` 是提示框的开关。`.alert` 的第一个参数是标题，`isPresented: $isAdding` 说明这个提示框在 `isAdding` 为真时弹出、关闭时自动置回假，所以按钮里只需要写 `isAdding = true`，不需要写关闭逻辑。提示框的内容区可以放多个按钮，带 `role: .cancel` 的按钮会排在系统认为合适的位置，并且点它不会触发别的按钮。取消时把输入清空，是为了下次打开是干净的空框。

`@Environment(\.modelContext)` 是写数据的入口。SwiftData 里所有的增删改都要经过上下文：`insert` 加一条，`delete` 删一条，改属性直接改对象。上下文还负责把改动写回磁盘，这一步默认自动完成，你不用手动调用保存。为什么不直接操作数据库？因为上下文是一层缓存和事务边界，它让一批改动一次性落盘，也让界面在改动后立刻知道哪些数据变了。

输入为空要过滤掉，这不是可选项。用户点了添加却没输内容、或者只敲了空格，都应该被拦住，否则列表里会出现一行看不见文字的空条目，后面想删都难找。

`trimmingCharacters(in: .whitespacesAndNewlines)` 去掉首尾空白，`guard` 在结果为空时直接清空输入并返回。注意保存的是 `trimmed` 而不是 `newTitle`，避免把用户误敲的空格一起存进数据库。

这时候运行应该看到什么：右上角出现一个加号按钮，点击后弹出提示框，里面有输入框和"取消""添加"两个按钮。输入"买牛奶"点添加，提示框关闭，列表立刻多出一行。点添加时不输任何内容，列表不会多出空行。

5.6 第 5 步：完成、删除与排序

前三步做出了一个只能新增的列表，这一步补齐三个日常操作：勾选完成、左滑删除、按创建时间排序。同时会引入这一章最重要的界面设计习惯——把一行拆成独立的子视图。

先新建文件 `ItemRow.swift`。它负责画一行，并处理点击勾选。

```
import SwiftUI

struct ItemRow: View {
    // 只读地持有一条待办。Item 是类，改它的属性能直接反映到数据库
    let item: Item

    var body: some View {
        HStack(spacing: 12) {
            Image(systemName: item.isDone ? "checkmark.circle.fill" : "circle")
                .font(.title2)
                .foregroundStyle(item.isDone ? Color.accentColor : Color.secondary)

            Text(item.title)
                // 已完成就加删除线，颜色同时变浅
```

```
                .strikethrough(item.isDone, color: .secondary)
                .foregroundStyle(item.isDone ? Color.secondary : Color.primary)

            Spacer()
        }
        // 让整行都能响应点击，而不只是图标和文字本身
        .contentShape(Rectangle())
        .onTapGesture {
            // 包在动画里，勾选状态切换时界面会平滑过渡
            withAnimation {
                item.isDone.toggle()
            }
        }
    }
}
```

再回到 `ContentView.swift`，把查询改成带排序的版本，给 `ForEach` 加上滑动删除，并把行内容换成 `ItemRow`。

```
import SwiftUI
import SwiftData

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext
    // 按创建时间倒序，最新的排最上面
    @Query(sort: \Item.createdAt, order: .reverse) private var items: [Item]
    @State private var newTitle = ""
    @State private var isAdding = false

    var body: some View {
        NavigationStack {
            List {
                ForEach(items) { item in
                    ItemRow(item: item)
                }
                // 滑动删除：系统负责手势和"删除"按钮，我们只处理删除结果
                .onDelete(perform: deleteItems)
            }
            .navigationTitle("待办清单")
            .toolbar {
                ToolbarItem(placement: .topBarTrailing) {
                    Button {
                        isAdding = true
                    } label: {
                        Label("新增", systemImage: "plus")
                    }
                }
            }
            .alert("新增待办", isPresented: $isAdding) {
                TextField("要做什么？", text: $newTitle)
                Button("取消", role: .cancel) {
                    newTitle = ""
                }
                Button("添加") {
                    addItem()
                }
            } message: {
                Text("输入标题后点添加")
            }
        }
    }

    private func addItem() {
```

```
        let trimmed = newTitle.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else {
            newTitle = ""
            return
        }
        modelContext.insert(Item(title: trimmed))
        newTitle = ""
    }

    private func deleteItems(at offsets: IndexSet) {
        // offsets 是列表里的行号，必须用同一份 items 去取，顺序不能换
        for index in offsets {
            modelContext.delete(items[index])
        }
    }
}
```

逐点解释。

勾选为什么能生效？`item` 是类实例，`item.isDone.toggle()` 直接改了这个对象在内存中的属性。`@Model` 生成的通知机制告诉 SwiftUI"这个对象的这个属性变了"，读到过它的视图就会刷新，同时 SwiftData 会把改动写进数据库。注意这里没有 `@State`，也没有绑定——数据的变化源头不在视图里，视图只是恰好读了它。把一行抽成 `ItemRow` 的意义也在这里：`ContentView` 只负责"有哪些数据、怎么组织列表"，`ItemRow` 只负责"一条数据长什么样、怎么交互"，两边的职责不会互相污染。将来要给行加一个备注按钮，你只需要打开 `ItemRow.swift` 改十行，不用碰列表页。

`@Query(sort: \Item.createdAt, order: .reverse)` 里的 `\Item.createdAt` 是键路径（Key Path），意思是"按 createdAt 这个属性排序"；`order: .reverse` 表示倒序，新加的待办排在最前面。排序交给数据库做，而不是取出数组后再用 `sorted` 排一遍，因为数据量大时数据库有索引可用，而且排序条件和查询条件写在一起更不容易漏。

`.onDelete(perform:)` 必须写在 `ForEach` 上，不能写在 `List` 上。写对之后，系统自动为每一行加上左滑手势和"删除"按钮，删除时把被删行的下标集合（`IndexSet`）回调给你的函数。函数里必须用 `modelContext.delete(items[index])` 删除真正的对象：只从 `items` 里移除是没用的，`items` 是查询结果，不是可变数组，而且下次查询它还会回来。这里的关键是"用同一份数组按同样的顺序取下标"，所以排序参数和 `ForEach` 用的是同一个 `items`，绝对不要在中间再插一次排序。

`.contentShape(Rectangle())` 的作用是把整行的可点击区域补成一个矩形。不加它的话，只有图标和文字这些实际绘制了内容的地方能响应点击，行中间的空白处点了没反应。`withAnimation` 把这次改动标记成需要动画，勾选图标从空心变实心、文字变灰、删除线出现都会平滑过渡，而不是瞬间跳变。

这时候运行应该看到什么：列表按时间倒序排列，最新添加的在最上面。点任意一行的任意位置，圆图标在空心和实心之间切换，已完成的行文字变灰并带上删除线。在某一行上向左滑动，行尾出现红色"删除"按钮，点击后这一行消失。

5.7 第 6 步：数据持久化

前面的代码每次运行都能记住数据，靠的就是 `TodoListApp.swift` 里那一句 `.modelContainer(for: Item.self)`。这一步把它讲透，并教你验证数据真的落到了磁盘上。

`.modelContainer(for: Item.self)` 挂在 `WindowGroup` 上，含义是：App 启动时创建（或复用）一个 SwiftData 容器，这个容器知道 `Item` 这张表的完整结构；容器里有一个主上下文（Main Context），供主线程读写；容器和上下文都会注入 SwiftUI 环境，`@Query` 从容器读，`@Environment(\.modelContext)` 从环境取上下文写。挂在 `WindowGroup` 上意味着"这个窗口下的所有视图共享同一套数据"，所以新建一个子视图不需要再传任何参数就能访问数据。

数据存在哪里？默认情况下，SwiftData 会在 App 沙盒（Sandbox）的 `Library/Application Support` 目录下建一个 SQLite 数据库文件，文件名是 `default.store`，旁边还会有 `default.store-wal` 和 `default.store-shm` 两个辅助文件，它们是 SQLite 的日志和共享内存文件。所谓"持久化"就是这么朴素：内存里的对象被写进这个文件，下次启动再读回来。沙盒是 iOS 给每个 App 划出的独立目录，别的 App 看不到，App 被删除时一起清空。

怎么验证数据真的存下来了？最简单的办法就是杀掉 App。在模拟器里运行一次，加几条待办，勾掉一条，然后在 Xcode 里点停止按钮，或者按模拟器的 Command+Shift+H 回到主屏后上滑把 App 卡片划掉，再重新运行。如果列表里还是刚才那几条、勾选状态也没变，说明数据确实落盘了。如果能加不能存，先检查 `.modelContainer` 是不是漏了。

想知道数据库文件到底在哪，不要靠猜目录，直接在代码里打印真实路径最稳妥。在 `ContentView` 上临时加一段，运行后看 Xcode 底部的控制台输出。

```
.task {
    // 临时：把数据库所在的目录打印到控制台
    print(URL.applicationSupportDirectory.path(percentEncoded: false))
}
```

把打印出来的路径复制下来，在 Finder 里用"前往文件夹"（快捷键 Shift+Command+G）粘贴这个路径，就能看到 `default.store` 那几个文件。除此之外，Xcode 的调试区域本身也会显示当前运行的 App 的容器位置，两种方式都能到达同一个目录。查完记得把这段临时代码删掉。

> 提示：SwiftData 默认开启自动保存，改动会在合适的时机写回磁盘。如果你希望某次修改立刻落盘，就在修改后调用 `try? modelContext.save()`。这个方法会抛错，所以正常用 `try?` 或 `do-catch`。

如果不用 SwiftData，还有两条更轻的路子。第一条是 `@AppStorage`，它本质是 `UserDefaults` 的包装器，适合存"用户偏好"这种零散的小数据，比如主题、是否开启提醒；写法简单，但不适合存一个会不断增长的列表，因为 `UserDefaults` 不是为结构化数据设计的。第二条是自己把数据编码成 JSON 写进 Documents 目录：用 `JSONEncoder` 把待办数组编成 `Data` 写进文件，启动时读回来，用 `JSONDecoder` 解成数组。这条路完全可控，但排序、查询、增量更新、数据迁移都要自己写，数据量一大就得自己造一个数据库出来。所以对"一堆有结构的记录"这类需求，SwiftData 是默认选择，`@AppStorage` 留给设置项，JSON 文件留给导出和备份。

5.8 第 7 步：界面美化与体验优化

功能齐了，现在让它看起来像个正常的 App。这一步做四件事：把行内布局做得更清楚、加上创建时间、列表为空时给提示、把视觉层级用颜色区分开。改动集中在 `ItemRow.swift` 和 `ContentView.swift` 两个文件。

先把 `ItemRow.swift` 换成最终版本。

```
import SwiftUI

struct ItemRow: View {
    let item: Item

    var body: some View {
        HStack(spacing: 12) {
            // 勾选区域做成真正的按钮，点击范围明确，也不会和整行手势互相干扰
            Button {
                withAnimation {
                    item.isDone.toggle()
                }
            } label: {
                Image(systemName: item.isDone ? "checkmark.circle.fill" : "circle")
                    .font(.title2)
                    .foregroundStyle(item.isDone ? Color.accentColor : Color.secondary)
            }
            .buttonStyle(.plain)

            VStack(alignment: .leading, spacing: 4) {
                Text(item.title)
```

```
                    .font(.body)
                    .strikethrough(item.isDone, color: .secondary)
                    .foregroundStyle(item.isDone ? Color.secondary : Color.primary)

                Text(item.createdAt.formatted(date: .abbreviated, time: .shortened))
                    .font(.caption)
                    .foregroundStyle(.secondary)
            }

            Spacer()
        }
        .padding(.vertical, 4)
    }
}

#Preview {
    // 预览不依赖数据库，直接造一个对象即可
    ItemRow(item: Item(title: "写读书笔记"))
}
```

再把 `ContentView.swift` 换成最终版本，它比上一节多了空列表占位视图和统一着色，其余部分不变。

```
import SwiftUI
import SwiftData

struct ContentView: View {
    @Environment(\.modelContext) private var modelContext
    @Query(sort: \Item.createdAt, order: .reverse) private var items: [Item]
    @State private var newTitle = ""
    @State private var isAdding = false

    var body: some View {
        NavigationStack {
            List {
                ForEach(items) { item in
                    ItemRow(item: item)
                }
                .onDelete(perform: deleteItems)
            }
            // 列表为空时盖一层占位视图，避免用户面对一片空白
            .overlay {
                if items.isEmpty {
                    ContentUnavailableView(
                        "还没有待办",
                        systemImage: "checklist",
                        description: Text("点右上角的加号，添加第一条待办")
                    )
                }
            }
            .navigationTitle("待办清单")
            .toolbar {
                ToolbarItem(placement: .topBarTrailing) {
                    Button {
                        isAdding = true
                    } label: {
                        Label("新增", systemImage: "plus")
                    }
                }
            }
            .alert("新增待办", isPresented: $isAdding) {
                TextField("要做什么？", text: $newTitle)
                Button("取消", role: .cancel) {
```

```
                    newTitle = ""
                }
                Button("添加") {
                    addItem()
                }
            } message: {
                Text("输入标题后点添加")
            }
        }
        // 统一设置强调色，勾选图标和工具栏按钮都会跟着变
        .tint(.blue)
    }

    private func addItem() {
        let trimmed = newTitle.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else {
            newTitle = ""
            return
        }
        modelContext.insert(Item(title: trimmed))
        newTitle = ""
    }

    private func deleteItems(at offsets: IndexSet) {
        for index in offsets {
            modelContext.delete(items[index])
        }
    }
}

#Preview {
    ContentView()
        .modelContainer(for: Item.self, inMemory: true)
}
```

逐点解释这几处改动。

图标用 SF Symbols 里现成的两个符号：`circle` 是空心圆，`checkmark.circle.fill` 是带对勾的实心圆。用 `item.isDone ? ... : ...` 在三元表达式里二选一，比准备两张图片省事得多，而且符号会跟着字体大小和粗细自动适配。配色上用 `foregroundStyle`：未完成用 `Color.secondary`（系统灰），完成用 `Color.accentColor`（强调色）。`foregroundStyle` 是 iOS 17 之后推荐的写法，它比旧的 `foregroundColor` 更通用，可以直接接收颜色、材质、渐变等各种样式。强调色不写死在每个图标上，而是在 `ContentView` 的根视图用 `.tint(.blue)` 统一设置，这样以后换色只改一行。

时间显示用 `item.createdAt.formatted(date: .abbreviated, time: .shortened)`。`formatted` 是 Foundation 提供的格式化方法，它会按照用户设备的语言和地区自动调整：中文环境显示成"2024 年 8 月 3 日 14:20"这类样子，英文环境显示成"Aug 3, 2024"。不要自己拼字符串，自己拼出来的格式在其他地区会显得很怪。`.abbreviated` 表示日期用简写，`.shortened` 表示时间精确到分钟。

把勾选做成 `Button` 而不是整行的手势，区别在于点击区域的语义更清楚：用户点的是"切换完成状态"这个动作，而不是"这一整行"。`buttonStyle(.plain)` 是为了去掉按钮默认的按下高亮和蓝色文字，让它看起来就是一个图标。行内用 `VStack(alignment: .leading, spacing: 4)` 把标题和时间上下排开，标题是正文尺寸，时间是 `caption` 小字，视觉层级一眼就能分清。外面留一点垂直内边距，行与行之间不挤在一起。

空列表的占位用 `ContentUnavailableView`，这是 iOS 17 新增的组件，专门用来表示"这里本该有内容但现在没有"。它自动居中、自动配图标和说明文字，比自己摆一个 `VStack` 省事，风格也和其他系统 App 一致。写在外层 `.overlay` 里，是因为列表本身要一直存在，只是上面盖一层提示；`items.isEmpty` 为假时不渲染任何东西。

这时候运行应该看到什么：列表每一行左边是圆圈、右边是标题和一行灰色小字时间，行与行之间有舒服的间距。点击圆圈切换状态，图标颜色和标题上的删除线都会平滑过渡。加号按钮和勾选图标都是统一的蓝色。把最后一条待办删掉，列表区域出现"还没有待办"的居中提示和说明文字。

5.9 第 8 步：真机运行与打包

模拟器上的效果已经完整了。这一步讲怎么把它装到真手机上，以及如果将来要发布，还需要准备什么。先给结论：学习阶段模拟器足够，真机和打包都属于"知道流程即可"，不要卡在这里。

接真机的大致流程是这样。用数据线把 iPhone 连到 Mac，手机上会弹出是否信任这台电脑，选信任。在 Xcode 顶部的运行目标选择器里，把目标从模拟器切换成这台设备。首次运行需要签名：在工程的 Signing 设置里勾选自动签名，然后选择一个 Team。登录过 Apple ID 的账号都可以作为 Team，免费账号也能装到自己的设备上，只是签名有效期短，需要定期重新运行一次。手机的 iOS 版本要高于或等于工程设置的最低部署版本，否则运行时会提示不支持的平台。较新的 iOS 要求在手机上手动开启开发者模式，这个开关在设置里的隐私与安全性分类下，开启后需要重启手机，具体位置以你手机上的实际显示为准。

运行到真机时，Xcode 会先编译、再安装、再启动，屏幕上出现的样子应该和模拟器完全一致。如果报签名相关的错，九成是 Team 没选或者 Bundle Identifier 和别人重复了，把标识符改成一个只属于你的字符串即可。真机和模拟器有两点实际差别值得知道：真机的性能数据才是真实数据，模拟器跑得再流畅也不能说明真机流畅；另外真机上有些能力（比如相机、部分传感器）模拟器没有，用到这

些能力时必须上真机验证。待办清单只用了存储和界面，不涉及这类硬件，所以模拟器验证过的行为在真机上基本一致。

打包上架是另一条更长的路。第一步是归档（Archive）：在 Xcode 的 Product 菜单里选择 Archive，Xcode 会以发布配置重新编译，完成后打开归档管理器，里面列出这次生成的包。第二步是把包上传。第三步是回到 App Store Connect 网站，新建一个 App 记录，填好名称、副标题、分类、价格，然后提交审核。审核前必须准备好的东西包括：一张 1024 点见方的 App 图标（放在工程资源目录里，不要有透明通道）、若干张应用截图（尺寸要求以 App Store Connect 页面上的提示为准，因为要求会随屏幕尺寸变化）、隐私政策链接、以及"App 隐私"表单里对数据收集情况的如实填写。想先小范围试用的可以用 TestFlight，它同样通过 App Store Connect 分发，不用等正式审核通过就能安装。

> 提示：上架的门槛不在写代码，而在准备材料和等待审核。这个阶段的目标是"能跑起来、能记住数据"，真机跑通一次就够了，不要为了上架去改一堆配置。

5.10 从这个项目继续走下去

TodoList 只有 4 个文件、不到 200 行代码，但它已经具备了真实 App 的骨架：数据模型、列表界面、增删改查、持久化、子视图拆分。下面这些方向都可以在现有代码上直接长出来，每一个都只需要动一两处。

扩展方向	主要改动	新增文件
分类与标签	Item 加 category 属性，列表按分类分组展示	Category.swift
截止日期与提醒	Item 加 dueDate，用通知框架排本地提醒	ReminderScheduler.swift
搜索	列表加 searchable 修饰符，按关键词过滤查询	无
小组件	新建 Widget Extension，与主 App 共享数据容器	TodoWidget.swift
iCloud 同步	换成接入 CloudKit 的容器配置	无

分类与标签是性价比最高的一步。给 `Item` 加一个 `category: String = "默认"` 属性，`ContentView` 里的 `List` 改成按分类分组的两层结构，或者干脆加一个标签栏切换不同分类。这一

步能让你练到"改数据模型"这条链路：改了模型之后旧数据怎么办、默认值为什么重要，都会立刻感受到。

截止日期与本地通知会让这个 App 真正有用。给 `Item` 加 `dueDate`，在新增界面里加一个日期选择器，然后在保存时向通知中心申请权限并排一条本地通知，到点系统会弹出提醒。要处理的重点是权限被拒绝的情况：功能要能优雅降级，不能因为用户不给通知权限就崩掉。

搜索用 `.searchable` 修饰符加在列表上，SwiftUI 会自动生成搜索框。搜索词存进 `@State`，过滤方式有两种：数据少的时候直接在内存里用 `filter` 过滤 `items`，简单直观；数据多的时候需要在视图的初始化器里把搜索词传给 `@Query` 的过滤条件，让数据库去筛。先用第一种，等数据量真的上来了再换。

小组件（WidgetKit）的价值在于"不用打开 App 就能看到待办"。做法是新建一个 Widget Extension，然后在主 App 和小组件之间共享同一个数据容器，让小组件也能查询到 `Item`。难点全在共享上：数据放在 App 沙盒里，小组件默认看不到，需要把数据库放到 App Group 共享目录里。

iCloud 同步是 SwiftData 提供的能力，改动量出乎意料地小：换成接入 CloudKit 的容器配置即可。但有两个硬性约束要提前记住：参与同步的模型，属性必须有默认值或者写成可选值，并且不能有唯一性约束。回头看 5.3 里"属性一律给默认值"的建议，这里就是回报。

MVVM 重构是把逻辑从视图里搬出去。新建一个 `ItemStore` 类型，把 `addItem`、`deleteItems` 这类操作搬进去，视图只负责渲染和把用户操作转发给它。做到这一步你会发现，界面代码变得很短，逻辑也可以单独测试了。这是从"能跑"走向"可维护"的关键一步，建议在功能稳定之后再动手，不要一上来就重构。

最后给几条练习建议。第一，把这一章从头抄一遍，但不要复制粘贴，手打的肌肉记忆能让你记住 API 的形状。第二，抄完之后故意改坏几处：把 `.modelContainer` 删掉看看报什么错，把 `@Model` 换成 `@Observable` 看看报什么错，把 `@Query` 的排序键路径写错看看报什么错——看懂报错信息是最实用的技能。第三，找一个不复杂的开源 SwiftUI 项目通读一遍，重点看它的文件是怎么拆的、数据是怎么流动的。第四，动手做冰岩的新生作业项目：作业的验收标准通常是"能跑、有交互、数据能存住"，这三点这一章的 App 全都满足，你可以把它当成模板，换成自己想做的题目，比如记账

本、单词本、训练打卡。第五，把代码提交到 Git 仓库，从第一天就养成写提交信息的习惯，这比多学一个 API 有用得多。

第 6 章 学习路线与资源

前面几章把 Swift 语法、App 的开发过程和第一个实战项目分开讲了。这一章把它们装回同一条时间线：一条是三周速成计划，用最短时间把 Swift 与 SwiftUI 的骨架跑通；另一条是三个月进阶路线，把「能跟着做出来」提升到「能进项目、能独立扛需求」。两条路线共享同一个前提，每天都要写代码。看懂别人的代码和写出自己的代码是两种能力，只有后者会在真实开发和面试里救你。

6.1 三周速成计划

计划按每天 2 到 3 小时设计：白天零碎时间读章节和查概念，晚上留出连续一小时纯写代码。某天提前做完，多出来的时间不要跳到后面，而是关掉示例把当天的代码凭记忆重写一遍。表里的章节号都指本手册。

> 这 21 天没有一天是「只读不写」。哪怕只写了 15 行有效代码，也强过读三小时没敲一个字符。读书给的是熟悉感，写代码给的是能力。

天数	目标	做什么	完成标志
第 1 天	工具链跑通	装好 Xcode 27，按第 4 章 12 步新建 SwiftUI 项目并运行，把模板文字改成自己的名字	模拟器里出现改过的文字，能说清 Build、Run、Preview 各做什么
第 2 天	变量、类型、可选值	读第 2 章前半，示例在 Playground 敲一遍；分别用 `if let`、`guard let`、`??` 处理同一批可选值	能解释 `String?` 与 `String` 的区别，解包代码不崩溃
第 3 天	流程控制、函数、枚举	读第 2 章后半；写命令行程序：输入分数用 `switch` 输出等级，等级用枚举表示	边界值正确，`switch` 穷尽所有分支
第 4 天	结构体、类、协议、扩展、泛型	读第 3 章前半；定义 `Shape` 协议，用结构体实现矩形与圆，再用协议扩展加默认实现	新增图形不必改已有代码，能说清为何优先用结构体

天数	目标	做什么	完成标志
第 5 天	闭包、属性包装器、错误处理	读第 3 章后半；把第 3 天的程序改成抛错函数并用 `do-catch` 处理，再写一个 `async` 函数	能说出 `throws` 与 `Result` 的取舍，知道 `await` 让出了什么
第 6 天	SwiftUI 布局	读第 4 章界面部分；不看示例，用 `HStack`、`VStack`、`List`、`LazyVGrid` 拼一个个人主页	横竖屏都不错位，能给每个修饰符说出它改了什么
第 7 天	状态与数据流	用 `@State` 写计数器，用 `@Binding` 抽子视图，用 `@Observable` 写共享模型，用 `@Environment` 传主题	能画出数据流向，知道为什么不该存两份状态
第 8 天	导航、表单、弹窗、列表	用 `NavigationStack` 做两级跳转，用 `Form` 做输入页，用 `.sheet` 与 `.alert` 做反馈，列表支持删除与排序	返回时数据不丢，删除后界面立即更新
第 9 天	独立复刻待办清单骨架	合上第 5 章，从空项目写数据模型、列表展示、添加与勾选完成	不看参考代码就能写到可以添加待办
第 10 天	补全待办清单	加上编辑、删除、过滤与空列表状态	关掉再打开数据正确重建，空状态有提示界面
第 11 天	加自己的功能	在标签分类、关键字搜索、按截止日期排序里至少做两件	搜索实时过滤，排序切换后顺序正确
第 12 天	收拾成作品	补图标与启动画面，适配深色模式与动态字体，检查小屏排版	浅色深色、最大最小字号下都不截断
第 13 天	网络请求与 JSON	读第 4 章网络部分；用 `URLSession` 请求公开接口，用 `Codable` 把 JSON 解成结构体	控制台能打出模型数组，能说出解码错误来自哪一层

天数	目标	做什么	完成标志
第 14 天	网络层与加载状态	把请求抽成独立服务类型，把加载中、成功、失败、空数据四种状态分别渲染	断网时显示可重试文案，不白屏不崩溃
第 15 天	持久化	用 SwiftData 的 `@Model` 与 `@Query` 把待办存到本地，改一次字段并理解迁移	杀掉 App 再打开数据仍在，字段改动后旧数据可读
第 16 天	读懂别人的代码	在 GitHub 挑一个星标较多的纯 SwiftUI 小项目，跑通并画出目录结构与数据流向	能指出入口、模型，以及视图之间的复用关系
第 17 天	复刻前两个界面	不看实现，凭截图复刻首页与列表页，只在卡住时查它的 API 用法	界面在视觉与交互上接近原项目
第 18 天	对照差异	逐行对照原项目，把差异记成不超过 20 条的清单	每条都能说出原作者写法好在哪里或差在哪里
第 19 天	设计自己的作品	写一页需求：解决什么问题、三个界面、数据来源、最小可用版本；画出草图	需求里删掉了所有附加功能，只剩核心闭环
第 20 天	实现核心功能	按草图写完三个界面与数据层，先跑通主流程再补边界	没用过的人能独立走完一次主流程
第 21 天	收尾与复盘	真机运行，写 README 与截图，录一段 30 秒演示，回顾最卡的三个知识点	作品能装到自己手机上，README 别人看得懂

三周结束时你应该留下四个东西：改造过的待办清单、复刻的界面练习、自己的小作品、一份记录卡点的笔记。最后一份是三个月路线的起点。

6.2 三个月进阶路线

速成计划解决「能跑起来」，三个月路线解决「能进项目」。三个月的划分不是硬性时间，而是三种能力的顺序：语言与界面、工程与协作、深度与方向。每段的检验标准不通过就不要往下走，欠的债后面要还双倍。

第 1 个月 打基础

学什么：把第 2、3 章的语言部分再过一遍，这次要求能给别人讲清楚，重点是可选值、值类型与引用类型的取舍、协议与协议扩展、泛型约束、闭包捕获、错误处理、结构化并发。SwiftUI 侧掌握视图组合、修饰符顺序语义、`@State` 与 `@Binding` 与 `@Observable` 的边界、`NavigationStack` 的路径管理、`List` 与 `LazyVGrid` 的性能差异。

练什么：做三个 200 到 400 行的小 Demo。第一个是计时器，练状态，要求支持暂停、重置，切到后台再回前台时间仍然正确。第二个是天气卡片，练布局，要求同一套视图在不同尺寸下都成立，不允许写死坐标。第三个是单词本，练数据，用 SwiftData 存词条并支持搜索与分组。

怎么检验自己会了：能不看文档写出一个带 `@Observable` 模型、两个页面、一次网络请求和一个列表的完整流程；被问「这个状态为什么放在这里」时能给出理由；看到一张设计稿能说出它由哪几种容器和修饰符组成。

第 2 个月 工程能力

学什么：架构上先掌握 MVVM 的边界，即视图负责渲染、视图模型负责状态与业务编排、模型负责数据与规则，同时知道什么时候不该为了模式而模式。网络层学请求构造、响应解析、错误分类、重试与取消，并把服务抽象成协议以便替换。依赖注入先学最朴素的形式，即通过初始化参数传入依赖。测试学 Swift Testing，先写纯逻辑的单元测试，再给视图模型写带假数据的测试。Git 学到能独立处理分支、变基、解决冲突与写清楚提交信息。

练什么：把速成计划里的小作品重构一遍，把网络请求、数据存储、业务规则全部从视图里抽出来；给数据层与服务层补测试，覆盖率不追数字，但每个错误分支都要有一条测试；用分支开发一个功能再合并回主分支，中间故意制造一次冲突并解决它。

怎么检验自己会了：能把一个 800 行的视图文件拆成五个职责清楚的文件且行为不变；不启动模拟器就能跑通全部单元测试；更换网络实现时只改一处依赖注入；队友只看提交记录就明白你这两天做了什么。

第 3 个月 深入方向

前两个月所有人都要走，第三个月开始分化。从下面六个方向里选一到两个，标准是你想去的岗位和想做的产品，而不是哪个听起来厉害。

方向	学什么	练什么	怎么检验自己会了
性能优化与 Instruments	启动耗时、卡顿掉帧、内存增长与泄漏检测、Time Profiler 与 Allocations 模板	给旧项目做一次体检，找出最慢的界面并优化到肉眼可见的流畅	能拿出优化前后的数据对比，解释每处改动的收益来源
SwiftUI 深入与自定义布局	布局协议、`Layout` 自定义容器、动画事务、转场、Canvas 绘制	实现一个不等高瀑布流容器和一组连贯转场	容器通过不同尺寸实测，动画在低端设备不掉帧
Swift 并发深入	结构化并发、任务组、取消传播、隔离域、actor 与 `Sendable`、数据竞争排查	把共享缓存的类改成 actor 并写出并发测试	开启严格并发检查后零警告编译通过
数据持久化	SwiftData 的关系与查询、谓词、模型迁移、Core Data 的栈与轻量迁移	做一个带多表关系和一次破坏性迁移的本地数据练习	旧数据能升级到新模型且不丢失，大数据量下查询仍够快
WidgetKit 与 App Intents	小组件时间线机制、配置式小组件、快捷指令与系统级动作	给自己的 App 加一个主屏小组件和一个系统指令入口	小组件在锁屏与主屏正确刷新，指令能在系统搜索里唤起
上架一个自己的 App	签名与证书、描述文件、TestFlight 内测、审核条款、隐私清单与权限文案	从打包到提审走完整流程，处理至少一次被拒	App 在 App Store 可见，能读懂审核回复并做对应修改

6.3 资源清单

资源不在多，在于知道什么时候用哪一个。下面每一类都标注了适用时机，遇到对应问题再打开，不要收藏一堆然后一个都不看。

官方资源

名称	入口	什么时候用它
Apple Developer 网站	developer.apple.com	需要账号、证书、描述文件、TestFlight 与上架资料时，这是唯一权威入口
Swift 官方文档《The Swift Programming Language》	docs.swift.org 的 swift-book	想系统读一遍语言本身时用，官方提供简体中文版，中英对照读效果最好
SwiftUI 官方文档	developer.apple.com 的 documentation 下的 swiftui	查某个修饰符有没有某个参数时的第一选择，比任何博客都准
Human Interface Guidelines	developer.apple.com 的 design 部分	做界面拿不准间距、层级与控件用法时查，它是苹果设计审美的说明书
WWDC 视频	developer.apple.com 的 videos 部分	新 SDK 出来想知道某个 API 为什么这么设计时看

课程

名称	入口	什么时候用它
斯坦福 CS193p	斯坦福课程站与 B 站搬运	公认最好的 iOS 入门课，讲思路而不只是操作，做完作业再看本手册会轻松很多
苹果官方 SwiftUI 教程	developer.apple.com 的 tutorials	想按步骤一步步敲出界面时用，适合上手第一周

中文资料

名称	入口	什么时候用它
菜鸟教程 Swift	runoob.com 的 swift 部分	语法忘了想快速找个小例子时用，当速查而不是教材
SwiftGG 翻译组	swiftgg.net	想把英文长文与官方博客读成中文时用，翻译质量整体可靠
常见中文 Swift 博客	掘金、知乎的 iOS 专栏	想知道国内工程实践怎么做时用，注意区分经验与过时结论

社区

名称	入口	什么时候用它
Stack Overflow	stackoverflow.com	遇到具体报错或 API 用法问题的第一站，先搜再问
GitHub	github.com	找开源项目读代码、看某个库如何实现某个功能时用
Swift Forums	forums.swift.org	想了解语言演进、提案讨论与官方答复时用
掘金与知乎的 iOS 话题	juejin.cn 与 zhihu.com	想看中文面试题、踩坑记录与工具推荐时用

工具

名称	入口	什么时候用它
SwiftLint	github.com 上的 realm 项目	项目一开始就接上，让风格问题在提交前暴露，而不是在评审时争论
Proxyman 或 Charles	proxyman.com 与 charlesproxy.com	需要看 App 发了什么请求、返回了什么时用，调接口必备
SF Symbols App	developer.apple.com 的 sf-symbols	找图标、比较不同渲染模式与字重时用，避免自己画图标
Instruments	Xcode 自带，Product 菜单里的 Profile	怀疑卡顿、内存涨、耗电时用，先测再改，不要凭感觉优化

练习场

名称	入口	什么时候用它
LeetCode 的 Swift 支持	leetcode.cn 与 leetcode.com	练算法与 Swift 手感时用，求职笔试前集中刷一段
HackerRank	hackerrank.com	想按专题循序渐进练语法与数据结构时用
自己复刻 App	手机里任意一个你常用的应用	不知道做什么项目时用，挑一个界面复刻，是提升最快的练习

6.4 给新成员的学习建议

组里原来的《iOS 开发指北》有几条精神值得原样传下去：养成好的编程习惯，遇到不懂的概念就一路查下去直到知识闭环，别怕跨界，细节极其重要。下面把它们展开成可直接执行的十条。

1. **代码风格从第一天固定下来**。命名用完整英文单词而不是缩写，类型用大驼峰、变量与方法用小驼峰，布尔值用 `is` 或 `has` 开头。风格不统一的项目，评审时间会浪费在格式争论上而不是逻辑本身。项目建好就接上 SwiftLint，让它替你做这件事。
2. **注释解释为什么，不解释是什么**。写「自增 1」是噪音，写「用字典缓存避免每帧重复解析，实测掉帧次数从 12 降到 0」才是资产。复杂分支、绕过系统缺陷的写法、有性能取舍的地方必须写清楚，否则三个月后的你会恨现在的你。
3. **尽早习惯读英文文档**。绝大多数权威资料是英文的，中文资料往往滞后一到两个大版本。做法不是硬啃：先用英文关键词搜索，打开官方文档只看方法签名和讨论段落，生词当场查，坚持一个月阅读速度就会有质变。
4. **遇到不懂的概念就一路查下去**。查 A 遇到 B，查 B 遇到 C，一直走到所有概念能互相解释，这就是知识闭环。链上出现的概念可能看起来和 iOS 毫无关系，比如内存模型、哈希、依赖注入、序列化协议，它们都是计算机世界的基础设施，早晚会以某种方式支持你的工作。
5. **不要害怕跨界的知识**。学一点 Python 能在写脚本处理数据时省下大量时间，学一点 Rust 或 C 能让你真正理解值类型、内存所有权与并发模型，学一点 HTML 与 CSS 能让你理解盒模型，回头看 SwiftUI 布局会豁然开朗。这些回报通常超出预期。
6. **把细节做对，包括大小写**。苹果的写法是 iOS，不是 IOS、ios、Ios；是 SwiftUI 不是 Swift Ui；是 App Store 不是 AppStore；是 Xcode 不是 XCode。这些名字会出现在简历、提交信息、文档和注释里，写对不费力气，写错会让人立刻判断出你不够严谨。
7. **每天写一点，比周末写一整天有效**。编程是肌肉记忆，隔五天再碰一次，前两小时都在恢复状态。进度再紧，也保持每天至少半小时的有效代码时间。
8. **先让程序跑起来，再让它变漂亮**。新手最常见的卡点是过早追求完美结构，界面还没出来就开始纠结协议怎么分层。正确顺序是先写出能跑的逻辑，再在测试保护下重构。没有可运行版本的重构是赌博。

9. **学会用调试器，而不是靠打印**。断点、条件断点、`po` 与 `p` 命令、视图层级调试，这四样能解决大部分排查。把问题当侦探游戏：先复现，再缩小范围，最后才改代码。凭猜测改代码是在浪费生命。
10. **把做过的东西留下来**。每个练习项目都写一份简短 README，记录做了什么、卡在哪里、后来怎么解决。三个月后写简历和面试时，这些笔记就是现成素材，也是你进步速度的唯一客观证据。

> 新成员和成熟开发者的差距通常不在聪明程度，而在于遇到问题时是绕过去还是查到闭环。前者欠下的债会在项目里以 bug 的形式还回来。

第 7 章 附录

本附录按查阅场景组织：术语表用来对齐英文说法，FAQ 回答入门阶段的典型疑问，快捷键表放在手边提高效率，自查清单在开工和提交两个节点各过一遍。

附录 A 术语中英对照表

读英文文档、看开源代码、写提交信息时，最大的障碍常常不是语法而是术语。下表按类别分组，建议先通读一遍混个眼熟，之后遇到再回来查。

Swift 语言概念

中文	英文	一句话解释
可选值	Optional	可能没有值的类型，写作 `String?`，使用前必须先解包
强制解包	force unwrap	用感叹号直接取值，值为空时崩溃，只在该值必然存在时使用
可选绑定	optional binding	用 `if let` 或 `guard let` 安全取出可选值里的真实值
空合并运算符	nil-coalescing operator	写作 `??`，左边为空时返回右边的默认值
结构体	struct	值类型，赋值即复制，默认线程安全，优先于类使用
类	class	引用类型，多个变量指向同一份实例，支持继承
枚举	enum	一组互斥取值的集合，配合 `switch` 可做穷尽检查
闭包	closure	可以传递的代码块，能捕获定义处的上下文变量
协议	protocol	描述一组能力要求的契约，任何类型都可以选择遵守
扩展	extension	不修改原类型就为它添加方法、计算属性或协议实现
泛型	generic	让类型和函数在未知具体类型时也能编写，用占位类型参数表示

中文	英文	一句话解释
泛型约束	generic constraint	限定泛型参数必须满足某个协议，从而能调用该协议的方法
属性包装器	property wrapper	用 `@` 标注的类型，把读写逻辑抽成可复用的包装，如 `@State`
自动引用计数	ARC	编译器自动插入的引用计数内存管理机制，循环引用会导致泄漏
参与者	actor	用隔离保护可变状态的类型，同一时刻只允许一个任务访问它
可发送类型	Sendable	标记可以安全跨并发域传递的类型，编译器据此检查数据竞争

SwiftUI 概念

中文	英文	一句话解释
视图	View	描述界面的一段声明，通过组合而不是继承来构建
修饰符	modifier	挂在视图后面的链式方法，按书写顺序逐层包裹视图
状态	State	视图私有的可变数据，用 `@State` 声明，变化触发重绘
绑定	Binding	对别处状态的读写引用，让子视图能修改父视图的数据
可观察对象	Observable	用 `@Observable` 标注的引用类型模型，属性变化自动驱动视图更新
环境	Environment	沿视图树自上而下传递的共享值，用 `@Environment` 读取
场景	Scene	App 的顶层结构，描述窗口与生命周期，如 `WindowGroup`
预览	preview	用 `#Preview` 在画布中实时查看界面，不必启动模拟器
导航栈	NavigationStack	管理页面压栈与返回的现代导航容器

工程与发布概念

中文	英文	一句话解释
包标识符	bundle identifier	App 的唯一身份，形如反向域名，上架后不可随意更改
证书	certificate	证明开发者身份的密钥文件，签名时必须用到
描述文件	provisioning profile	把证书、设备与 App 标识绑在一起的授权文件
代码签名	code signing	用证书给产物签名的过程，不签名就无法装到真机
沙盒	sandbox	每个 App 独立的文件与权限空间，不能随意访问他人数据
权限声明	entitlement	描述 App 被授予哪些系统能力的配置项
能力	capability	在项目里开启某项系统功能，如推送、后台模式、小组件
部署目标	deployment target	允许运行的最低系统版本，决定你能用哪些 API
模拟器	simulator	Mac 上模拟 iOS 运行环境的工具，方便但不能替代真机
归档	archive	用于分发或上架的构建产物，含符号信息供崩溃分析
测试分发	TestFlight	苹果官方的内测渠道，可邀请外部测试者安装
应用商店连接	App Store Connect	管理 App 信息、版本、定价与审核的后台

数据与持久化概念

中文	英文	一句话解释
可编解码	Codable	让类型能被自动编码与解码的协议组合，JSON 解析的基石
JSON	JavaScript Object Notation	最常见的网络数据交换格式，键值对与数组的文本表示

中文	英文	一句话解释
编码键	CodingKeys	把模型属性名映射到 JSON 字段名的枚举
持久化	persistence	把数据保存到磁盘，使 App 重启后仍然可用
迁移	migration	数据模型变更时把旧数据升级到新结构的过程
模式	schema	描述数据有哪些实体、字段与关系的整体定义
模型上下文	model context	SwiftData 中负责增删改查与保存的入口对象
谓词	predicate	描述筛选条件的表达式，用于构造高效查询

并发概念

中文	英文	一句话解释
异步函数	async function	用 `async` 标注、可能中途挂起的函数
等待	await	在异步函数里等待另一个异步操作完成的挂起点
任务	Task	并发执行的工作单元，可以取消，也可以组合成任务组
主参与者	MainActor	代表主线程的隔离域，所有界面更新都应在这里进行
线程	thread	操作系统调度的执行流，主线程负责界面响应
数据竞争	race condition	多个执行流无序访问同一份可变数据，导致结果不可预测
死锁	deadlock	两个任务互相等待对方释放资源，导致双方都停住
任务组	task group	动态派发多个子任务并汇总结果的并发结构

附录 B 常见问题 FAQ

英语不好能不能学 iOS

能学，但天花板会被英语水平限制。入门知识用中文资料完全够用，一旦进入真实开发，报错信息、官方文档、审核回复几乎都是英文。务实的做法是边学边补：报错整句复制去搜，生词当场查，坚持两个月阅读速度就会明显改善。不要等英语学好了再开始，那一天不会到来。

要不要买 Mac

必须有能跑当前版本 Xcode 的 Mac，这是硬性要求，Windows 与 Linux 都无法构建 iOS App。预算有限就考虑二手 MacBook Air 或 Mac mini，内存至少 16GB，硬盘至少 512GB。学校机房或云 Mac 可以过渡，但日常练习最好有随时能用的设备。

只用模拟器够不够

入门够用，但真机不可替代。模拟器在传感器、摄像头、推送、后台运行、性能表现、内存压力上都与真机不同，某些第三方库甚至只在真机上正常工作。免费开发者账号就能给自己手机装 App，建议每个项目都真机跑一次。

要不要先学 Objective-C

不需要。它是上一代语言，新项目不会再用它写业务。你只要能看懂少量遗留写法，比如头文件声明、方括号调用、`@property`，在读老代码或老库文档时不卡住就够了。时间投给 Swift 的并发与 SwiftUI 收益高得多。

要不要先学 UIKit

主线不需要。SwiftUI 是当前与未来的方向，新项目基本都从它起步。你只需要知道 UIKit 是命令式的老框架，看到 `UIViewController`、`UITableView` 这类字样时不慌，必要时补一两节。真正需要深入 UIKit 的场景通常是维护老项目，那是入门之后的事。

SwiftUI 能不能找到工作

能，但看市场。国内不少存量项目仍以 UIKit 为主，只写 SwiftUI 会缩小一部分机会；但新项目、创业团队与外包里 SwiftUI 的比例在上升。稳妥策略是主线用 SwiftUI，同时具备读改 UIKit 代码的能力，这样两类岗位都能投。语言与工程能力比框架名称更重要。

CocoaPods 还要不要学

了解即可，不必深学。新项目优先用 Swift Package Manager，它集成在 Xcode 里，不需要额外工具链。CocoaPods 仍存在于大量老项目中，你至少要能看懂 `Podfile` 与 `Podfile.lock`，会执行安装更新命令，知道出问题去哪查。

Xcode 太占磁盘怎么办

Xcode 本体加模拟器运行环境和缓存，占用几十 GB 很正常。可以在设备管理里删掉不用的模拟器版本，清理 Derived Data 目录里的历史构建产物，删除旧的 iOS 支持文件，并在设置里关闭不必要的调试符号。定期清理通常能回收十 GB 以上。

学多久能找到实习

按每天两到三小时估算，走完三周速成计划就能做出可展示的作品，走完三个月进阶路线通常具备投递实习的技术条件。关键变量不是总时长而是作品质量：两个能真机运行、功能完整、讲得清技术取舍的项目，比十个半成品有用。越早投递越好，面试反馈本身就是最高效的学习指南。

报错了怎么办

把报错当题目而不是事故。先读完整条错误信息和它给出的文件与行号，很多错误的信息本身就是答案；再判断是编译期还是运行期，缩小到最小复现代码；接着去掉变量名拿关键句去搜，优先看 Stack Overflow 与官方论坛；最后用断点或打印验证你的假设。不要在不理解原因时改代码，那只是把问题推迟。

要不要背 API

不要背，要记住位置和形状。你需要形成的是「这个功能大概由哪个框架提供、方法名大概长什么样」的直觉，具体参数交给补全和文档。真正需要牢记的是语言概念、常见的坑与调试方法，这些搜索不出来。反复使用自然就记住了，用不到的 API 背了也会忘。

代码写在一个文件里行不行

练习阶段可以，超过 300 行就必须拆。拆分依据是职责而不是行数：视图、视图模型、数据模型、网络服务、工具函数各归其位。全部逻辑塞进一个文件会导致无法测试、无法复用、冲突频发，而且两天后你自己都找不到东西。从第一个项目就按职责分文件，成本最低。

要不要一开始就学架构

不要。架构是为了解决规模带来的问题，你还没有规模就没必要引入复杂度。起步只需三条纪律：界面代码不写业务规则、数据访问不散落在视图里、依赖通过参数传入而不是全局单例。等项目开始因为改一处而崩三处时再学 MVVM，那时你会真正理解它存在的理由。

免费开发者账号能做什么

免费账号可以真机调试、使用 Xcode 全部功能、在 App Store Connect 创建 App 记录并使用 TestFlight 内测，但不能提交正式上架，签名有效期也更短，需要定期重装。学习与做作品完全够用，准备上架时再购买开发者计划，个人账号每年付费。

怎么找练手项目

三个来源最可靠：复刻手机里最常用的 App 的一两个界面；把课程作业或社团需求做成小工具；在 GitHub 找一个结构清晰的开源 SwiftUI 项目，先复刻再改造。选题标准只有一条，你自己会用。自己不会用的东西，做到一半就会失去动力。

附录 C Xcode 常用快捷键

下表是 macOS 键位，`Cmd` 指 Command，`Ctrl` 指 Control，`Option` 指 Option。只列日常使用频率最高且默认键位稳定的操作。

快捷键	作用	什么时候用
Cmd+R	运行当前 Scheme	改完代码想看效果，最常用的一个
Cmd+.	停止运行	程序卡住、死循环或想重新开始时
Cmd+B	编译	只想确认能否编译通过，不想启动模拟器时
Cmd+U	运行全部测试	提交前跑一遍，确认没有回归
Cmd+Shift+K	清除构建缓存	出现莫名其妙的链接错误或改了资源没生效时
Cmd+Shift+O	快速打开文件或符号	知道名字但不知道文件在哪时
Cmd+Shift+F	在整个工程里搜索	找某段文案、某个方法被谁调用时
Cmd+Shift+J	在导航栏中定位当前文件	编辑到一半想看清自己在项目里的位置时
Cmd+0	显示或隐藏左侧导航栏	想给编辑器让出宽度看长代码时

快捷键	作用	什么时候用
Cmd+Option+0	显示或隐藏右侧检查器	调视图属性或查看文件信息时
Cmd+Shift+Y	显示或隐藏底部调试区	需要看控制台输出或变量时打开，平时关掉
Cmd+1	切到项目导航器	从搜索或问题列表回到文件树时
Cmd+5	切到问题导航器	报错很多，想集中看全部错误时
Cmd+/	注释或取消注释选中行	临时屏蔽代码排查问题时
Ctrl+I	重新缩进选中代码	粘贴进来的代码格式乱掉时
Cmd+Ctrl+左方向键	返回上一个编辑位置	跳转太多次后想原路退回时
Cmd+Ctrl+右方向键	前进到下一个编辑位置	退过头了再回来
Cmd+Option+左方向键	折叠当前代码块	文件很长，想把无关实现收起来时
Cmd+Option+右方向键	展开当前代码块	需要重新看被折叠的实现时
Cmd+Option+P	恢复或刷新预览	预览暂停、报错或改动没反映时

附录 D 新项目自查清单

两张清单对应两个最容易埋雷的时刻：开工前的疏漏会在后期付出十倍代价，提交前的疏漏会污染整个团队的历史记录。

新项目开工前

1. 项目名称、Bundle ID 与团队设置是否正确，Bundle ID 是否与已有 App 冲突，是否采用反向域名风格。
2. 部署目标是否与你打算支持的设备范围一致，是否确认过最低版本上可用的 API。
3. Swift 语言版本与并发检查级别是否确定，是否决定开启严格并发检查。
4. 目录结构是否按职责分好，视图、模型、服务、工具各自有明确位置。
5. 是否接入 SwiftLint 或同类工具，是否用局部忽略规则而不是整体关掉。
6. 依赖管理方式是否确定，新增依赖是否评估过维护状态与体积。
7. 是否从第一天就初始化 Git 仓库，是否提交了合适的 `.gitignore`，避免把用户数据与构建产物纳入版本控制。
8. 最小可用版本的功能范围是否想清楚，有没有把「顺便加一个」的需求写进第一期。
9. 是否需要联网、推送、定位、相册等能力，对应的 capability 与权限说明文案是否准备好。

10. 数据模型的大致形状是否确定，数据打算存放在沙盒内的哪个目录。

提交代码前

1. 代码能否在干净环境下完整编译通过，是否残留未使用的变量与导入。

2. 单元测试是否全部通过，新增逻辑是否补了对应测试。

3. 是否清理了调试用的 `print`、临时代码、被注释掉的整段旧实现和随手写的 `TODO`。

4. 是否存在硬编码的密钥、令牌、测试账号或内网地址，敏感配置是否改为从本地配置或环境读取。

5. 网络请求是否处理了超时、断网、服务端错误与空数据，界面是否有加载、失败与空状态。

6. 列表是否给每条数据提供了稳定唯一的 `id`，避免刷新时重复渲染或状态错乱。

7. 是否在深色与浅色模式下都检查过界面，是否支持动态字体且在最大字号下不截断。

8. 是否存在把用户数据写到沙盒外的路径，是否有未清理的大文件缓存。

9. 提交信息是否说明了改动目的，是否一次提交只做一件事。

10. 是否误提交了构建产物、用户数据或本地配置。

11. 改动涉及接口或数据模型时，是否更新了文档并提醒了相关同学。

12. 是否在自己机器上真机验证过一次，尤其是涉及权限、后台与传感器的功能。